In [27…

```
# ======================================================================
# CCR-Diffusion v3  --  improvements + complete ablation suite
# SynthRAD2023 Brain (centre A, 60 patients), 2D 128x128 PNGs, MRI -> sCT
#
# Paste each "# %%" block into its own notebook cell, or run as a script.
#
# WHAT IS NEW vs v2
#  Improvements (each is also an ablation row, so you can prove it helps)
#   I1  2.5D Stage-1 (neighbouring slices as extra input channels)
#   I2  Stage-1 retrained with a LEAKAGE-FREE recipe (fixed epochs + EMA,
#       checkpoint selection on the held-out fold) -> clean OOF residuals
#   I3  Raw MRI (all slices) fed to the denoiser as extra input channels
#   I4  Optional bone-weighted loss, optional x0-L1 (breaks the "no CT L1
#       so it is reported as a separate variant)
#   I5  Horizontal-flip test-time augmentation (TTA)
#   I6  Aux-SSIM target outside the MRI mask fixed to -1 (consistent with
#   I7  Per-variant least-squares residual weight w* (+ val-tuned gate bl
#
#  Ablations (val set only; test is touched ONCE at the end):
#   Stage-1: 2D vs 2.5D(+-1) vs 2.5D(+-2)
#   Stage-2: full | deterministic refiner (no diffusion) | no FiLM |
#            no coarse channel | no MRI channel | eps- vs v-prediction |
#            no aux SSIM | v1-style fixed (in-sample) conditioning |
#            bone-weighted loss | x0 L1 | Stage-2 on 2D Stage-1
#   Inference (no retraining): 1/2/4 samples, 1/2/4 folds, TTA on/off,
#            DDIM steps, EMA vs raw weights, blend variants
#   Statistics: per-patient paired differences, patient bootstrap CI, Wil
#
# Metrics: normalised PNG intensity (NOT HU). Masked = inside the MRI-der
# mask (primary). Global = full frame (secondary). Global SSIM = mean of
# full SSIM map (one definition everywhere).
# ======================================================================
```

In [28…

```
# %% ====== [0] CONFIG & IMPORTS ====================================
import os, glob, math, time, json, copy, random, shutil, zipfile
import numpy as np
import pandas as pd
import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.transforms.functional as TF
from torchvision.transforms import InterpolationMode
import matplotlib.pyplot as plt
from PIL import Image
from torch.utils.data import Dataset, DataLoader, Subset
from tqdm.auto import tqdm
from skimage.metrics import structural_similarity as compute_ssim
try:
    from scipy.stats import wilcoxon
except Exception:
    wilcoxon = None
```

```
PIPELINE_VERSION = "ccr_v3_ablation_1"
SEED = 42
IMAGE_SIZE, BATCH_SIZE = 128, 8
NUM_WORKERS = 0 if os.name == "nt" else 2
N_FOLDS = 4

# ---- Stage-1 ----
S1_BASE, S1_EPOCHS, S1_LR, S1_EMA = 32, 40, 1e-4, 0.998
S1_VARIANTS = {"2d": (0,), "25d1": (-1, 0, 1), "25d2": (-2, -1, 0, 1, 2)}
S1_TO_RUN = ["2d", "25d1", "25d2"]          # trim to save time (each = 4
REUSE_LEGACY_2D = False                      # True: load your old 2D fol
                                             # (slightly different recipe

# ---- Stage-2 ----
T_DIM, T_STEPS, Z_CLIP = 128, 1000, 8.0
S2_LR, S2_WD, S2_WARMUP, EMA_DECAY = 2e-4, 1e-4, 500, 0.999
SSIM_T_CUTOFF = 250
LOSS_BONE_THR = -0.15                        # intensity gate for the opt
VAL_FREQ = 10
QV = dict(folds=(1, 2), samples=1, steps=20)                        # quic
ABL_EPOCHS, FINAL_EPOCHS = 40, 120                                  # abla
ABL_PROTOCOL = dict(folds=(1, 2, 3, 4), samples=2, steps=25)        # eval
FINAL_PROTOCOL = dict(folds=(1, 2, 3, 4), samples=4, steps=50, flips=(Fal

BASE_CFG = dict(s1=None,             # Stage-1 variant tag (None -> best S
                mode="diff",         # "diff" | "det" (deterministic refin
                pred="v",            # "v" | "eps"
                film=True, coarse_ch=True, mri_ch=True,
                aux_ssim=0.05, x0_l1=0.0, bone_w=1.0,
                cond="oof",          # "oof" (fold-consistent) | "fixed" (
                base_ch=48, seed=SEED)

ABLATIONS = {                         # name -> overrides of BASE_CFG
    "A_full":        {},
    "A_det":         {"mode": "det"},
    "A_noFiLM":      {"film": False},
    "A_noCoarseCh":  {"coarse_ch": False},
    "A_noMRIch":     {"mri_ch": False},
    "A_epsPred":     {"pred": "eps"},
    "A_noAuxSSIM":   {"aux_ssim": 0.0},
    "A_fixedCond":   {"cond": "fixed"},
    "A_boneLoss":    {"bone_w": 3.0},
    "A_x0L1":        {"x0_l1": 0.1},
    "A_onS1_2d":     {"s1": "2d"},
}
ABLATIONS_TO_RUN = list(ABLATIONS)           # trim to save time
TEST_ABLATIONS = ("A_det",)                  # extra variants also scored
FINAL_NAME = None                            # None -> best val Score(w*)
EXTRA_SEEDS = ()                             # e.g. (1, 2) to retrain the

# HU window of the PNGs is an ASSUMPTION until verified against the NIfTI
HU_MIN, HU_MAX = -1000.0, 1500.0
HU_MAPPING_VERIFIED = False
HU_SPAN = HU_MAX - HU_MIN
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
use_amp = device.type == "cuda"


def seed_everything(seed=42):
    random.seed(seed); os.environ["PYTHONHASHSEED"] = str(seed)
    np.random.seed(seed); torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.benchmark = True


seed_everything(SEED)

print("ok")
```

```
ok
```

In [29…

```
# %% ====== [1] NUMPY-SIDE UTILITIES (metrics, aggregation, blending, sta
def slice_metrics(pred, gt, mask):
    """Normalised-intensity metrics. masked = inside mask, global = full
    p = (np.clip(pred, -1, 1) + 1) / 2
    g = (np.clip(gt, -1, 1) + 1) / 2
    body = mask > 0.5
    d = p - g
    mse = float(np.mean(d[body] ** 2))
    _, smap = compute_ssim(g, p, data_range=1.0, full=True)
    gmse = float(np.mean(d ** 2))
    return {"mae": float(np.mean(np.abs(d[body]))), "rmse": float(np.sqrt
            "ssim": float(np.mean(smap[body])), "psnr": float(10 * np.log
            "global_mae": float(np.mean(np.abs(d))), "global_ssim": float
            "global_psnr": float(10 * np.log10(1.0 / max(gmse, 1e-12)))}


def patient_level(pred, gt, mask, pids):
    """Mean per patient, then macro-average over patients."""
    recs = {}
    for i in range(len(pred)):
        recs.setdefault(pids[i], []).append(slice_metrics(pred[i], gt[i],
    df = pd.DataFrame([{"patient_id": p, **pd.DataFrame(r).mean().to_dict
    mu = df.drop(columns=["patient_id"]).mean()
    out = {"MAE": mu["mae"], "RMSE": mu["rmse"], "SSIM": mu["ssim"], "PSN
           "Global_SSIM": mu["global_ssim"], "Global_PSNR": mu["global_ps
           "Score": mu["mae"] + 0.25 * (1 - mu["ssim"])}
    out["HU_MAE_provisional"] = out["MAE"] * HU_SPAN
    return out, df


def finalize(coarse, res, mask, w=1.0):
    sct = np.clip(coarse + w * res, -1, 1)
    sct[mask < 0.5] = -1.0
    return sct
```

```
def blend(coarse, res, mask, w_soft, w_bone, thr):
    gate = np.stack([cv2.GaussianBlur((c > thr).astype(np.float32), (7, 7
    return finalize(coarse, (w_soft + (w_bone - w_soft) * gate) * res, ma


def wstar(coarse, res, gt, mask):
    """Least-squares optimal scalar on the predicted residual (masked pix
    m = mask > 0.5
    r_true, r_hat = (gt - coarse)[m], res[m]
    return float(np.clip((r_hat * r_true).sum() / max((r_hat ** 2).sum(),


def aggregate(stack, fidx=None, lidx=None, sidx=None):
    """Average a prediction stack over chosen folds / flips / samples."""
    nf, nl, ns = stack["res"].shape[:3]
    fidx = list(range(nf)) if fidx is None else list(fidx)
    lidx = list(range(nl)) if lidx is None else list(lidx)
    sidx = list(range(ns)) if sidx is None else list(sidx)
    coarse = stack["coarse"][np.ix_(fidx, lidx)].mean((0, 1))
    res = stack["res"][np.ix_(fidx, lidx, sidx)].mean((0, 1, 2))
    return coarse, res


def paired_stats(pred_a, pred_b, gt, mask, pids, n_boot=10000, seed=0):
    """b minus a, per patient. 'improved' = in the better direction for t
    _, da = patient_level(pred_a, gt, mask, pids)
    _, db = patient_level(pred_b, gt, mask, pids)
    da = da.set_index("patient_id")
    db = db.set_index("patient_id").loc[da.index]
    rng, rows = np.random.RandomState(seed), []
    for m, sign in (("mae", -1), ("rmse", -1), ("ssim", 1), ("psnr", 1)):
        d = (db[m] - da[m]).values
        boots = rng.choice(d, size=(n_boot, len(d)), replace=True).mean(1
        lo, hi = np.percentile(boots, [2.5, 97.5])
        p = float(wilcoxon(d).pvalue) if (wilcoxon is not None and np.any
        rows.append({"metric": m, "mean_diff(b-a)": d.mean(), "CI95_lo":
                     "wilcoxon_p": p, "patients_improved": int((sign * d
    return pd.DataFrame(rows)


def summarize_pred(name, coarse, res, gt, mask, pids):
    """Row with w=1 metrics and the val-optimal scalar w*."""
    m1, _ = patient_level(finalize(coarse, res, mask), gt, mask, pids)
    w = wstar(coarse, res, gt, mask)
    mw, _ = patient_level(finalize(coarse, res, mask, w), gt, mask, pids)
    return {"variant": name, "MAE": m1["MAE"], "SSIM": m1["SSIM"], "PSNR"
            "w*": w, "MAE(w*)": mw["MAE"], "SSIM(w*)": mw["SSIM"], "PSNR(
```

In [30…

```
# %% ====== [2] DATA: discovery, split, preprocessing, 2.5D stacking ====
BASE_OUT_DIR = "/kaggle/working/synthrad_ccr_v3" if os.path.exists("/kagg
DIR_S1, DIR_S2, DIR_METRICS, DIR_FIG = [os.path.join(BASE_OUT_DIR, d) for
for d in (DIR_S1, DIR_S2, DIR_METRICS, DIR_FIG):
    os.makedirs(d, exist_ok=True)
```

```
CANDIDATES = ["/kaggle/input/datasets/lohithkgowda/synthrad2023-brain-2d-
              "/kaggle/input/synthrad2023-brain-2d-png/synthrad_png", "./
DATASET_ROOT = next((p for p in CANDIDATES if os.path.isdir(os.path.join(
if DATASET_ROOT is None:
    for m in glob.glob("/kaggle/input/**/synthrad_png", recursive=True):
        if os.path.isdir(os.path.join(m, "ct")) and os.path.isdir(os.path
            DATASET_ROOT = m; break
assert DATASET_ROOT, "Dataset not found"
CT_DIR, MRI_DIR = os.path.join(DATASET_ROOT, "ct"), os.path.join(DATASET_
common = sorted(set(f for f in os.listdir(CT_DIR) if f.lower().endswith("
                set(f for f in os.listdir(MRI_DIR) if f.lower().endswith(

PATIENT_SORTED = {}
for f in common:
    PATIENT_SORTED.setdefault(f.split("_")[0], []).append(f)
for l in PATIENT_SORTED.values():
    l.sort()
POS = {s: (pid, i) for pid, l in PATIENT_SORTED.items() for i, s in enume

TRAIN_PIDS = sorted(["1BA001", "1BA005", "1BA012", "1BA014", "1BA022", "1
    "1BA075", "1BA082", "1BA085", "1BA091", "1BA100", "1BA103", "1BA105",
    "1BA131", "1BA143", "1BA151", "1BA158", "1BA159", "1BA164", "1BA175",
    "1BA185", "1BA189", "1BA201", "1BA206", "1BA220", "1BA222", "1BA227",
    "1BA247", "1BA253", "1BA260", "1BA266", "1BA278", "1BA288", "1BA292",
    "1BA305", "1BA325", "1BA328", "1BA336", "1BA345", "1BA358", "1BA359",
VAL_PIDS = sorted(["1BA076", "1BA111", "1BA125", "1BA141", "1BA300", "1BA
TEST_PIDS = sorted(["1BA054", "1BA097", "1BA172", "1BA234", "1BA256", "1B
assert len(TRAIN_PIDS) == 48 and len(VAL_PIDS) == 6 and len(TEST_PIDS) ==
assert set(TRAIN_PIDS).isdisjoint(VAL_PIDS) and set(TRAIN_PIDS).isdisjoin
assert all(p in PATIENT_SORTED for p in TRAIN_PIDS + VAL_PIDS + TEST_PIDS

FOLDS = {k + 1: TRAIN_PIDS[k * 12:(k + 1) * 12] for k in range(N_FOLDS)}
PID_TO_FOLD = {pid: k for k, pids in FOLDS.items() for pid in pids}


def snames_of(pids):
    return [s for pid in pids for s in PATIENT_SORTED[pid]]


TRAIN_SNAMES, VAL_SNAMES, TEST_SNAMES = snames_of(TRAIN_PIDS), snames_of(
print(f"slices  train {len(TRAIN_SNAMES)} | val {len(VAL_SNAMES)} | test


def extract_brain_body_mask(img_arr, threshold=15.0):
    binary = (img_arr > threshold).astype(np.uint8) * 255
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
    closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
    contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_A
    mask = np.zeros_like(img_arr, dtype=np.uint8)
    if contours:
        cv2.drawContours(mask, [max(contours, key=cv2.contourArea)], -1,
    else:
        mask = closed
    return (mask > 0).astype(np.float32)
```

```
def process_slice(sname):
    """-> (mri [-1,1] masked, ct [-1,1] full frame, mask) at IMAGE_SIZE.
    mri = np.array(Image.open(os.path.join(MRI_DIR, sname)).convert("L"),
    ct = np.array(Image.open(os.path.join(CT_DIR, sname)).convert("L"), d
    m = extract_brain_body_mask(mri)
    mri = mri * m
    sz = (IMAGE_SIZE, IMAGE_SIZE)
    mri = cv2.resize(mri, sz, interpolation=cv2.INTER_LINEAR)
    ct = cv2.resize(ct, sz, interpolation=cv2.INTER_LINEAR)
    m = cv2.resize(m, sz, interpolation=cv2.INTER_NEAREST)
    mri = (mri / 255.0) * 2.0 - 1.0
    ct = (ct / 255.0) * 2.0 - 1.0
    mri[m < 0.5] = -1.0
    return mri.astype(np.float32), ct.astype(np.float32), m.astype(np.flo


PROC = {s: process_slice(s) for s in tqdm(common, desc="Preprocessing all
SPLITS = {}


def get_split(key, snames, offsets):
    """Preloaded tensors; MRI is a stack of neighbouring slices (offsets)
    ck = (key, tuple(offsets))
    if ck in SPLITS:
        return SPLITS[ck]
    mri, ct, mk, folds, pids = [], [], [], [], []
    for s in snames:
        pid, pos = POS[s]; lst = PATIENT_SORTED[pid]
        mri.append(np.stack([PROC[lst[min(max(pos + o, 0), len(lst) - 1)]
        ct.append(PROC[s][1][None]); mk.append(PROC[s][2][None])
        folds.append(PID_TO_FOLD.get(pid, 0)); pids.append(pid)
    SPLITS[ck] = {"mri": torch.from_numpy(np.stack(mri)), "ct": torch.fro
                  "mask": torch.from_numpy(np.stack(mk)), "fold": torch.t
                  "pid": pids, "sname": list(snames)}
    return SPLITS[ck]


def augment(mri, ct, mask):
    if random.random() > 0.5:
        mri, ct, mask = TF.hflip(mri), TF.hflip(ct), TF.hflip(mask)
    kw = dict(angle=random.uniform(-6.0, 6.0), translate=[int(random.unif
              scale=random.uniform(0.97, 1.03), shear=0)
    mri = TF.affine(mri, **kw, interpolation=InterpolationMode.BILINEAR,
    ct = TF.affine(ct, **kw, interpolation=InterpolationMode.BILINEAR, fi
    mask = (TF.affine(mask, **kw, interpolation=InterpolationMode.NEAREST
    return mri, ct, mask


class SplitDataset(Dataset):
    def __init__(self, d, is_train=False):
        self.d, self.is_train = d, is_train
```

```
    def __len__(self):
        return self.d["mri"].size(0)

    def __getitem__(self, i):
        mri, ct, mask = self.d["mri"][i].clone(), self.d["ct"][i].clone()
        if self.is_train:
            mri, ct, mask = augment(mri, ct, mask)
        return {"mri": mri, "ct": ct, "mask": mask, "fold": int(self.d["f


def iter_batches(d, bs):
    n = d["mri"].size(0)
    for i in range(0, n, bs):
        sl = slice(i, min(i + bs, n))
        yield d["mri"][sl].to(device), d["ct"][sl].to(device), d["mask"][[
```

```
slices  train 2879 | val 372 | test 363
Preprocessing all slices:   0%|          | 0/3614 [00:00<?, ?it/s]
```

In [31…

```
# %% ====== [3] MODELS: Stage-1 ResUNet, SSIM loss, EMA, diffusion UNet,
class ResBlock(nn.Module):
    def __init__(self, cin, cout, num_groups=8):
        super().__init__()
        g = num_groups if cout % num_groups == 0 else (4 if cout % 4 == 0
        self.conv1 = nn.Conv2d(cin, cout, 3, padding=1, bias=False); self
        self.act1 = nn.LeakyReLU(0.2, inplace=True)
        self.conv2 = nn.Conv2d(cout, cout, 3, padding=1, bias=False); sel
        self.act2 = nn.LeakyReLU(0.2, inplace=True)
        self.shortcut = (nn.Sequential(nn.Conv2d(cin, cout, 1, bias=False
                         if cin != cout else nn.Identity())

    def forward(self, x):
        r = self.shortcut(x)
        h = self.act1(self.norm1(self.conv1(x)))
        h = self.norm2(self.conv2(h))
        return self.act2(h + r)


class Stage1ResUNet(nn.Module):
    """Same layout/keys as v1 (in_channels=1, base 32 loads legacy checkp

    def __init__(self, in_channels=1, out_channels=1, base_filters=32):
        super().__init__()
        f = base_filters
        self.enc1 = ResBlock(in_channels, f); self.pool1 = nn.MaxPool2d(2
        self.enc2 = ResBlock(f, f * 2);       self.pool2 = nn.MaxPool2d(2
        self.enc3 = ResBlock(f * 2, f * 4);   self.pool3 = nn.MaxPool2d(2
        self.enc4 = ResBlock(f * 4, f * 8);   self.pool4 = nn.MaxPool2d(2
        self.bottleneck = nn.Sequential(ResBlock(f * 8, f * 8), ResBlock(
        self.up4 = nn.ConvTranspose2d(f * 8, f * 4, 2, 2); self.dec4 = Re
        self.up3 = nn.ConvTranspose2d(f * 4, f * 2, 2, 2); self.dec3 = Re
        self.up2 = nn.ConvTranspose2d(f * 2, f, 2, 2);     self.dec2 = Re
        self.up1 = nn.ConvTranspose2d(f, f, 2, 2);         self.dec1 = Re
        self.final_conv = nn.Conv2d(f, out_channels, 1)
        self.final_act = nn.Tanh()
```

```
    def extract_encoder_features(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool1(e1))
        e3 = self.enc3(self.pool2(e2))
        e4 = self.enc4(self.pool3(e3))
        return {"enc1": e1, "enc2": e2, "enc3": e3, "enc4": e4, "bot": se

    def forward(self, x, return_feats=False):
        fe = self.extract_encoder_features(x)
        d4 = self.dec4(torch.cat([self.up4(fe["bot"]), fe["enc4"]], 1))
        d3 = self.dec3(torch.cat([self.up3(d4), fe["enc3"]], 1))
        d2 = self.dec2(torch.cat([self.up2(d3), fe["enc2"]], 1))
        d1 = self.dec1(torch.cat([self.up1(d2), fe["enc1"]], 1))
        out = self.final_act(self.final_conv(d1))
        return (out, fe) if return_feats else out


class DifferentiableSSIM(nn.Module):
    def __init__(self, window_size=11, sigma=1.5, data_range=2.0):
        super().__init__()
        self.ws, self.dr = window_size, data_range
        c = torch.arange(window_size, dtype=torch.float32) - window_size
        g = torch.exp(-(c ** 2) / (2 * sigma ** 2)); g = g / g.sum()
        self.register_buffer("window", torch.outer(g, g)[None, None])

    def forward(self, a, b, mask=None):
        w, p = self.window.to(a.dtype), self.ws // 2
        mu1, mu2 = F.conv2d(a, w, padding=p), F.conv2d(b, w, padding=p)
        s1 = F.conv2d(a * a, w, padding=p) - mu1 ** 2
        s2 = F.conv2d(b * b, w, padding=p) - mu2 ** 2
        s12 = F.conv2d(a * b, w, padding=p) - mu1 * mu2
        c1, c2 = (0.01 * self.dr) ** 2, (0.03 * self.dr) ** 2
        m = ((2 * mu1 * mu2 + c1) * (2 * s12 + c2)) / ((mu1 ** 2 + mu2 **
        if mask is not None:
            return ((1 - m) * mask).sum() / (mask.sum() + 1e-8)
        return 1 - m.mean()


class EMA:
    def __init__(self, model, decay=0.999):
        self.decay, self.n = decay, 0
        self.shadow = copy.deepcopy(model).eval()
        for p in self.shadow.parameters():
            p.requires_grad_(False)

    @torch.no_grad()
    def update(self, model):
        self.n += 1
        d = min(self.decay, (1 + self.n) / (10 + self.n))
        for ps, p in zip(self.shadow.parameters(), model.parameters()):
            ps.mul_(d).add_(p.detach(), alpha=1 - d)
```

```python
class SinusoidalEmb(nn.Module):
    def __init__(self, dim):
        super().__init__(); self.dim = dim

    def forward(self, t):
        half = self.dim // 2
        f = torch.exp(torch.arange(half, device=t.device) * -(math.log(10
        a = t.float()[:, None] * f[None, :]
        return torch.cat([a.sin(), a.cos()], -1)


def _gn(ch):
    return nn.GroupNorm(8 if ch % 8 == 0 else 4, ch)


class FiLM(nn.Module):
    """(1+gamma)*x + beta from Stage-1 encoder features; zero-init => ide

    def __init__(self, ch, cond_ch):
        super().__init__()
        self.proj = nn.Sequential(nn.SiLU(), nn.Conv2d(cond_ch, ch * 2, 1
        nn.init.zeros_(self.proj[-1].weight); nn.init.zeros_(self.proj[-1

    def forward(self, x, cond):
        if cond.shape[2:] != x.shape[2:]:
            cond = F.interpolate(cond, size=x.shape[2:], mode="bilinear",
        gamma, beta = self.proj(cond).chunk(2, dim=1)
        return (1 + gamma) * x + beta


class DResBlock(nn.Module):
    def __init__(self, cin, cout, t_dim, cond_ch=None):
        super().__init__()
        self.t_proj = nn.Linear(t_dim, cout)
        self.conv1 = nn.Conv2d(cin, cout, 3, padding=1); self.gn1 = _gn(c
        self.conv2 = nn.Conv2d(cout, cout, 3, padding=1); self.gn2 = _gn(
        self.act = nn.SiLU()
        self.skip = nn.Conv2d(cin, cout, 1) if cin != cout else nn.Identi
        self.film = FiLM(cout, cond_ch) if cond_ch is not None else None

    def forward(self, x, t_emb, cond=None):
        h = self.act(self.gn1(self.conv1(x)))
        h = h + self.t_proj(t_emb)[:, :, None, None]
        h = self.act(self.gn2(self.conv2(h)))
        out = h + self.skip(x)
        if self.film is not None and cond is not None:
            out = self.film(out, cond)
        return out


class DiffUNet(nn.Module):
    """Denoiser / deterministic refiner. in_ch = z(1) + mask(1) + [coarse

    def __init__(self, in_ch, base=48, t_dim=128, cond_channels=(32, 64,
```

```
        super().__init__()
        c, td = base, t_dim
        cc = tuple(cond_channels) if film else (None, None, None, None)
        self.time_mlp = nn.Sequential(SinusoidalEmb(td), nn.Linear(td, td
                                      nn.Linear(td * 2, td), nn.SiLU())
        self.enc1 = DResBlock(in_ch, c, td);   self.down1 = nn.Conv2d(c,
        self.enc2 = DResBlock(c, c * 2, td);   self.down2 = nn.Conv2d(c *
        self.enc3 = DResBlock(c * 2, c * 4, td); self.down3 = nn.Conv2d(c
        self.enc4 = DResBlock(c * 4, c * 8, td); self.down4 = nn.Conv2d(c
        self.bot1 = DResBlock(c * 8, c * 8, td); self.bot2 = DResBlock(c
        self.up4 = nn.ConvTranspose2d(c * 8, c * 8, 2, 2); self.dec4 = DR
        self.up3 = nn.ConvTranspose2d(c * 4, c * 4, 2, 2); self.dec3 = DR
        self.up2 = nn.ConvTranspose2d(c * 2, c * 2, 2, 2); self.dec2 = DR
        self.up1 = nn.ConvTranspose2d(c, c, 2, 2);         self.dec1 = DR
        self.final_conv = nn.Conv2d(c, 1, 1)

    def forward(self, x, t, cond):
        te = self.time_mlp(t)
        e1 = self.enc1(x, te);               e2 = self.enc2(self.down1(e1
        e3 = self.enc3(self.down2(e2), te);  e4 = self.enc4(self.down3(e3
        b = self.bot2(self.bot1(self.down4(e4), te), te)
        g = (lambda k: cond[k]) if cond is not None else (lambda k: None)
        d = self.dec4(torch.cat([self.up4(b), e4], 1), te, g("enc4"))
        d = self.dec3(torch.cat([self.up3(d), e3], 1), te, g("enc3"))
        d = self.dec2(torch.cat([self.up2(d), e2], 1), te, g("enc2"))
        d = self.dec1(torch.cat([self.up1(d), e1], 1), te, g("enc1"))
        return self.final_conv(d)


class CosineScheduler:
    def __init__(self, T=1000, s=0.008, dev="cpu"):
        t = torch.linspace(0, T, T + 1, dtype=torch.float64)
        f = torch.cos(((t / T) + s) / (1 + s) * math.pi / 2) ** 2
        ab = f / f[0]
        betas = (1 - ab[1:] / ab[:-1]).clamp(max=0.999)
        self.alphas_cumprod = torch.cumprod(1 - betas, 0).float().to(dev)
        self.T = T

    def ab(self, t):
        a = self.alphas_cumprod[t].view(-1, 1, 1, 1)
        return a.sqrt(), (1 - a).sqrt()

    def q_sample(self, z0, t, noise):
        a, s = self.ab(t); return a * z0 + s * noise

    def v_target(self, z0, noise, t):
        a, s = self.ab(t); return a * noise - s * z0


sched = CosineScheduler(T_STEPS, dev=device)


def out_to_z0(out, z_t, t, pred):
    a, s = sched.ab(t)
```

```
    return a * z_t - s * out if pred == "v" else (z_t - s * out) / (a + 1


def build_input(cfg, z, mask, coarse, mri):
    ch = [z, mask]
    if cfg["coarse_ch"]:
        ch.append(coarse)
    if cfg["mri_ch"]:
        ch.append(mri)
    return torch.cat(ch, 1)
```

In [32…

```
# %% ====== [4] STAGE-1: leakage-free fold training, bundle (coarse + fea
def _l1(pred, tgt, mask):
    return (torch.abs(pred - tgt) * mask).sum() / (mask.sum() + 1e-8)


def _find_legacy(k):
    for root in ("/kaggle/working/synthrad_ccr_diffusion/checkpoints/coar
        hits = glob.glob(os.path.join(root, "**", f"coarse_resunet_fold_{
        if hits:
            return sorted(hits, key=len)[0]
    return None


def train_stage1_fold(tag, offsets, k):
    path = os.path.join(DIR_S1, f"s1_{tag}_fold{k}.pth")
    model = Stage1ResUNet(len(offsets), 1, S1_BASE).to(device)
    if os.path.exists(path):
        model.load_state_dict(torch.load(path, map_location=device, weigh
        return model.eval()
    if REUSE_LEGACY_2D and tag == "2d" and _find_legacy(k):
        model.load_state_dict(torch.load(_find_legacy(k), map_location=de
        print(f"[legacy] 2d fold {k} loaded from {_find_legacy(k)}")
        return model.eval()
    seed_everything(SEED + k)
    d = get_split("train", TRAIN_SNAMES, offsets)
    tr_idx = (d["fold"] != k).nonzero().squeeze(1).tolist()
    loader = DataLoader(Subset(SplitDataset(d, True), tr_idx), batch_size
                        num_workers=NUM_WORKERS, drop_last=True)
    opt = torch.optim.AdamW(model.parameters(), lr=S1_LR, weight_decay=1e
    sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=S1_EPOCHS
    ssim = DifferentiableSSIM().to(device)
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
    ema = EMA(model, S1_EMA)
    for ep in range(1, S1_EPOCHS + 1):
        model.train(); run = 0.0
        for b in loader:
            mri, ct, mask = b["mri"].to(device), b["ct"].to(device), b["m
            opt.zero_grad(set_to_none=True)
            with torch.amp.autocast("cuda", enabled=use_amp):
                pred = model(mri)
            pred = pred.float()
            ct_t = torch.where(mask > 0.5, ct, torch.full_like(ct, -1.0))
            pred_m = torch.where(mask > 0.5, pred, torch.full_like(pred,
```

```
            loss = _l1(pred, ct, mask) + 0.10 * ssim(pred_m, ct_t, mask=m
            scaler.scale(loss).backward(); scaler.unscale_(opt)
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(opt); scaler.update(); ema.update(model); run +=
        sch.step()
        if ep % 10 == 0 or ep == S1_EPOCHS:
            print(f"  [{tag}] fold {k} ep {ep}/{S1_EPOCHS} train loss {ru
    final = ema.shadow                                    # fixed schedule
    ho_idx = (d["fold"] == k).nonzero().squeeze(1)
    with torch.no_grad():                                 # report held-ou
        v = []
        for i in range(0, len(ho_idx), 32):
            ii = ho_idx[i:i + 32]
            mri, ct, mask = d["mri"][ii].to(device), d["ct"][ii].to(devic
            v.append(_l1(final(mri), ct, mask).item())
    print(f"  [{tag}] fold {k} held-out masked L1 (info only): {np.mean(v
    torch.save({"model_state_dict": final.state_dict(), "tag": tag, "fold
                "provenance": {"oof_pids": FOLDS[k], "train_pids": [p for
                               "selection": "none (fixed epochs, final EM
    return final.eval()


class Stage1Bundle:
    def __init__(self, tag):
        self.tag, self.offsets = tag, S1_VARIANTS[tag]
        self.models = {}
        for k in range(1, N_FOLDS + 1):
            m = train_stage1_fold(tag, self.offsets, k).eval()
            for p in m.parameters():
                p.requires_grad_(False)
            self.models[k] = m
        self.cond_ch = tuple(S1_BASE * x for x in (1, 2, 4, 8))
        self.S = self._residual_scale()

    @torch.no_grad()
    def _residual_scale(self):
        f = os.path.join(DIR_METRICS, f"residual_scale_{self.tag}.json")
        if os.path.exists(f):
            return json.load(open(f))["S"]
        d = get_split("train", TRAIN_SNAMES, self.offsets)
        vals = []
        for k in range(1, N_FOLDS + 1):
            idx = (d["fold"] == k).nonzero().squeeze(1)
            for i in range(0, len(idx), 32):
                ii = idx[i:i + 32]
                mri, ct, mask = d["mri"][ii].to(device), d["ct"][ii].to(d
                vals.append((ct - self.models[k](mri))[mask > 0.5].cpu())
        S = float(torch.cat(vals).std(unbiased=False).clamp_min(1e-6))
        json.dump({"S": S}, open(f, "w"))
        return S

    @torch.no_grad()
    def coarse_feats_k(self, mri, mask, k):
        c, f = self.models[k](mri, return_feats=True)
```

```
        return torch.where(mask > 0.5, c, torch.full_like(c, -1.0)), f

    @torch.no_grad()
    def coarse_feats(self, mri, mask, folds):
        """Per-sample: coarse + features from the fold model that HELD TH
        B, coarse, feats = mri.size(0), torch.empty_like(mri[:, :1]), Non
        for k in folds.unique().tolist():
            idx = (folds == k).nonzero(as_tuple=True)[0]
            c_k, f_k = self.models[k](mri[idx], return_feats=True)
            if feats is None:
                feats = {n: torch.empty((B,) + f.shape[1:], device=mri.de
            coarse[idx] = c_k
            for n in f_k:
                feats[n][idx] = f_k[n]
        return torch.where(mask > 0.5, coarse, torch.full_like(coarse, -1


BUNDLES = {}


def get_bundle(tag):
    if tag not in BUNDLES:
        BUNDLES[tag] = Stage1Bundle(tag)
    return BUNDLES[tag]


@torch.no_grad()
def stage1_ensemble(bundle, d, bs=32):
    out = []
    for mri, ct, mask, sl in iter_batches(d, bs):
        out.append(torch.stack([bundle.coarse_feats_k(mri, mask, k)[0] fo
    return np.concatenate(out)
```

In [33…

```
# %% ====== [5] STAGE-2: sampler, prediction stack, training ============
@torch.no_grad()
def sample_residual(model, cfg, S, mri, coarse, feats, mask, steps=50, ge
    """Residual in intensity units (= s * z0). 'det' = one forward pass;
    model.eval()
    b, dev = coarse.size(0), coarse.device
    if cfg["mode"] == "det":
        t = torch.zeros(b, dtype=torch.long, device=dev)
        with torch.amp.autocast("cuda", enabled=use_amp):
            out = model(build_input(cfg, torch.zeros_like(coarse), mask,
        return out.float() * S * mask
    z = torch.randn(coarse.shape, device=dev, generator=generator) * mask
    ts = sorted(set(torch.linspace(0, sched.T - 1, steps).long().tolist()
    ab = sched.alphas_cumprod
    for i, tv in enumerate(ts):
        t = torch.full((b,), tv, device=dev, dtype=torch.long)
        with torch.amp.autocast("cuda", enabled=use_amp):
            out = model(build_input(cfg, z, mask, coarse, mri), t, feats)
        out = out.float()
        a = ab[tv]
        a_prev = ab[ts[i + 1]] if i + 1 < len(ts) else torch.ones((), dev
```

```
        if cfg["pred"] == "v":
            z0 = a.sqrt() * z - (1 - a).sqrt() * out
            eps = (1 - a).sqrt() * z + a.sqrt() * out
        else:
            eps = out
            z0 = (z - (1 - a).sqrt() * eps) / (a.sqrt() + 1e-8)
        z0 = z0.clamp(-Z_CLIP, Z_CLIP) * mask
        z = (a_prev.sqrt() * z0 + (1 - a_prev).sqrt() * eps * mask) * mas
    return z * S * mask


@torch.no_grad()
def predict_stack(d, model, cfg, bundle, n_samples, steps, folds=(1, 2, 3
    """coarse[F,L,N,H,W] and res[F,L,S,N,H,W] for every (fold, flip, samp
    N, H = d["mri"].size(0), IMAGE_SIZE
    ns = n_samples if cfg["mode"] == "diff" else 1
    coarse = np.zeros((len(folds), len(flips), N, H, H), np.float32)
    res = np.zeros((len(folds), len(flips), ns, N, H, H), np.float32)
    g = torch.Generator(device=device).manual_seed(seed)
    for mri, ct, mask, sl in iter_batches(d, bs):
        for fi, k in enumerate(folds):
            for li, fl in enumerate(flips):
                m_in, mk = (mri.flip(-1), mask.flip(-1)) if fl else (mri,
                c, f = bundle.coarse_feats_k(m_in, mk, k)
                coarse[fi, li, sl] = (c.flip(-1) if fl else c)[:, 0].cpu(
                for si in range(ns):
                    r = sample_residual(model, cfg, bundle.S, m_in, c, f,
                    res[fi, li, si, sl] = (r.flip(-1) if fl else r)[:, 0]
    return {"coarse": coarse, "res": res}


def quick_validate(model, cfg, bundle, d_val):
    st = predict_stack(d_val, model, cfg, bundle, QV["samples"], QV["step
    c, r = aggregate(st)
    m, _ = patient_level(finalize(c, r, d_val["mask"][:, 0].numpy()), d_v
                         d_val["mask"][:, 0].numpy(), d_val["pid"])
    return m


def make_sig(cfg, epochs):
    return {"cfg": cfg, "S": round(get_bundle(cfg["s1"]).S, 8), "epochs":


def _paths(name, epochs):
    b = os.path.join(DIR_S2, f"s2_{name}_e{epochs}")
    return b + "_latest.pth", b + "_best.pth"


def train_stage2(name, cfg, epochs):
    bundle = get_bundle(cfg["s1"])
    d_tr, d_va = get_split("train", TRAIN_SNAMES, bundle.offsets), get_sp
    loader = DataLoader(SplitDataset(d_tr, True), batch_size=BATCH_SIZE,
                        num_workers=NUM_WORKERS, drop_last=True, pin_memo
    seed_everything(cfg["seed"])
```

```
    in_ch = 2 + int(cfg["coarse_ch"]) + (len(bundle.offsets) if cfg["mri_
    model = DiffUNet(in_ch, cfg["base_ch"], T_DIM, bundle.cond_ch, film=c
    ema = EMA(model, EMA_DECAY)
    opt = torch.optim.AdamW(model.parameters(), lr=S2_LR, weight_decay=S2
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
    ssim_crit = DifferentiableSSIM().to(device)
    latest, best = _paths(name, epochs)
    sig = make_sig(cfg, epochs)
    start, step, best_score, hist = 0, 0, float("inf"), []
    if os.path.exists(latest):
        ck = torch.load(latest, map_location=device, weights_only=False)
        if ck.get("sig") == sig:
            model.load_state_dict(ck["model"]); ema.shadow.load_state_dic
            opt.load_state_dict(ck["opt"]); scaler.load_state_dict(ck["sc
            start, best_score, hist = ck["epoch"], ck["best_score"], ck["
            print(f"[resume] {name} from epoch {start}")
        else:
            shutil.move(latest, latest.replace("_latest", f"_stale{int(ti
    total = epochs * len(loader)

    def lr_at(s):
        return (1e-6 + (S2_LR - 1e-6) * 0.5 * (1 + math.cos(math.pi * min

    t0 = time.time()
    for epoch in range(start + 1, epochs + 1):
        model.train(); acc, n = np.zeros(4), 0
        for b in tqdm(loader, desc=f"{name} {epoch}/{epochs}", leave=Fals
            mri, ct, mask = b["mri"].to(device), b["ct"].to(device), b["m
            folds = b["fold"].to(device)
            if cfg["cond"] == "fixed":
                folds = torch.ones_like(folds)
            coarse, feats = bundle.coarse_feats(mri, mask, folds)
            S = bundle.S
            z0 = ((ct - coarse) / S) * mask
            bs_ = z0.size(0)
            if cfg["mode"] == "diff":
                t = torch.randint(0, T_STEPS, (bs_,), device=device)
                noise = torch.randn_like(z0) * mask
                z_in = sched.q_sample(z0, t, noise) * mask
                tgt = sched.v_target(z0, noise, t) if cfg["pred"] == "v"
            else:
                t = torch.zeros(bs_, dtype=torch.long, device=device)
                z_in, tgt = torch.zeros_like(z0), z0
            for g_ in opt.param_groups:
                g_["lr"] = lr_at(step)
            opt.zero_grad(set_to_none=True)
            with torch.amp.autocast("cuda", enabled=use_amp):
                out = model(build_input(cfg, z_in, mask, coarse, mri), t,
            out = out.float()
            if cfg["bone_w"] != 1.0:
                gate = F.avg_pool2d((coarse > LOSS_BONE_THR).float(), 7,
                W = (1 + (cfg["bone_w"] - 1) * gate) * mask
            else:
                W = mask
```

```
            err = torch.abs(out - tgt) if cfg["mode"] == "det" else (out
            l_main = ((err * W).sum((1, 2, 3)) / (W.sum((1, 2, 3)) + 1e-8
            if cfg["mode"] == "diff":
                sel = t <= SSIM_T_CUTOFF
                z0_hat = out_to_z0(out, z_in, t, cfg["pred"]) if sel.any(
                z0_hat = z0_hat[sel] if sel.any() else None
            else:
                sel, z0_hat = torch.ones(bs_, dtype=torch.bool, device=de
            l_ssim = l_x0 = torch.zeros((), device=device)
            if sel.any() and (cfg["aux_ssim"] > 0 or cfg["x0_l1"] > 0):
                ct_hat = (coarse[sel] + z0_hat * mask[sel] * S).clamp(-1,
                ct_t = torch.where(mask[sel] > 0.5, ct[sel], torch.full_l
                if cfg["aux_ssim"] > 0:
                    l_ssim = ssim_crit(ct_hat, ct_t, mask=mask[sel])
                if cfg["x0_l1"] > 0:
                    l_x0 = _l1(ct_hat, ct_t, mask[sel])
            loss = l_main + cfg["aux_ssim"] * l_ssim + cfg["x0_l1"] * l_x
            scaler.scale(loss).backward(); scaler.unscale_(opt)
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(opt); scaler.update(); ema.update(model); step +=
            acc += [loss.item(), l_main.item(), l_ssim.item(), l_x0.item(
        acc /= n
        rec = {"epoch": epoch, "loss": acc[0], "main": acc[1], "aux_ssim"
        if epoch % VAL_FREQ == 0 or epoch == epochs:
            m = quick_validate(ema.shadow, cfg, bundle, d_va)
            rec.update({f"val_{k}": v for k, v in m.items()})
            star = ""
            if m["Score"] < best_score:
                best_score, star = m["Score"], "  <-- best"
                torch.save({"ema": ema.shadow.state_dict(), "epoch": epoc
            print(f"[{name}] ep {epoch:3d} loss {acc[0]:.4f} | EMA val Sc
                  f"SSIM {m['SSIM']:.4f} PSNR {m['PSNR']:.2f}{star}  ({(t
        hist.append(rec)
        torch.save({"epoch": epoch, "model": model.state_dict(), "ema": e
                    "opt": opt.state_dict(), "scaler": scaler.state_dict(
                    "history": hist, "sig": sig}, latest)
    pd.DataFrame(hist).to_csv(os.path.join(DIR_METRICS, f"history_{name}_


def get_stage2(name, cfg, epochs, which="best"):
    """Train if needed, then load. which: 'best' (val-selected EMA) | 'fi
    latest, best = _paths(name, epochs)
    done = False
    if os.path.exists(latest):
        ck = torch.load(latest, map_location=device, weights_only=False)
        done = ck["epoch"] >= epochs and os.path.exists(best) and ck.get(
    if not done:
        train_stage2(name, cfg, epochs)
    bundle = get_bundle(cfg["s1"])
    in_ch = 2 + int(cfg["coarse_ch"]) + (len(bundle.offsets) if cfg["mri_
    model = DiffUNet(in_ch, cfg["base_ch"], T_DIM, bundle.cond_ch, film=c
    if which == "best":
        model.load_state_dict(torch.load(best, map_location=device, weigh
    else:
```

```
        ck = torch.load(latest, map_location=device, weights_only=False)
        model.load_state_dict(ck["ema"] if which == "final_ema" else ck["
    return model.eval()


def make_cfg(overrides, main_s1):
    c = dict(BASE_CFG); c.update(overrides)
    if c["s1"] is None:
        c["s1"] = main_s1
    return c
```

In [13…

```
# %% ====== [6] STAGE-1 ABLATION (2D vs 2.5D) + choose the Stage-1 varian
VALd0 = get_split("val", VAL_SNAMES, (0,))
v_gt, v_mask, v_pid = VALd0["ct"][:, 0].numpy(), VALd0["mask"][:, 0].nump
s1_rows, S1_VAL_PRED = [], {}
for tag in S1_TO_RUN:
    b = get_bundle(tag)
    dv = get_split("val", VAL_SNAMES, b.offsets)
    pred = finalize(stage1_ensemble(b, dv), 0, v_mask)
    S1_VAL_PRED[tag] = pred
    m, _ = patient_level(pred, v_gt, v_mask, v_pid)
    s1_rows.append({"stage1": tag, "offsets": str(b.offsets), **m})
s1_table = pd.DataFrame(s1_rows); print(s1_table.round(5).to_string(index
s1_table.to_csv(os.path.join(DIR_METRICS, "ablation_stage1_val.csv"), ind
MAIN_S1 = s1_table.sort_values("Score").iloc[0]["stage1"]
print("Selected Stage-1 variant (lowest val Score):", MAIN_S1)
```

```
[2d] fold 1 ep 10/40 train loss 0.0744
[2d] fold 1 ep 20/40 train loss 0.0608
[2d] fold 1 ep 30/40 train loss 0.0526
[2d] fold 1 ep 40/40 train loss 0.0502
[2d] fold 1 held-out masked L1 (info only): 0.0693
[2d] fold 2 ep 10/40 train loss 0.0715
[2d] fold 2 ep 20/40 train loss 0.0594
[2d] fold 2 ep 30/40 train loss 0.0518
[2d] fold 2 ep 40/40 train loss 0.0490
[2d] fold 2 held-out masked L1 (info only): 0.0735
[2d] fold 3 ep 10/40 train loss 0.0713
[2d] fold 3 ep 20/40 train loss 0.0590
[2d] fold 3 ep 30/40 train loss 0.0513
[2d] fold 3 ep 40/40 train loss 0.0482
[2d] fold 3 held-out masked L1 (info only): 0.0675
[2d] fold 4 ep 10/40 train loss 0.0746
[2d] fold 4 ep 20/40 train loss 0.0608
[2d] fold 4 ep 30/40 train loss 0.0532
[2d] fold 4 ep 40/40 train loss 0.0507
[2d] fold 4 held-out masked L1 (info only): 0.0636
[25d1] fold 1 ep 10/40 train loss 0.0726
[25d1] fold 1 ep 20/40 train loss 0.0591
[25d1] fold 1 ep 30/40 train loss 0.0512
[25d1] fold 1 ep 40/40 train loss 0.0488
[25d1] fold 1 held-out masked L1 (info only): 0.0673
[25d1] fold 2 ep 10/40 train loss 0.0744
[25d1] fold 2 ep 20/40 train loss 0.0611
[25d1] fold 2 ep 30/40 train loss 0.0527
[25d1] fold 2 ep 40/40 train loss 0.0494
[25d1] fold 2 held-out masked L1 (info only): 0.0729
[25d1] fold 3 ep 10/40 train loss 0.0699
[25d1] fold 3 ep 20/40 train loss 0.0575
[25d1] fold 3 ep 30/40 train loss 0.0499
[25d1] fold 3 ep 40/40 train loss 0.0472
[25d1] fold 3 held-out masked L1 (info only): 0.0656
[25d1] fold 4 ep 10/40 train loss 0.0738
[25d1] fold 4 ep 20/40 train loss 0.0599
[25d1] fold 4 ep 30/40 train loss 0.0524
[25d1] fold 4 ep 40/40 train loss 0.0500
[25d1] fold 4 held-out masked L1 (info only): 0.0637
[25d2] fold 1 ep 10/40 train loss 0.0717
[25d2] fold 1 ep 20/40 train loss 0.0584
[25d2] fold 1 ep 30/40 train loss 0.0505
[25d2] fold 1 ep 40/40 train loss 0.0484
[25d2] fold 1 held-out masked L1 (info only): 0.0677
[25d2] fold 2 ep 10/40 train loss 0.0693
[25d2] fold 2 ep 20/40 train loss 0.0562
[25d2] fold 2 ep 30/40 train loss 0.0492
[25d2] fold 2 ep 40/40 train loss 0.0466
[25d2] fold 2 held-out masked L1 (info only): 0.0720
[25d2] fold 3 ep 10/40 train loss 0.0688
[25d2] fold 3 ep 20/40 train loss 0.0567
[25d2] fold 3 ep 30/40 train loss 0.0494
[25d2] fold 3 ep 40/40 train loss 0.0466
[25d2] fold 3 held-out masked L1 (info only): 0.0654
```

```
  [25d2] fold 4 ep 10/40 train loss 0.0719
  [25d2] fold 4 ep 20/40 train loss 0.0579
  [25d2] fold 4 ep 30/40 train loss 0.0506
  [25d2] fold 4 ep 40/40 train loss 0.0482
  [25d2] fold 4 held-out masked L1 (info only): 0.0630
stage1           offsets     MAE    RMSE    SSIM     PSNR  Global_SSIM  G
lobal_PSNR   Score  HU_MAE_provisional
    2d              (0,) 0.03665 0.06402 0.85804 24.14118      0.82192
22.79917 0.07214            91.62905
  25d1        (-1, 0, 1) 0.03658 0.06341 0.86026 24.21899      0.82301
22.82838 0.07151            91.44021
  25d2 (-2, -1, 0, 1, 2) 0.03629 0.06272 0.86205 24.32174      0.82388
22.85376 0.07078            90.73395
Selected Stage-1 variant (lowest val Score): 25d2
```

In [14…

```
# %% ====== [7] STAGE-2 ABLATION (validation only) ======================
ABL_CSV = os.path.join(DIR_METRICS, f"ablation_stage2_val_e{ABL_EPOCHS}.c
abl_done = pd.read_csv(ABL_CSV).to_dict("records") if os.path.exists(ABL_
done_names = {r["variant"] for r in abl_done}
for name in ABLATIONS_TO_RUN:
    if name in done_names:
        continue
    cfg = make_cfg(ABLATIONS[name], MAIN_S1)
    model = get_stage2(name, cfg, ABL_EPOCHS)
    b = get_bundle(cfg["s1"]); dv = get_split("val", VAL_SNAMES, b.offset
    folds_ = (1,) if cfg["cond"] == "fixed" else ABL_PROTOCOL["folds"]
    st = predict_stack(dv, model, cfg, b, ABL_PROTOCOL["samples"], ABL_PR
    c, r = aggregate(st)
    row = summarize_pred(name, c, r, v_gt, v_mask, v_pid)
    row["stage1_used"] = cfg["s1"]
    row["Stage1_only_Score"] = patient_level(finalize(c, 0 * r, v_mask),
    abl_done.append(row)
    pd.DataFrame(abl_done).to_csv(ABL_CSV, index=False)
    del model; torch.cuda.empty_cache()
abl_table = pd.DataFrame(abl_done)
print(abl_table.round(5).to_string(index=False))
```

```
A_full 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_full] ep  10 loss 0.2992 | EMA val Score 0.0857 MAE 0.0432 SSIM 0.8303
PSNR 22.97  <-- best  (8.7 min)
A_full 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
A_full 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_full] ep  20 loss 0.2658 | EMA val Score 0.0817 MAE 0.0412 SSIM 0.8382
PSNR 23.28  <-- best  (17.0 min)
A_full 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_full] ep  30 loss 0.2486 | EMA val Score 0.0794 MAE 0.0404 SSIM 0.8441
PSNR 23.47  <-- best  (25.4 min)
A_full 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_full 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_full] ep  40 loss 0.2428 | EMA val Score 0.0785 MAE 0.0399 SSIM 0.8458
PSNR 23.54  <-- best  (33.7 min)
A_det 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_det] ep  10 loss 0.4641 | EMA val Score 0.0729 MAE 0.0374 SSIM 0.8581
PSNR 24.12  <-- best  (7.4 min)
A_det 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_det] ep  20 loss 0.4237 | EMA val Score 0.0723 MAE 0.0370 SSIM 0.8584
PSNR 24.21  <-- best  (15.0 min)
A_det 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
A_det 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_det] ep  30 loss 0.3940 | EMA val Score 0.0721 MAE 0.0367 SSIM 0.8587
PSNR 24.24  <-- best  (22.6 min)
A_det 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_det 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_det] ep  40 loss 0.3842 | EMA val Score 0.0720 MAE 0.0366 SSIM 0.8587
PSNR 24.25  <-- best  (30.2 min)
A_noFiLM 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noFiLM] ep  10 loss 0.3022 | EMA val Score 0.0829 MAE 0.0417 SSIM 0.83
49 PSNR 23.15  <-- best  (7.6 min)
A_noFiLM 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noFiLM] ep  20 loss 0.2738 | EMA val Score 0.0798 MAE 0.0404 SSIM 0.84
23 PSNR 23.44  <-- best  (15.5 min)
A_noFiLM 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
A_noFiLM 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noFiLM] ep  30 loss 0.2516 | EMA val Score 0.0772 MAE 0.0393 SSIM 0.84
83 PSNR 23.65  <-- best  (23.4 min)
A_noFiLM 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noFiLM 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noFiLM] ep  40 loss 0.2458 | EMA val Score 0.0765 MAE 0.0390 SSIM 0.85
01 PSNR 23.72  <-- best  (31.4 min)
A_noCoarseCh 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noCoarseCh] ep  10 loss 0.3269 | EMA val Score 0.0873 MAE 0.0437 SSIM
0.8259 PSNR 22.89  <-- best  (8.2 min)
A_noCoarseCh 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noCoarseCh] ep  20 loss 0.3040 | EMA val Score 0.0854 MAE 0.0428 SSIM
0.8298 PSNR 23.02  <-- best  (16.5 min)
A_noCoarseCh 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noCoarseCh] ep  30 loss 0.2921 | EMA val Score 0.0842 MAE 0.0422 SSIM
0.8321 PSNR 23.11  <-- best  (24.8 min)
A_noCoarseCh 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
A_noCoarseCh 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noCoarseCh 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noCoarseCh] ep  40 loss 0.2879 | EMA val Score 0.0838 MAE 0.0421 SSIM
0.8331 PSNR 23.15  <-- best  (33.2 min)
A_noMRIch 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noMRIch] ep  10 loss 0.3010 | EMA val Score 0.0851 MAE 0.0427 SSIM 0.8
302 PSNR 23.03  <-- best  (8.2 min)
A_noMRIch 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noMRIch] ep  20 loss 0.2717 | EMA val Score 0.0830 MAE 0.0416 SSIM 0.8
343 PSNR 23.19  <-- best  (16.5 min)
A_noMRIch 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noMRIch] ep  30 loss 0.2601 | EMA val Score 0.0815 MAE 0.0410 SSIM 0.8
379 PSNR 23.30  <-- best  (24.8 min)
A_noMRIch 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noMRIch 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
[A_noMRIch] ep  40 loss 0.2523 | EMA val Score 0.0807 MAE 0.0406 SSIM 0.8
393 PSNR 23.38  <-- best  (33.1 min)
A_epsPred 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_epsPred] ep  10 loss 0.1231 | EMA val Score 0.1992 MAE 0.1044 SSIM 0.6
209 PSNR 17.80  <-- best  (8.1 min)
A_epsPred 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_epsPred] ep  20 loss 0.1106 | EMA val Score 0.1866 MAE 0.1025 SSIM 0.6
636 PSNR 18.10  <-- best  (16.5 min)
A_epsPred 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_epsPred] ep  30 loss 0.1039 | EMA val Score 0.1806 MAE 0.1042 SSIM 0.6
943 PSNR 18.17  <-- best  (24.8 min)
A_epsPred 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_epsPred 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_epsPred] ep  40 loss 0.1056 | EMA val Score 0.1759 MAE 0.1033 SSIM 0.7
095 PSNR 18.29  <-- best  (33.2 min)
A_noAuxSSIM 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
A_noAuxSSIM 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noAuxSSIM] ep  10 loss 0.2992 | EMA val Score 0.0854 MAE 0.0431 SSIM
0.8307 PSNR 22.99  <-- best  (8.1 min)
A_noAuxSSIM 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noAuxSSIM] ep  20 loss 0.2662 | EMA val Score 0.0818 MAE 0.0412 SSIM
0.8377 PSNR 23.29  <-- best  (16.3 min)
A_noAuxSSIM 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noAuxSSIM] ep  30 loss 0.2491 | EMA val Score 0.0794 MAE 0.0403 SSIM
0.8436 PSNR 23.49  <-- best  (24.5 min)
A_noAuxSSIM 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_noAuxSSIM 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_noAuxSSIM] ep  40 loss 0.2436 | EMA val Score 0.0786 MAE 0.0399 SSIM
0.8453 PSNR 23.56  <-- best  (32.7 min)
A_fixedCond 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_fixedCond] ep  10 loss 0.2651 | EMA val Score 0.0815 MAE 0.0412 SSIM
0.8388 PSNR 23.38  <-- best  (7.7 min)
```

```
A_fixedCond 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_fixedCond] ep  20 loss 0.2383 | EMA val Score 0.0802 MAE 0.0407 SSIM
0.8420 PSNR 23.48  <-- best  (15.7 min)
A_fixedCond 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_fixedCond] ep  30 loss 0.2262 | EMA val Score 0.0790 MAE 0.0402 SSIM
0.8447 PSNR 23.53  <-- best  (23.6 min)
A_fixedCond 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_fixedCond 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_fixedCond] ep  40 loss 0.2224 | EMA val Score 0.0784 MAE 0.0398 SSIM
0.8454 PSNR 23.58  <-- best  (31.5 min)
A_boneLoss 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_boneLoss] ep  10 loss 0.3739 | EMA val Score 0.0858 MAE 0.0432 SSIM
0.8294 PSNR 22.96  <-- best  (8.0 min)
A_boneLoss 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
A_boneLoss 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_boneLoss] ep  20 loss 0.3349 | EMA val Score 0.0822 MAE 0.0413 SSIM
0.8364 PSNR 23.26  <-- best  (16.5 min)
A_boneLoss 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_boneLoss] ep  30 loss 0.3141 | EMA val Score 0.0801 MAE 0.0405 SSIM
0.8418 PSNR 23.42  <-- best  (24.8 min)
A_boneLoss 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_boneLoss 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_boneLoss] ep  40 loss 0.3060 | EMA val Score 0.0791 MAE 0.0401 SSIM
0.8441 PSNR 23.50  <-- best  (33.2 min)
A_x0L1 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_x0L1] ep  10 loss 0.3001 | EMA val Score 0.0857 MAE 0.0433 SSIM 0.8302
PSNR 22.97  <-- best  (8.2 min)
A_x0L1 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_x0L1] ep  20 loss 0.2664 | EMA val Score 0.0817 MAE 0.0412 SSIM 0.8382
PSNR 23.30  <-- best  (16.6 min)
A_x0L1 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
A_x0L1 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_x0L1] ep  30 loss 0.2490 | EMA val Score 0.0792 MAE 0.0402 SSIM 0.8442
PSNR 23.50  <-- best  (24.9 min)
A_x0L1 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_x0L1 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_x0L1] ep  40 loss 0.2435 | EMA val Score 0.0782 MAE 0.0397 SSIM 0.8460
PSNR 23.60  <-- best  (33.3 min)
A_onS1_2d 1/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 2/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 3/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 4/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 5/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 6/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 7/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 8/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 9/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 10/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_onS1_2d] ep  10 loss 0.2981 | EMA val Score 0.0858 MAE 0.0424 SSIM 0.8
262 PSNR 22.94  <-- best  (8.2 min)
A_onS1_2d 11/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 12/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 13/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 14/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 15/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 16/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 17/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 18/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 19/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 20/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_onS1_2d] ep  20 loss 0.2687 | EMA val Score 0.0830 MAE 0.0411 SSIM 0.8
323 PSNR 23.11  <-- best  (16.6 min)
A_onS1_2d 21/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 22/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 23/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 24/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 25/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 26/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 27/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 28/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 29/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 30/40:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
[A_onS1_2d] ep  30 loss 0.2484 | EMA val Score 0.0810 MAE 0.0403 SSIM 0.8
372 PSNR 23.26  <-- best  (24.9 min)
A_onS1_2d 31/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 32/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 33/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 34/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 35/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 36/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 37/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 38/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 39/40:   0%|          | 0/359 [00:00<?, ?it/s]
A_onS1_2d 40/40:   0%|          | 0/359 [00:00<?, ?it/s]
[A_onS1_2d] ep  40 loss 0.2441 | EMA val Score 0.0800 MAE 0.0399 SSIM 0.8
398 PSNR 23.35  <-- best  (33.3 min)
     variant     MAE    SSIM     PSNR   Score      w*  MAE(w*)  SSIM(w*)
PSNR(w*)  Score(w*) stage1_used  Stage1_only_Score
      A_full 0.03683 0.86195 24.34498 0.07134 0.53776  0.03591   0.86573
24.53928    0.06948        25d2            0.07078
       A_det 0.03589 0.86233 24.40128 0.07031 0.69695  0.03579   0.86297
24.42346    0.07005        25d2            0.07078
    A_noFiLM 0.03616 0.86409 24.45069 0.07013 0.57288  0.03545   0.86720
24.62462    0.06865        25d2            0.07078
A_noCoarseCh 0.03775 0.85518 24.09371 0.07396 0.31725  0.03630   0.86225
24.36683    0.07074        25d2            0.07078
   A_noMRIch 0.03709 0.85727 24.25825 0.07277 0.48927  0.03607   0.86304
24.47038    0.07031        25d2            0.07078
   A_epsPred 0.07410 0.80061 20.99904 0.12395 0.07787  0.03707   0.86245
24.34657    0.07146        25d2            0.07078
 A_noAuxSSIM 0.03668 0.86156 24.35591 0.07129 0.53398  0.03585   0.86544
24.53979    0.06949        25d2            0.07078
 A_fixedCond 0.03997 0.84765 23.45161 0.07806 0.44508  0.03875   0.85507
23.86148    0.07498        25d2            0.07756
  A_boneLoss 0.03669 0.86117 24.36214 0.07140 0.53844  0.03583   0.86519
24.54082    0.06954        25d2            0.07078
      A_x0L1 0.03664 0.86188 24.37326 0.07117 0.56878  0.03583   0.86560
24.55186    0.06943        25d2            0.07078
   A_onS1_2d 0.03704 0.85683 24.16314 0.07283 0.54174  0.03620   0.86093
24.34109    0.07097          2d            0.07214
```

In [17…

```python
# %% ====== [8] CHOOSE + TRAIN THE FINAL MODEL (full epochs) ============
diff_rows = abl_table[abl_table["variant"].isin([n for n in ABLATIONS if
if FINAL_NAME is None:
    FINAL_NAME = diff_rows.sort_values("Score(w*)").iloc[0]["variant"]
print("Final config:", FINAL_NAME, ABLATIONS[FINAL_NAME])
FINAL_CFG = make_cfg(ABLATIONS[FINAL_NAME], MAIN_S1)
FINAL_TAG = FINAL_NAME if ABL_EPOCHS == FINAL_EPOCHS else f"final_{FINAL_
final_model = get_stage2(FINAL_TAG, FINAL_CFG, FINAL_EPOCHS)
FB = get_bundle(FINAL_CFG["s1"])
VALd = get_split("val", VAL_SNAMES, FB.offsets)
```

```
Final config: A_noFiLM {'film': False}
final_A_noFiLM 1/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 2/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 3/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 4/120:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
final_A_noFiLM 5/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 6/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 7/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 8/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 9/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 10/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep  10 loss 0.3029 | EMA val Score 0.0829 MAE 0.0416 SSI
M 0.8349 PSNR 23.15  <-- best  (7.8 min)
final_A_noFiLM 11/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 12/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 13/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 14/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 15/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 16/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 17/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 18/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 19/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 20/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep  20 loss 0.2770 | EMA val Score 0.0797 MAE 0.0404 SSI
M 0.8426 PSNR 23.43  <-- best  (15.7 min)
final_A_noFiLM 21/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 22/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 23/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 24/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 25/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 26/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 27/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 28/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 29/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 30/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep  30 loss 0.2565 | EMA val Score 0.0765 MAE 0.0391 SSI
M 0.8505 PSNR 23.68  <-- best  (23.7 min)
final_A_noFiLM 31/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 32/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 33/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 34/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 35/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 36/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 37/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 38/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 39/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 40/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep  40 loss 0.2452 | EMA val Score 0.0743 MAE 0.0384 SSI
M 0.8564 PSNR 23.88  <-- best  (31.6 min)
final_A_noFiLM 41/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 42/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 43/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 44/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 45/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 46/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 47/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 48/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 49/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 50/120:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
[final_A_noFiLM] ep  50 loss 0.2253 | EMA val Score 0.0722 MAE 0.0375 SSI
M 0.8611 PSNR 24.06  <-- best  (39.6 min)
final_A_noFiLM 51/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 52/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 53/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 54/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 55/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 56/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 57/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 58/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 59/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 60/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep  60 loss 0.2199 | EMA val Score 0.0706 MAE 0.0368 SSI
M 0.8648 PSNR 24.20  <-- best  (47.5 min)
final_A_noFiLM 61/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 62/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 63/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 64/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 65/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 66/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 67/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 68/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 69/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 70/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep  70 loss 0.2055 | EMA val Score 0.0697 MAE 0.0366 SSI
M 0.8676 PSNR 24.28  <-- best  (55.5 min)
final_A_noFiLM 71/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 72/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 73/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 74/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 75/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 76/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 77/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 78/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 79/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 80/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep  80 loss 0.2002 | EMA val Score 0.0686 MAE 0.0360 SSI
M 0.8697 PSNR 24.37  <-- best  (63.4 min)
final_A_noFiLM 81/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 82/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 83/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 84/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 85/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 86/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 87/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 88/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 89/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 90/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep  90 loss 0.1923 | EMA val Score 0.0680 MAE 0.0358 SSI
M 0.8710 PSNR 24.44  <-- best  (71.4 min)
final_A_noFiLM 91/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 92/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 93/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 94/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 95/120:   0%|          | 0/359 [00:00<?, ?it/s]
```

```
final_A_noFiLM 96/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 97/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 98/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 99/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 100/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep 100 loss 0.1874 | EMA val Score 0.0676 MAE 0.0355 SSI
M 0.8717 PSNR 24.46  <-- best  (79.3 min)
final_A_noFiLM 101/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 102/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 103/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 104/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 105/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 106/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 107/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 108/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 109/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 110/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep 110 loss 0.1858 | EMA val Score 0.0675 MAE 0.0355 SSI
M 0.8720 PSNR 24.48  <-- best  (87.3 min)
final_A_noFiLM 111/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 112/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 113/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 114/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 115/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 116/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 117/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 118/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 119/120:   0%|          | 0/359 [00:00<?, ?it/s]
final_A_noFiLM 120/120:   0%|          | 0/359 [00:00<?, ?it/s]
[final_A_noFiLM] ep 120 loss 0.1867 | EMA val Score 0.0674 MAE 0.0354 SSI
M 0.8723 PSNR 24.49  <-- best  (95.2 min)
```

In [19…

```python
import os
import shutil
import zipfile
from pathlib import Path
from IPython.display import display, FileLink

# ======================================================
# 1. FIND THE OUTPUT FOLDER
# ======================================================

working_dir = Path("/kaggle/working")

preferred_folders = [
    working_dir / "synthrad_ccr_v2",
    working_dir / "synthrad_ccr_v3",
    working_dir / "synthrad_ccr_diffusion",
]

folder_path = next(
    (p for p in preferred_folders if p.is_dir()),
    None
)
```

```
# Search other top-level output folders if necessary
if folder_path is None:
    candidates = [
        p for p in working_dir.iterdir()
        if p.is_dir()
        and ("synthrad" in p.name.lower() or "ccr" in p.name.lower())
        and any(f.is_file() for f in p.rglob("*"))
    ]

    if candidates:
        print("Available candidate folders:")
        for i, p in enumerate(candidates, 1):
            print(f"{i}. {p}")

        # Prefer the most recently modified candidate
        folder_path = max(
            candidates,
            key=lambda p: p.stat().st_mtime
        )

    else:
        print("No matching output folders found.")
        print("\nCurrent /kaggle/working contents:")
        for p in working_dir.iterdir():
            print(" -", p.name)

        raise FileNotFoundError(
            "Run the model/output cells first, or set folder_path "
            "to the actual directory shown above."
        )

print("Folder selected:", folder_path)

# ======================================================
# 2. CREATE THE ZIP ARCHIVE
# ======================================================

output_base = working_dir / f"{folder_path.name}_backup"

zip_path = shutil.make_archive(
    str(output_base),
    "zip",
    root_dir=str(folder_path)
)

# ======================================================
# 3. VERIFY THE ZIP
# ======================================================

with zipfile.ZipFile(zip_path, "r") as archive:
    bad_file = archive.testzip()

if bad_file:
```

```
    raise RuntimeError(f"ZIP verification failed: {bad_file}")

print("\nZIP created and verified successfully.")
print("File:", zip_path)
print(f"Size: {os.path.getsize(zip_path) / (1024**2):.2f} MB")

# ====================================================
# 4. DISPLAY DOWNLOAD LINK
# ====================================================

print("\nClick the link below to download your ZIP file:")
display(FileLink(zip_path))
```

```
Folder selected: /kaggle/working/synthrad_ccr_v3

ZIP created and verified successfully.
File: /kaggle/working/synthrad_ccr_v3_backup.zip
Size: 3027.69 MB

Click the link below to download your ZIP file:
/kaggle/working/synthrad_ccr_v3_backup.zip
```

In [21…

```
# %% ====== [9] INFERENCE ABLATIONS (no retraining, validation) =========
P = FINAL_PROTOCOL
val_stack = predict_stack(VALd, final_model, FINAL_CFG, FB, P["samples"],
nF, nL, nS = val_stack["res"].shape[:3]
sweep = []


def add(name, **kw):
    c, r = aggregate(val_stack, **kw)
    sweep.append(summarize_pred(name, c, r, v_gt, v_mask, v_pid))


sweep.append({"variant": "Stage-1 only", **{k: v for k, v in patient_leve
add("1 sample, 1 fold, no TTA", fidx=[0], lidx=[0], sidx=[0])
for s in range(1, nS + 1):
    add(f"{s} sample(s), all folds, no TTA", lidx=[0], sidx=list(range(s)
for f_ in (1, 2, nF):
    add(f"{nS} samples, {f_} fold(s), no TTA", fidx=list(range(f_)), lidx
if nL > 1:
    add(f"{nS} samples, all folds, flip-TTA", sidx=list(range(nS)))
sweep_table = pd.DataFrame(sweep); print(sweep_table.round(5).to_string(i
sweep_table.to_csv(os.path.join(DIR_METRICS, "inference_ablation_val.csv"

# DDIM steps (1 fold, 1 sample) and EMA vs raw weights (cheap protocol)
extra = []
if FINAL_CFG["mode"] == "diff":
    for st_ in (10, 25, 50, 100):
        s_ = predict_stack(VALd, final_model, FINAL_CFG, FB, 1, st_, fold
        c, r = aggregate(s_); extra.append(summarize_pred(f"DDIM steps={s
for which in ("best", "final_ema", "final_raw"):
    m_ = get_stage2(FINAL_TAG, FINAL_CFG, FINAL_EPOCHS, which=which)
    s_ = predict_stack(VALd, m_, FINAL_CFG, FB, 2, 25, folds=(1, 2, 3, 4)
```

```
    c, r = aggregate(s_); extra.append(summarize_pred(f"weights={which} (
extra_table = pd.DataFrame(extra); print(extra_table.round(5).to_string(i
extra_table.to_csv(os.path.join(DIR_METRICS, "steps_and_ema_ablation_val.

# Blend tuning on VAL ONLY: scalar w and the coarse-intensity gate
c_val, r_val = aggregate(val_stack)                          # final proto
w_best = wstar(c_val, r_val, v_gt, v_mask)
best_cfg, best_sc = None, 1e9
for thr in (-0.15, 0.04):                                    # intensity g
    for w_soft in (0.0, 0.25, 0.5, 0.75, 1.0):
        for w_bone in (0.5, 0.75, 1.0):
            m_, _ = patient_level(blend(c_val, r_val, v_mask, w_soft, w_b
            if m_["Score"] < best_sc:
                best_sc, best_cfg = m_["Score"], (w_soft, w_bone, thr)
print(f"val-optimal scalar w* = {w_best:.3f} | best gate (w_soft, w_bone,
```

```
                     variant     MAE    SSIM     PSNR   Score      w*
MAE(w*)  SSIM(w*)  PSNR(w*)  Score(w*)
                  Stage-1 only 0.03629 0.86205 24.32174 0.07078     NaN
NaN       NaN       NaN        NaN
      1 sample, 1 fold, no TTA 0.03777 0.86056 23.81273 0.07263 0.54435
0.03595   0.86788  24.44595    0.06898
1 sample(s), all folds, no TTA 0.03423 0.87771 24.80903 0.06481 0.63137
0.03358   0.87911  25.04858    0.06380
2 sample(s), all folds, no TTA 0.03371 0.88085 24.97676 0.06350 0.69153
0.03332   0.88105  25.12720    0.06305
3 sample(s), all folds, no TTA 0.03361 0.88174 25.02336 0.06318 0.71119
0.03327   0.88163  25.14994    0.06287
4 sample(s), all folds, no TTA 0.03349 0.88231 25.05623 0.06292 0.72607
0.03319   0.88207  25.16891    0.06268
  4 samples, 1 fold(s), no TTA 0.03501 0.87619 24.67338 0.06596 0.75333
0.03462   0.87662  24.82460    0.06547
  4 samples, 2 fold(s), no TTA 0.03401 0.88057 24.92883 0.06387 0.74080
0.03377   0.88067  25.05244    0.06360
  4 samples, 4 fold(s), no TTA 0.03349 0.88231 25.05623 0.06292 0.72607
0.03319   0.88207  25.16891    0.06268
4 samples, all folds, flip-TTA 0.03322 0.88503 25.22622 0.06196 0.79709
0.03304   0.88436  25.27359    0.06195
                                        variant     MAE    SSIM     PSN
R   Score      w*  MAE(w*)  SSIM(w*)  PSNR(w*)  Score(w*)
                  DDIM steps=10 (1 fold, 1 sample) 0.03719 0.86502 24.0340
0 0.07094 0.58834  0.03589   0.86978  24.50436    0.06845
                  DDIM steps=25 (1 fold, 1 sample) 0.03812 0.86144 23.8403
1 0.07275 0.54873  0.03618   0.86817  24.44345    0.06914
                  DDIM steps=50 (1 fold, 1 sample) 0.03851 0.86004 23.7667
9 0.07350 0.53477  0.03629   0.86760  24.42306    0.06939
                 DDIM steps=100 (1 fold, 1 sample) 0.03874 0.85920 23.7256
1 0.07394 0.52720  0.03636   0.86728  24.41070    0.06954
     weights=best (4 folds x 2 samples, 25 steps) 0.03376 0.88068 24.9714
3 0.06359 0.69401  0.03339   0.88091  25.12012    0.06316
weights=final_ema (4 folds x 2 samples, 25 steps) 0.03376 0.88068 24.9714
3 0.06359 0.69401  0.03339   0.88091  25.12012    0.06316
weights=final_raw (4 folds x 2 samples, 25 steps) 0.03377 0.88071 24.9815
9 0.06359 0.69802  0.03333   0.88099  25.13120    0.06308
val-optimal scalar w* = 0.797 | best gate (w_soft, w_bone, thr) = (0.75,
1.0, -0.15) (Score 0.06170)
```

In [22…

```python
# %% ====== [10] FINAL TEST (run ONCE; nothing below may be used for tuni
TESTd = get_split("test", TEST_SNAMES, FB.offsets)
t_gt, t_mask, t_pid = TESTd["ct"][:, 0].numpy(), TESTd["mask"][:, 0].nump
test_stack = predict_stack(TESTd, final_model, FINAL_CFG, FB, P["samples"
c1, r1 = aggregate(test_stack, fidx=[0], lidx=[0], sidx=[0])
cE, rE = aggregate(test_stack, lidx=[0])                    # ensemble, n
cT, rT = aggregate(test_stack)                              # ensemble +
preds = {
    f"Stage-1 [{FINAL_CFG['s1']}] (4-fold ensemble)": finalize(cE, 0, t_m
    f"Diffusion {FINAL_NAME}: 1 sample, 1 fold": finalize(c1, r1, t_mask)
    "+ ensemble (4 folds x samples), w=1": finalize(cE, rE, t_mask),
    "+ flip-TTA, w=1": finalize(cT, rT, t_mask),
    f"+ val-optimal scalar w*={w_best:.2f}": finalize(cT, rT, t_mask, w_b
```

```
    f"+ val-tuned gate {best_cfg}": blend(cT, rT, t_mask, *best_cfg),
}
# fixed-in-advance baselines for context: Stage-1 2D, and the determinist
if "2d" in S1_TO_RUN and FINAL_CFG["s1"] != "2d":
    b2 = get_bundle("2d"); d2 = get_split("test", TEST_SNAMES, b2.offsets
    preds["Stage-1 [2d] (4-fold ensemble)"] = finalize(stage1_ensemble(b2
for name in TEST_ABLATIONS:
    cfg_a = make_cfg(ABLATIONS[name], MAIN_S1)
    m_a = get_stage2(name, cfg_a, ABL_EPOCHS)
    b_a = get_bundle(cfg_a["s1"]); d_a = get_split("test", TEST_SNAMES, b
    s_a = predict_stack(d_a, m_a, cfg_a, b_a, ABL_PROTOCOL["samples"], AB
    c_a, r_a = aggregate(s_a)
    preds[f"[ablation @{ABL_EPOCHS}ep] {name}, w=1"] = finalize(c_a, r_a,

rows = [{"variant": k, **patient_level(p, t_gt, t_mask, t_pid)[0]} for k,
test_table = pd.DataFrame(rows); print(test_table.round(5).to_string(inde
test_table.to_csv(os.path.join(DIR_METRICS, "test_comparison.csv"), index
if not HU_MAPPING_VERIFIED:
    print("[CAUTION] HU_MAE_provisional assumes the HU window above; do n

# paired statistics (per patient, patient-bootstrap CI, Wilcoxon). b = ne
keys = list(preds)
final_key = f"+ val-tuned gate {best_cfg}"
stats_out = {}
for base_key in [keys[0]] + [k for k in keys if k.startswith("Stage-1 [2d
    s = paired_stats(preds[base_key], preds[final_key], t_gt, t_mask, t_p
    stats_out[base_key] = s
    print(f"\n=== {final_key}  minus  {base_key} ===")
    print(s.round(5).to_string(index=False))
pd.concat({k: v for k, v in stats_out.items()}).to_csv(os.path.join(DIR_M
print("\nPer-patient test metrics, final vs Stage-1:")
print(pd.concat({"stage1": patient_level(preds[keys[0]], t_gt, t_mask, t_
                 "final": patient_level(preds[final_key], t_gt, t_mask, t

# qualitative figure
pids_u = sorted(set(t_pid))
fig, ax = plt.subplots(len(pids_u), 6, figsize=(20, 3.3 * len(pids_u)), s
fin = preds[final_key]; s1p = preds[keys[0]]
for r_, pid in enumerate(pids_u):
    idx = [i for i, p in enumerate(t_pid) if p == pid]; i = idx[len(idx)
    c_ = FB.offsets.index(0)
    panels = [(TESTd["mri"][i, c_].numpy(), f"MRI {pid}", "gray"), (t_gt[
              (fin[i], "Final sCT", "gray"), (np.abs(s1p[i] - t_gt[i]) *
              (np.abs(fin[i] - t_gt[i]) * mk, "|err| Final", "hot")]
    for c, (img, ttl, cm) in enumerate(panels):
        ax[r_, c].imshow(img, cmap=cm, vmin=-1 if cm == "gray" else 0, vm
        ax[r_, c].set_title(ttl, fontsize=9); ax[r_, c].axis("off")
plt.tight_layout(); plt.savefig(os.path.join(DIR_FIG, "test_qualitative.p
```

```
                            variant     MAE    RMSE    SSIM     PSNR  Gl
obal_SSIM  Global_PSNR   Score  HU_MAE_provisional
    Stage-1 [25d2] (4-fold ensemble) 0.02908 0.05596 0.88533 25.27893
0.85139     23.76638 0.05775            72.70065
Diffusion A_noFiLM: 1 sample, 1 fold 0.03062 0.05900 0.88280 24.86052
0.85023     23.62998 0.05992            76.56105
 + ensemble (4 folds x samples), w=1 0.02644 0.05081 0.90276 26.15710
0.85912     24.04258 0.05075            66.10534
                    + flip-TTA, w=1 0.02594 0.04960 0.90576 26.35798
0.86041     24.10272 0.04950            64.84977
        + val-optimal scalar w*=0.80 0.02598 0.04955 0.90489 26.35917
0.86006     24.10201 0.04976            64.95454
 + val-tuned gate (0.75, 1.0, -0.15) 0.02584 0.04946 0.90550 26.37915
0.86033     24.10737 0.04947            64.61180
      Stage-1 [2d] (4-fold ensemble) 0.02946 0.05760 0.88154 25.03039
0.84966     23.67524 0.05908            73.65805
         [ablation @40ep] A_det, w=1 0.02874 0.05581 0.88439 25.31024
0.85122     23.78175 0.05765            71.85944
[CAUTION] HU_MAE_provisional assumes the HU window above; do not report i
t until verified.

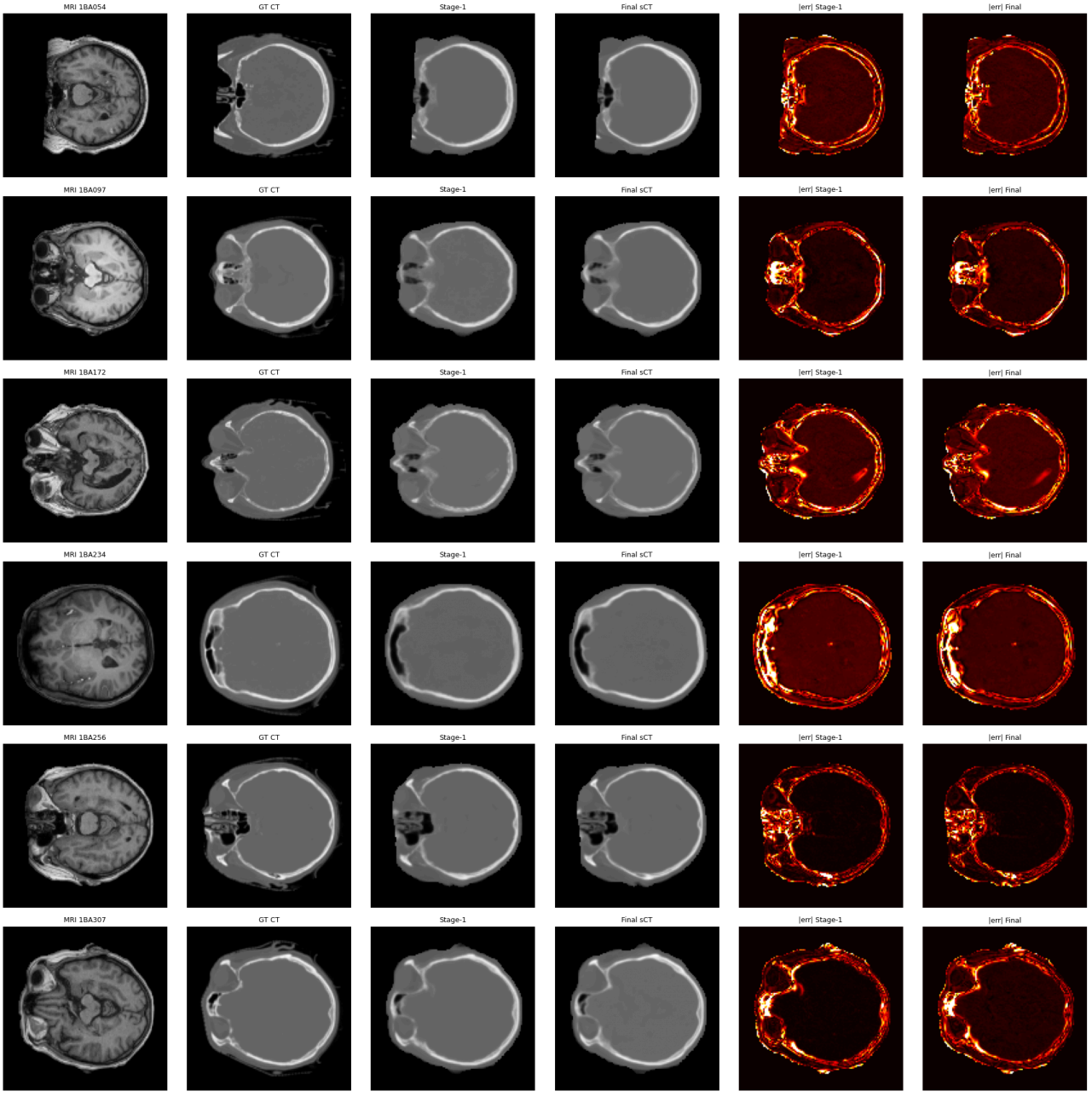
=== + val-tuned gate (0.75, 1.0, -0.15)  minus  Stage-1 [25d2] (4-fold en
semble) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae        -0.00324 -0.00376 -0.00264     0.03125                  6
6
  rmse        -0.00650 -0.00699 -0.00602     0.03125                  6
6
  ssim         0.02017  0.01727  0.02230     0.03125                  6
6
  psnr         1.10022  0.96389  1.29021     0.03125                  6
6

=== + val-tuned gate (0.75, 1.0, -0.15)  minus  Stage-1 [2d] (4-fold ense
mble) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae        -0.00362 -0.00457 -0.00263     0.03125                  6
6
  rmse        -0.00814 -0.00897 -0.00741     0.03125                  6
6
  ssim         0.02396  0.01989  0.02691     0.03125                  6
6
  psnr         1.34876  1.18092  1.58074     0.03125                  6
6

=== + val-tuned gate (0.75, 1.0, -0.15)  minus  [ablation @40ep] A_det,
w=1 ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae        -0.00290 -0.00375 -0.00208     0.03125                  6
6
  rmse        -0.00635 -0.00710 -0.00554     0.03125                  6
6
```

```
  ssim         0.02111  0.01790  0.02367     0.03125                  6
6
  psnr         1.06891  0.89080  1.26379     0.03125                  6
6

Per-patient test metrics, final vs Stage-1:
            stage1                    final
               mae    ssim     psnr     mae    ssim     psnr
patient_id
1BA054      0.0306  0.8641  25.1473  0.0272  0.8864  26.2898
1BA097      0.0327  0.8851  24.1111  0.0288  0.9080  25.1060
1BA172      0.0322  0.8631  24.5190  0.0302  0.8859  25.4487
1BA234      0.0265  0.9087  25.0862  0.0239  0.9222  26.0135
1BA256      0.0262  0.8838  26.2784  0.0227  0.9040  27.3527
1BA307      0.0263  0.9071  26.5316  0.0222  0.9265  28.0642
```



In [23…

```
# ======================================================================
# STEP 11: FINAL TEST METRICS + BONE DICE + NCC
# Run AFTER the existing [10] FINAL TEST cell.
# Uses existing predictions: preds, t_gt, t_mask, t_pid
# No training or additional diffusion inference.
# ======================================================================

import os
import numpy as np
import pandas as pd
from skimage.metrics import structural_similarity as compute_ssim

print("=" * 110)
print("FINAL TEST METRICS — IMAGE QUALITY + PROVISIONAL BONE DICE + NCC")
print("=" * 110)

# ----------------------------------------------------------------------
# 1. Verify variables from the preceding final-test cell
# ----------------------------------------------------------------------

required = ["preds", "t_gt", "t_mask", "t_pid", "DIR_METRICS"]

missing = [name for name in required if name not in globals()]
if missing:
    raise RuntimeError(
        f"Missing variables: {missing}. Run the existing [10] FINAL TEST
    )

gt_all = np.asarray(t_gt, dtype=np.float32)
mask_all = np.asarray(t_mask, dtype=np.float32)
patient_ids = list(t_pid)

assert len(gt_all) == len(mask_all) == len(patient_ids)
assert len(set(patient_ids)) == 6, (
    f"Expected the fixed six test patients; found {len(set(patient_ids))}
)

# ----------------------------------------------------------------------
# 2. Intensity configuration for the provisional bone metric
# ----------------------------------------------------------------------

HU_MIN_EVAL = float(globals().get("HU_MIN", -1000.0))
HU_MAX_EVAL = float(globals().get("HU_MAX", 1500.0))
BONE_HU_THRESHOLD = 300.0

if HU_MAX_EVAL <= HU_MIN_EVAL:
    raise ValueError("HU_MAX must be greater than HU_MIN.")

# Converts the assumed 300-HU threshold into normalized [-1, 1].
BONE_THRESHOLD_NORM = (
    2.0 * (BONE_HU_THRESHOLD - HU_MIN_EVAL)
    / (HU_MAX_EVAL - HU_MIN_EVAL)
    - 1.0
)
```

```python
HU_SPAN_EVAL = HU_MAX_EVAL - HU_MIN_EVAL
HU_VERIFIED = bool(globals().get("HU_MAPPING_VERIFIED", False))

print(f"Assumed HU range: [{HU_MIN_EVAL:.1f}, {HU_MAX_EVAL:.1f}]")
print(f"Bone threshold: {BONE_HU_THRESHOLD:.1f} HU")
print(f"Equivalent normalized threshold: {BONE_THRESHOLD_NORM:.4f}")
print("HU mapping verified:", HU_VERIFIED)

if not HU_VERIFIED:
    print(
        "CAUTION: Bone Dice and HU-MAE are provisional because "
        "the PNG-to-HU mapping has not been verified."
    )

# ---------------------------------------------------------------------
# 3. NCC calculation
# ---------------------------------------------------------------------

def calculate_ncc(a, b, valid_mask=None):
    """
    Zero-mean normalized cross-correlation (Pearson correlation)
    over selected pixels.
    """
    a = np.asarray(a, dtype=np.float64)
    b = np.asarray(b, dtype=np.float64)

    if valid_mask is not None:
        valid = np.asarray(valid_mask, dtype=bool)
        a = a[valid]
        b = b[valid]
    else:
        a = a.ravel()
        b = b.ravel()

    finite = np.isfinite(a) & np.isfinite(b)
    a, b = a[finite], b[finite]

    if a.size < 2:
        return np.nan

    a = a - a.mean()
    b = b - b.mean()

    denominator = np.sqrt(np.sum(a ** 2) * np.sum(b ** 2))

    if denominator < 1e-12:
        return np.nan

    return float(np.sum(a * b) / denominator)

# ---------------------------------------------------------------------
# 4. Bone Dice calculation
# ---------------------------------------------------------------------
```

```python
def calculate_bone_dice(pred_norm, gt_norm, body_mask, threshold_norm):
    """
    Threshold both prediction and reference at the same normalized
    intensity threshold, then calculate Dice inside the MRI mask.

    If both bone masks are empty, return 1.0 by convention.
    """
    body = np.asarray(body_mask) > 0.5

    pred_bone = (np.asarray(pred_norm) > threshold_norm) & body
    gt_bone = (np.asarray(gt_norm) > threshold_norm) & body

    denominator = int(pred_bone.sum() + gt_bone.sum())

    if denominator == 0:
        return 1.0

    intersection = np.logical_and(pred_bone, gt_bone).sum()

    return float(2.0 * intersection / denominator)

# ---------------------------------------------------------------------
# 5. Calculate all metrics for one slice
# ---------------------------------------------------------------------

def evaluate_all_metrics(pred_norm, gt_norm, body_mask):
    pred = np.clip(np.asarray(pred_norm, dtype=np.float32), -1.0, 1.0)
    gt = np.clip(np.asarray(gt_norm, dtype=np.float32), -1.0, 1.0)
    body = np.asarray(body_mask) > 0.5

    if not body.any():
        raise ValueError("Empty MRI-derived evaluation mask.")

    # Same normalized PNG intensity convention as the CCR-v3 evaluator.
    p = (pred + 1.0) / 2.0
    g = (gt + 1.0) / 2.0

    diff = p - g
    masked_diff = diff[body]

    masked_mae = float(np.mean(np.abs(masked_diff)))
    masked_mse = float(np.mean(masked_diff ** 2))
    masked_rmse = float(np.sqrt(masked_mse))

    _, ssim_map = compute_ssim(g, p, data_range=1.0, full=True)
    masked_ssim = float(np.mean(ssim_map[body]))

    masked_psnr = (
        float(10.0 * np.log10(1.0 / masked_mse))
        if masked_mse > 0 else float("inf")
    )

    # Global metrics: original, unmodified ground-truth CT reference.
```

```
    global_mae = float(np.mean(np.abs(diff)))
    global_mse = float(np.mean(diff ** 2))
    global_rmse = float(np.sqrt(global_mse))
    global_ssim = float(np.mean(ssim_map))

    global_psnr = (
        float(10.0 * np.log10(1.0 / global_mse))
        if global_mse > 0 else float("inf")
    )

    bone_dice = calculate_bone_dice(
        pred, gt, body, BONE_THRESHOLD_NORM
    )

    # Affine intensity normalization does not alter correlation.
    ncc_masked = calculate_ncc(pred, gt, body)
    ncc_global = calculate_ncc(pred, gt)

    return {
        "Masked_MAE_norm": masked_mae,
        "Masked_RMSE_norm": masked_rmse,
        "Masked_SSIM": masked_ssim,
        "Masked_PSNR_dB": masked_psnr,

        "Global_MAE_norm": global_mae,
        "Global_RMSE_norm": global_rmse,
        "Global_SSIM": global_ssim,
        "Global_PSNR_dB": global_psnr,

        "NCC_masked": ncc_masked,
        "NCC_global": ncc_global,

        "Bone_Dice_300HU_provisional": bone_dice,

        "HU_MAE_assumed_window_provisional": (
            masked_mae * HU_SPAN_EVAL
        ),

        "Score": masked_mae + 0.25 * (1.0 - masked_ssim),
    }

# ---------------------------------------------------------------------
# 6. Run evaluation for every prediction variant
# ---------------------------------------------------------------------

slice_records = []

for variant_name, predictions in preds.items():

    predictions = np.asarray(predictions, dtype=np.float32)

    if predictions.shape != gt_all.shape:
        raise ValueError(
            f"{variant_name}: predictions have shape {predictions.shape},
```

```
            f"but reference CT has shape {gt_all.shape}."
        )

    print("Evaluating:", variant_name)

    for i in range(len(gt_all)):
        metrics = evaluate_all_metrics(
            predictions[i],
            gt_all[i],
            mask_all[i]
        )

        slice_records.append({
            "variant": variant_name,
            "patient_id": patient_ids[i],
            "slice_index": i,
            **metrics
        })

slice_df = pd.DataFrame(slice_records)

# ---------------------------------------------------------------------
# 7. Patient-level aggregation
# First average slices within each patient, then patients equally.
# ---------------------------------------------------------------------

metric_cols = [
    col for col in slice_df.columns
    if col not in ["variant", "patient_id", "slice_index"]
]

patient_df = (
    slice_df
    .groupby(["variant", "patient_id"], as_index=False)[metric_cols]
    .mean()
)

# Mean and sample standard deviation across patient-level means.
summary_df = patient_df.groupby("variant")[metric_cols].agg(
    ["mean", "std"]
)

summary_df.columns = [
    f"{metric}_{stat}"
    for metric, stat in summary_df.columns
]
summary_df = summary_df.reset_index()

# ---------------------------------------------------------------------
# 8. Print the mean of every metric across the six patients
# ---------------------------------------------------------------------

display_cols = [
    "variant",
```

```
    "Masked_MAE_norm_mean",
    "Masked_RMSE_norm_mean",
    "Masked_SSIM_mean",
    "Masked_PSNR_dB_mean",
    "Global_MAE_norm_mean",
    "Global_SSIM_mean",
    "Global_PSNR_dB_mean",
    "NCC_masked_mean",
    "NCC_global_mean",
    "Bone_Dice_300HU_provisional_mean",
    "HU_MAE_assumed_window_provisional_mean",
    "Score_mean",
]

print("\n" + "=" * 160)
print("PATIENT-LEVEL MEAN METRICS")
print("=" * 160)
print(summary_df[display_cols].to_string(
    index=False,
    float_format=lambda x: f"{x:.5f}"
))

print("\n" + "=" * 160)
print("PATIENT-LEVEL STANDARD DEVIATIONS")
print("=" * 160)

std_cols = [
    "variant",
    "Masked_MAE_norm_std",
    "Masked_SSIM_std",
    "Masked_PSNR_dB_std",
    "NCC_masked_std",
    "Bone_Dice_300HU_provisional_std",
]

print(summary_df[std_cols].to_string(
    index=False,
    float_format=lambda x: f"{x:.5f}"
))

# ---------------------------------------------------------------------
# 9. Save slice, patient and summary tables
# ---------------------------------------------------------------------

os.makedirs(DIR_METRICS, exist_ok=True)

slice_path = os.path.join(
    DIR_METRICS, "test_metrics_all_models_slice_level.csv"
)
patient_path = os.path.join(
    DIR_METRICS, "test_metrics_all_models_patient_level.csv"
)
summary_path = os.path.join(
    DIR_METRICS, "test_metrics_all_models_mean_std.csv"
```

```
)

slice_df.to_csv(slice_path, index=False)
patient_df.to_csv(patient_path, index=False)
summary_df.to_csv(summary_path, index=False)

print("\nSaved slice metrics:", slice_path)
print("Saved patient metrics:", patient_path)
print("Saved mean/std summary:", summary_path)

if not HU_VERIFIED:
    print(
        "\nIMPORTANT: HU-MAE and Bone Dice are provisional. "
        "Do not claim verified physical-HU accuracy or clinical bone "
        "segmentation performance until the PNG intensity mapping is conf
    )

print("\nEvaluation completed. No model weights were modified.")
```

```
================================================================================
FINAL TEST METRICS — IMAGE QUALITY + PROVISIONAL BONE DICE + NCC
================================================================================
Assumed HU range: [-1000.0, 1500.0]
Bone threshold: 300.0 HU
Equivalent normalized threshold: 0.0400
HU mapping verified: False
CAUTION: Bone Dice and HU-MAE are provisional because the PNG-to-HU mapping has not b
Evaluating: Stage-1 [25d2] (4-fold ensemble)
Evaluating: Diffusion A_noFiLM: 1 sample, 1 fold
Evaluating: + ensemble (4 folds x samples), w=1
Evaluating: + flip-TTA, w=1
Evaluating: + val-optimal scalar w*=0.80
Evaluating: + val-tuned gate (0.75, 1.0, -0.15)
Evaluating: Stage-1 [2d] (4-fold ensemble)
Evaluating: [ablation @40ep] A_det, w=1

================================================================================
PATIENT-LEVEL MEAN METRICS
================================================================================
                             variant  Masked_MAE_norm_mean  Masked_RMSE_norm_mean  Ma
Global_PSNR_dB_mean  NCC_masked_mean  NCC_global_mean  Bone_Dice_300HU_provisional_me
 + ensemble (4 folds x samples), w=1               0.02644                0.05081
24.04258          0.92591          0.95667                             0.86680
                   + flip-TTA, w=1               0.02594                0.04960
24.10272          0.92931          0.95715                             0.87173
        + val-optimal scalar w*=0.80               0.02598                0.04955
24.10201          0.92936          0.95714                             0.87035
 + val-tuned gate (0.75, 1.0, -0.15)               0.02584                0.04946
24.10737          0.92959          0.95718                             0.87126
Diffusion A_noFiLM: 1 sample, 1 fold               0.03062                0.05900
23.62998          0.90237          0.95313                             0.84673
    Stage-1 [25d2] (4-fold ensemble)               0.02908                0.05596
23.76638          0.90979          0.95429                             0.84352
      Stage-1 [2d] (4-fold ensemble)               0.02946                0.05760
23.67524          0.90364          0.95342                             0.84088
         [ablation @40ep] A_det, w=1               0.02874                0.05581
23.78175          0.90962          0.95432                             0.84111

================================================================================
PATIENT-LEVEL STANDARD DEVIATIONS
================================================================================
                             variant  Masked_MAE_norm_std  Masked_SSIM_std  Masked_PS
 + ensemble (4 folds x samples), w=1              0.00385          0.01799
                    + flip-TTA, w=1               0.00352          0.01733
        + val-optimal scalar w*=0.80               0.00338          0.01722
 + val-tuned gate (0.75, 1.0, -0.15)               0.00335          0.01718
Diffusion A_noFiLM: 1 sample, 1 fold               0.00402          0.02121
    Stage-1 [25d2] (4-fold ensemble)               0.00307          0.01982
      Stage-1 [2d] (4-fold ensemble)               0.00331          0.01991
         [ablation @40ep] A_det, w=1               0.00339          0.02028

Saved slice metrics: /kaggle/working/synthrad_ccr_v3/metrics/test_metrics_all_models_
Saved patient metrics: /kaggle/working/synthrad_ccr_v3/metrics/test_metrics_all_model
Saved mean/std summary: /kaggle/working/synthrad_ccr_v3/metrics/test_metrics_all_mode
```

```
IMPORTANT: HU-MAE and Bone Dice are provisional. Do not claim verified physical-HU ac
pping is confirmed.

Evaluation completed. No model weights were modified.
```

In [24…

```python
# ============================================================================
# STEP 12: ALL-MODEL VISUALIZATION + PERFORMANCE IMPROVEMENT REPORT
# Run after STEP 10 (FINAL TEST) and STEP 11 (ALL METRICS).
#
# Creates:
# 1. Metric comparison charts with best model highlighted.
# 2. Improvement table compared with Stage-1.
# 3. Same-slice visual comparison of every prediction variant.
# 4. CSV reports and PNG figures.
# ============================================================================

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from skimage.metrics import structural_similarity as compute_ssim
from IPython.display import display

# ----------------------------------------------------------------------------
# 1. Validate existing results
# ----------------------------------------------------------------------------

required = [
    "preds", "summary_df", "t_gt", "t_mask",
    "t_pid", "TESTd"
]

missing = [x for x in required if x not in globals()]
if missing:
    raise RuntimeError(
        f"Missing variables: {missing}. "
        "Run the final test and metrics cells first."
    )

summary = summary_df.copy()
gt = np.asarray(t_gt)
mask = np.asarray(t_mask)
patient_ids = list(t_pid)

if "variant" not in summary.columns:
    summary = summary.reset_index()

if "variant" not in summary.columns:
    raise RuntimeError("summary_df must contain a 'variant' column.")

summary["variant"] = summary["variant"].astype(str)

# Select the Stage-1 baseline for comparison.
```

```python
baseline_candidates = [
    name for name in preds if name.startswith("Stage-1")
]

if not baseline_candidates:
    raise RuntimeError(
        "No Stage-1 baseline found in preds. "
        "Cannot calculate improvements relative to Stage-1."
    )

baseline_name = baseline_candidates[0]

# Save figures beside the experiment outputs.
plot_dir = globals().get(
    "DIR_FIG",
    os.path.join(
        globals().get("DIR_METRICS", "/kaggle/working"),
        "figures"
    )
)
os.makedirs(plot_dir, exist_ok=True)

print("=" * 100)
print("ALL-MODEL PERFORMANCE COMPARISON")
print("=" * 100)
print("Stage-1 baseline:", baseline_name)
print("Prediction variants:", len(preds))
print("Test patients:", len(set(patient_ids)))

# ------------------------------------------------------------------------
# 2. Metric definitions
# ------------------------------------------------------------------------

# metric name, dataframe column, optimization direction
metric_specs = [
    ("Masked MAE", "Masked_MAE_norm_mean", "lower"),
    ("Masked SSIM", "Masked_SSIM_mean", "higher"),
    ("Masked PSNR (dB)", "Masked_PSNR_dB_mean", "higher"),
    ("Masked NCC", "NCC_masked_mean", "higher"),
    (
        "Provisional Bone Dice",
        "Bone_Dice_300HU_provisional_mean",
        "higher"
    ),
    ("Composite Score", "Score_mean", "lower"),
]

# Support the alternative provisional-Dice column name produced
# by some earlier metric cells.
if (
    "Bone_Dice_300HU_provisional_mean" not in summary.columns
    and "Provisional_Intensity_Dice_mean" in summary.columns
):
    summary["Bone_Dice_300HU_provisional_mean"] = (
```

```
        summary["Provisional_Intensity_Dice_mean"]
    )

available_metrics = [
    spec for spec in metric_specs
    if spec[1] in summary.columns
]

if not available_metrics:
    raise RuntimeError("No expected summary metrics were found.")

# Keep only rows corresponding to predictions actually available.
summary = summary[summary["variant"].isin(preds)].copy()

if baseline_name not in summary["variant"].values:
    raise RuntimeError(
        "Stage-1 baseline is missing from summary_df. "
        "Run the metric calculation cell again."
    )

summary = summary.set_index("variant").loc[list(preds.keys())].reset_inde

# ---------------------------------------------------------------------
# 3. Find and mark the winner for each metric
# ---------------------------------------------------------------------

winner_report = []
winner_by_metric = {}

for label, col, direction in available_metrics:
    vals = pd.to_numeric(summary[col], errors="coerce")
    valid = vals.notna() & np.isfinite(vals)

    if not valid.any():
        continue

    valid_vals = vals[valid]
    winner_idx = valid_vals.idxmin() if direction == "lower" else valid_v

    winner_name = summary.loc[winner_idx, "variant"]
    winner_value = float(summary.loc[winner_idx, col])
    winner_by_metric[col] = winner_name

    baseline_value = float(
        summary.loc[summary["variant"] == baseline_name, col].iloc[0]
    )

    if direction == "lower":
        improvement = baseline_value - winner_value
    else:
        improvement = winner_value - baseline_value

    winner_report.append({
        "metric": label,
```

```
        "best_variant": winner_name,
        "best_value": winner_value,
        "Stage1_baseline_value": baseline_value,
        "improvement_vs_Stage1": improvement,
        "direction": direction,
    })

winner_df = pd.DataFrame(winner_report)

print("\nBEST VARIANT FOR EACH METRIC")
display(winner_df)

# ---------------------------------------------------------------------
# 4. Plot all metric variants; highlight winners and improvements
# ---------------------------------------------------------------------

n_metrics = len(available_metrics)
ncols = 2
nrows = int(np.ceil(n_metrics / ncols))

fig, axes = plt.subplots(
    nrows, ncols,
    figsize=(19, 5.5 * nrows),
    squeeze=False
)
axes = axes.ravel()

variant_names = summary["variant"].tolist()
baseline_idx = variant_names.index(baseline_name)

for ax_idx, (label, col, direction) in enumerate(available_metrics):
    ax = axes[ax_idx]
    values = pd.to_numeric(summary[col], errors="coerce").to_numpy()

    finite = np.isfinite(values)
    if not finite.any():
        ax.set_title(f"{label} — no valid values")
        ax.axis("off")
        continue

    valid_values = values[finite]
    valid_indices = np.flatnonzero(finite)

    best_idx = (
        valid_indices[np.argmin(valid_values)]
        if direction == "lower"
        else valid_indices[np.argmax(valid_values)]
    )

    bars = ax.bar(
        np.arange(len(values)),
        np.nan_to_num(values, nan=0.0),
        color="#4C78A8",
        alpha=0.85
```

```
    )

    # Highlight the Stage-1 baseline and the winner.
    bars[baseline_idx].set_color("#F58518")
    bars[best_idx].set_color("#54A24B")

    baseline_val = values[baseline_idx]

    for i, bar in enumerate(bars):
        if not np.isfinite(values[i]):
            continue

        val = values[i]
        if direction == "lower":
            delta = baseline_val - val
        else:
            delta = val - baseline_val

        label_text = f"{val:.3f}\n"
        if i == baseline_idx:
            label_text += "BASELINE"
        elif i == best_idx:
            label_text += "BEST"
        label_text += f"\nΔ={delta:+.3f}"

        ax.annotate(
            label_text,
            (bar.get_x() + bar.get_width() / 2, bar.get_height()),
            xytext=(0, 4),
            textcoords="offset points",
            ha="center",
            va="bottom",
            fontsize=7,
            rotation=0
        )

    ax.set_title(
        f"{label} ({'lower is better' if direction == 'lower' else 'highe
        fontsize=11
    )
    ax.set_xticks(np.arange(len(variant_names)))
    ax.set_xticklabels(
        variant_names,
        rotation=70,
        ha="right",
        fontsize=8
    )
    ax.grid(axis="y", alpha=0.25)
    ax.set_axisbelow(True)

for j in range(n_metrics, len(axes)):
    axes[j].axis("off")

fig.suptitle(
```

```
    "Model comparison: patient-level means across the fixed test cohort\n
    "Green = best on this metric | Orange = Stage-1 baseline | Δ = improv
    fontsize=14
)
fig.tight_layout(rect=[0, 0.02, 1, 0.95])

chart_path = os.path.join(plot_dir, "all_variants_metric_comparison.png")
fig.savefig(chart_path, dpi=160, bbox_inches="tight")
plt.show()
plt.close(fig)

print("\nSaved metric comparison plot:", chart_path)

# ---------------------------------------------------------------------
# 5. Report improvement or decline relative to Stage-1
# ---------------------------------------------------------------------

baseline_row = summary.loc[
    summary["variant"] == baseline_name
].iloc[0]

improvement_rows = []

for _, row in summary.iterrows():
    rec = {"variant": row["variant"]}

    for label, col, direction in available_metrics:
        current = pd.to_numeric(
            pd.Series([row[col]]), errors="coerce"
        ).iloc[0]
        base = pd.to_numeric(
            pd.Series([baseline_row[col]]), errors="coerce"
        ).iloc[0]

        if not np.isfinite(current) or not np.isfinite(base):
            rec[f"{label}_delta"] = np.nan
            rec[f"{label}_status"] = "N/A"
            continue

        delta = base - current if direction == "lower" else current - bas
        rec[f"{label}_delta"] = delta

        if row["variant"] == baseline_name:
            status = "BASELINE"
        elif abs(delta) < 1e-10:
            status = "NO CHANGE"
        else:
            status = "IMPROVED" if delta > 0 else "DECLINED"

        rec[f"{label}_status"] = status

    improvement_rows.append(rec)

improvement_df = pd.DataFrame(improvement_rows)
```

```python
print("\nIMPROVEMENT REPORT — POSITIVE Δ MEANS BETTER")
display(improvement_df)

improvement_csv = os.path.join(
    plot_dir, "all_variants_improvement_vs_stage1.csv"
)
winner_csv = os.path.join(
    plot_dir, "best_variant_by_metric.csv"
)

improvement_df.to_csv(improvement_csv, index=False)
winner_df.to_csv(winner_csv, index=False)

print("Saved improvement report:", improvement_csv)
print("Saved metric winner report:", winner_csv)

# ---------------------------------------------------------------------
# 6. Select one fixed representative test slice.
# Use the middle slice of the first test patient in sorted order.
# Do not select the slice because it looks best.
# ---------------------------------------------------------------------

selected_patient = sorted(set(patient_ids))[0]
patient_indices = [
    i for i, pid in enumerate(patient_ids)
    if pid == selected_patient
]
slice_idx = patient_indices[len(patient_indices) // 2]

# Select the centre MRI channel if the model uses 2.5D input.
mri_stack = TESTd["mri"][slice_idx]
if hasattr(mri_stack, "cpu"):
    mri_stack = mri_stack.cpu().numpy()

if mri_stack.ndim == 3:
    offsets = getattr(globals().get("FB", None), "offsets", None)
    center_channel = offsets.index(0) if offsets and 0 in offsets else mr
    mri_image = mri_stack[center_channel]
else:
    mri_image = np.asarray(mri_stack).squeeze()

reference_ct = gt[slice_idx]
body = mask[slice_idx] > 0.5

print("\nRepresentative visualization:")
print("Patient:", selected_patient)
print("Slice index:", slice_idx)

# ---------------------------------------------------------------------
# 7. Calculate per-slice captions using the common normalized metrics
# ---------------------------------------------------------------------

def calculate_display_metrics(pred_norm, gt_norm, body_mask):
```

```
    p = (np.clip(np.asarray(pred_norm), -1, 1) + 1) / 2
    g = (np.clip(np.asarray(gt_norm), -1, 1) + 1) / 2
    valid = np.asarray(body_mask) > 0.5

    diff = p[valid] - g[valid]
    mae = float(np.mean(np.abs(diff)))
    mse = float(np.mean(diff ** 2))
    psnr = float(10 * np.log10(1 / mse)) if mse > 0 else float("inf")

    _, ssim_map = compute_ssim(g, p, data_range=1.0, full=True)
    ssim = float(np.mean(ssim_map[valid]))

    return mae, ssim, psnr

# ---------------------------------------------------------------------
# 8. Plot MRI, reference CT, every variant and each error map
# ---------------------------------------------------------------------

n_variants = len(preds)
ncols = n_variants + 2

fig, axes = plt.subplots(
    2, ncols,
    figsize=(max(20, 3.1 * ncols), 8.0),
    squeeze=False
)

axes[0, 0].imshow(mri_image, cmap="gray", vmin=-1, vmax=1)
axes[0, 0].set_title(f"Input MRI\n{selected_patient}", fontsize=9)
axes[0, 0].axis("off")

axes[0, 1].imshow(reference_ct, cmap="gray", vmin=-1, vmax=1)
axes[0, 1].set_title("Ground-truth CT", fontsize=9)
axes[0, 1].axis("off")

# The second row's first two tiles explain the error maps.
axes[1, 0].imshow(mask[slice_idx], cmap="gray", vmin=0, vmax=1)
axes[1, 0].set_title("Evaluation mask", fontsize=9)
axes[1, 0].axis("off")

axes[1, 1].axis("off")
axes[1, 1].text(
    0.02, 0.9,
    "Masked absolute-error maps\n\n"
    "Same test slice for every variant.\n"
    "Smaller errors are better.\n\n"
    "Green chart bars identify the metric winner.",
    va="top",
    fontsize=10,
    wrap=True
)

for j, (variant_name, prediction) in enumerate(preds.items(), start=2):
    pred = np.asarray(prediction)[slice_idx]
```

```
    mae, ssim, psnr = calculate_display_metrics(
        pred, reference_ct, mask[slice_idx]
    )

    axes[0, j].imshow(pred, cmap="gray", vmin=-1, vmax=1)
    axes[0, j].set_title(
        f"{variant_name}\nMAE={mae:.4f}\nSSIM={ssim:.4f}\nPSNR={psnr:.2f}
        fontsize=8
    )
    axes[0, j].axis("off")

    # Error in [0,1] intensity units, restricted to the mask.
    error_map = (
        np.abs((pred + 1) / 2 - (reference_ct + 1) / 2)
        * body
    )

    axes[1, j].imshow(
        error_map,
        cmap="hot",
        vmin=0,
        vmax=0.35
    )
    axes[1, j].set_title(
        f"{variant_name}\nMasked error",
        fontsize=8
    )
    axes[1, j].axis("off")

fig.suptitle(
    f"Prediction comparison on the same test slice — patient {selected_pa
    fontsize=14
)
fig.tight_layout(rect=[0, 0, 1, 0.95])

image_path = os.path.join(
    plot_dir, "all_variants_same_slice_visualization.png"
)
fig.savefig(image_path, dpi=150, bbox_inches="tight")
plt.show()
plt.close(fig)

print("\nSaved image comparison:", image_path)

# ----------------------------------------------------------------------
# 9. Final machine-readable winner summary
# ----------------------------------------------------------------------

print("\n" + "=" * 100)
print("WHICH VARIANT OUTPERFORMED?")
print("=" * 100)

for _, row in winner_df.iterrows():
```

```
    print(
        f"{row['metric']}: {row['best_variant']} "
        f"(value={row['best_value']:.5f}; "
        f"Δ vs Stage-1={row['improvement_vs_Stage1']:+.5f})"
    )

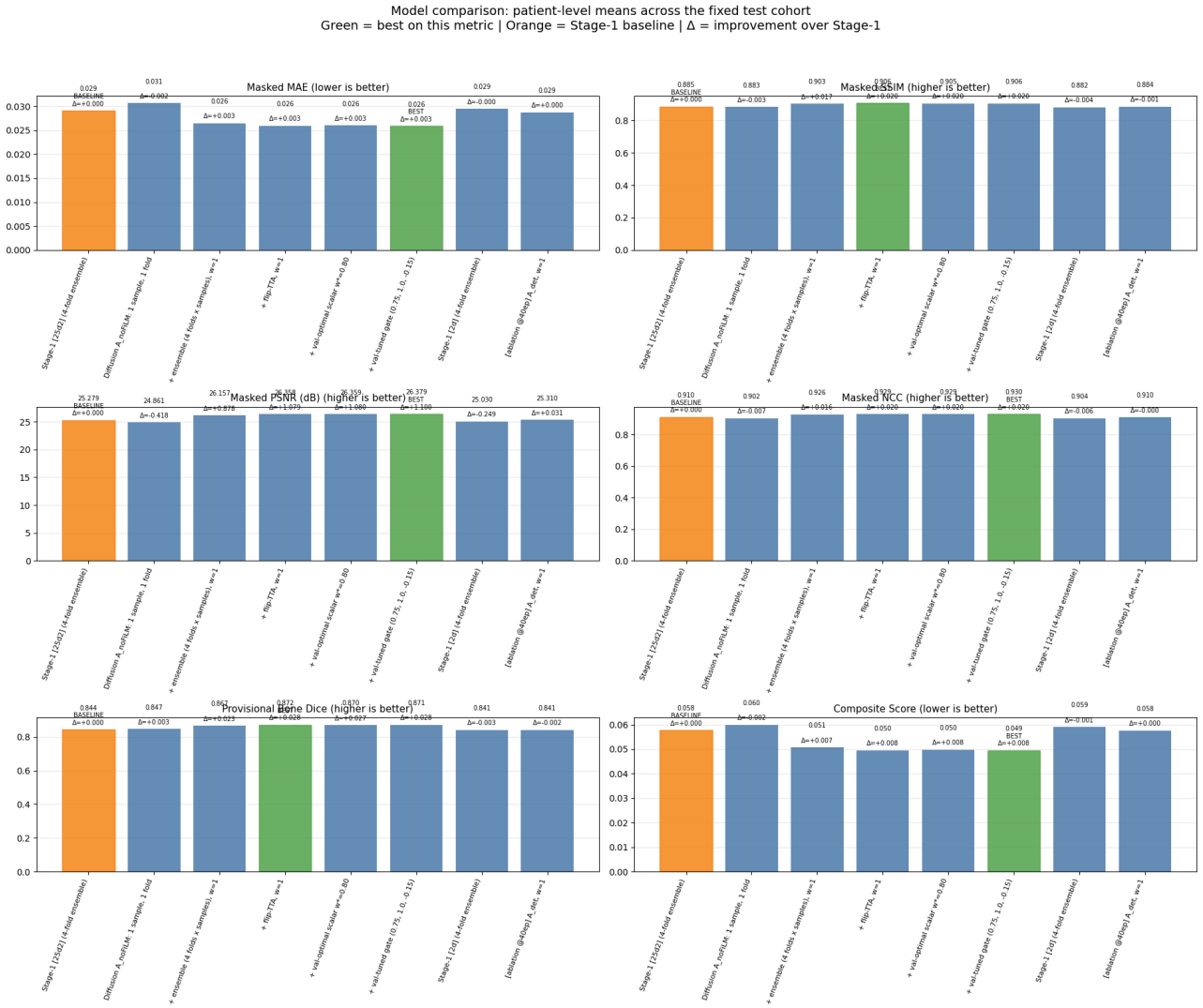
print("\nAll plots and comparison CSVs saved to:", plot_dir)

if not globals().get("HU_MAPPING_VERIFIED", False):
    print(
        "\nCAUTION: Bone Dice is based on the assumed 300-HU threshold. "
        "It is a provisional intensity-threshold diagnostic, not "
        "verified clinical Bone Dice."
    )
```

```
================================================================================
ALL-MODEL PERFORMANCE COMPARISON
================================================================================
Stage-1 baseline: Stage-1 [25d2] (4-fold ensemble)
Prediction variants: 8
Test patients: 6

BEST VARIANT FOR EACH METRIC
```

	metric	best_variant	best_value	Stage1_baseline_value	improvement_vs_Stage1	direct
0	Masked MAE	+ val-tuned gate (0.75, 1.0, -0.15)	0.025845	0.029080	0.003236	lo
1	Masked SSIM	+ flip-TTA, w=1	0.905756	0.885332	0.020424	hig
2	Masked PSNR (dB)	+ val-tuned gate (0.75, 1.0, -0.15)	26.379150	25.278926	1.100223	hig
3	Masked NCC	+ val-tuned gate (0.75, 1.0, -0.15)	0.929594	0.909787	0.019807	hig
4	Provisional Bone Dice	+ flip-TTA, w=1	0.871733	0.843524	0.028210	hig
5	Composite Score	+ val-tuned gate (0.75, 1.0, -0.15)	0.049469	0.057747	0.008278	lo



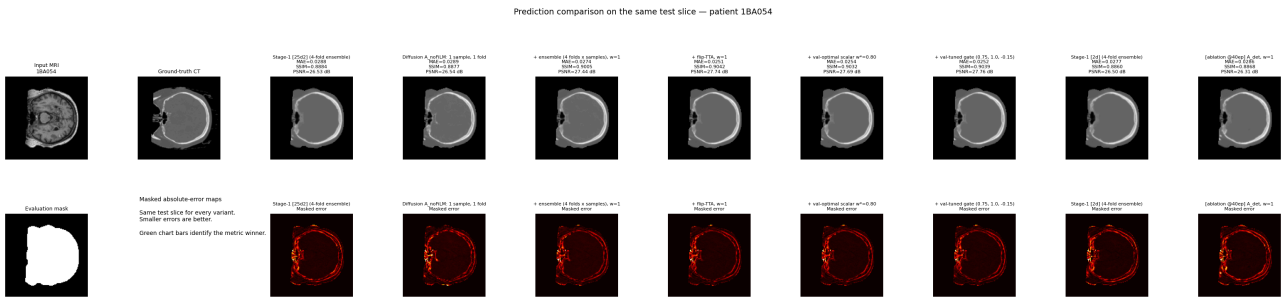
Saved metric comparison plot: /kaggle/working/synthrad_ccr_v3/figures/all_variants_metric_comparison.png

IMPROVEMENT REPORT — POSITIVE Δ MEANS BETTER

	variant	Masked MAE_delta	Masked MAE_status	Masked SSIM_delta	Masked SSIM_status	Masked PSNR (dB)_delta	Masked PSNR (dB)_status	Mas NCC_d
0	Stage-1 [25d2] (4-fold ensemble)	0.000000	BASELINE	0.000000	BASELINE	0.000000	BASELINE	0.000
1	Diffusion A_noFiLM: 1 sample, 1 fold	-0.001544	DECLINED	-0.002533	DECLINED	-0.418406	DECLINED	-0.007
2	+ ensemble (4 folds x samples), w=1	0.002638	IMPROVED	0.017432	IMPROVED	0.878171	IMPROVED	0.016
3	+ flip-TTA, w=1	0.003140	IMPROVED	0.020424	IMPROVED	1.079054	IMPROVED	0.019
4	+ val-optimal scalar w*=0.80	0.003098	IMPROVED	0.019557	IMPROVED	1.080244	IMPROVED	0.019
5	+ val-tuned gate (0.75, 1.0, -0.15)	0.003236	IMPROVED	0.020171	IMPROVED	1.100223	IMPROVED	0.019
6	Stage-1 [2d] (4-fold ensemble)	-0.000383	DECLINED	-0.003790	DECLINED	-0.248540	DECLINED	-0.006
7	[ablation @40ep] A_det, w=1	0.000336	IMPROVED	-0.000940	DECLINED	0.031314	IMPROVED	-0.000

```
Saved improvement report: /kaggle/working/synthrad_ccr_v3/figures/all_var
iants_improvement_vs_stage1.csv
Saved metric winner report: /kaggle/working/synthrad_ccr_v3/figures/bes
t_variant_by_metric.csv

Representative visualization:
Patient: 1BA054
Slice index: 33
```




```
Saved image comparison: /kaggle/working/synthrad_ccr_v3/figures/all_variants_same_sli
n.png

================================================================================
WHICH VARIANT OUTPERFORMED?
================================================================================
Masked MAE: + val-tuned gate (0.75, 1.0, -0.15) (value=0.02584; Δ vs Stage-1=+0.00324
Masked SSIM: + flip-TTA, w=1 (value=0.90576; Δ vs Stage-1=+0.02042)
Masked PSNR (dB): + val-tuned gate (0.75, 1.0, -0.15) (value=26.37915; Δ vs Stage-1=+
Masked NCC: + val-tuned gate (0.75, 1.0, -0.15) (value=0.92959; Δ vs Stage-1=+0.01981
Provisional Bone Dice: + flip-TTA, w=1 (value=0.87173; Δ vs Stage-1=+0.02821)
Composite Score: + val-tuned gate (0.75, 1.0, -0.15) (value=0.04947; Δ vs Stage-1=+0

All plots and comparison CSVs saved to: /kaggle/working/synthrad_ccr_v3/figures

CAUTION: Bone Dice is based on the assumed 300-HU threshold. It is a provisional inte
diagnostic, not verified clinical Bone Dice.
```

In [25…

```python
# ============================================================================
# STEP 13: VISUALIZE 1BA307 + OTHER PATIENTS WITH BETTER RESULTS
# Run after STEP 10 and STEP 11.
# Test results are for visualization/diagnosis only, not model tuning.
# ============================================================================

import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from skimage.metrics import structural_similarity as compute_ssim

# ------------------------------------------------------------
# 1. Verify the required variables
# ------------------------------------------------------------

required = ["preds", "t_gt", "t_mask", "t_pid", "TESTd"]

missing = [x for x in required if x not in globals()]
if missing:
    raise RuntimeError(
        f"Missing variables: {missing}. Run the final test and "
        "prediction cells first."
    )
```

```
gt = np.asarray(t_gt)
mask = np.asarray(t_mask)
pids = list(t_pid)

# ------------------------------------------------------------
# 2. Identify the final tuned model and Stage-1 baseline
# ------------------------------------------------------------

# Prefer the final_key selected in the FINAL TEST cell.
if "final_key" in globals() and final_key in preds:
    NEW_MODEL = final_key
else:
    candidates = [
        k for k in preds
        if "val-tuned gate" in k.lower()
        or "validation-tuned gate" in k.lower()
        or "validation_tuned_gate" in k.lower()
    ]

    if len(candidates) != 1:
        raise RuntimeError(
            "Could not identify one final validation-tuned model. "
            f"Prediction keys: {list(preds.keys())}"
        )

    NEW_MODEL = candidates[0]

stage1_candidates = [
    k for k in preds
    if k.lower().startswith("stage-1")
    and "4-fold ensemble" in k.lower()
]

if not stage1_candidates:
    stage1_candidates = [
        k for k in preds if k.lower().startswith("stage-1")
    ]

if not stage1_candidates:
    raise RuntimeError("Stage-1 baseline was not found in preds.")

BASELINE = stage1_candidates[0]
COMPETITORS = [k for k in preds if k != NEW_MODEL]

for name, arr in preds.items():
    if np.asarray(arr).shape != gt.shape:
        raise ValueError(
            f"{name}: prediction shape {np.asarray(arr).shape} "
            f"does not match reference shape {gt.shape}."
        )

print("Final model:", NEW_MODEL)
print("Stage-1 baseline:", BASELINE)
print("Test patients:", sorted(set(pids)))
```

```python
# ------------------------------------------------------------
# 3. Calculate slice-level masked image metrics
# ------------------------------------------------------------

def calculate_masked_metrics(pred, reference, body_mask):
    pred01 = (np.clip(pred, -1, 1) + 1.0) / 2.0
    gt01 = (np.clip(reference, -1, 1) + 1.0) / 2.0
    body = np.asarray(body_mask) > 0.5

    if not body.any():
        return {"MAE": np.nan, "SSIM": np.nan, "PSNR": np.nan}

    diff = pred01[body] - gt01[body]
    mae = float(np.mean(np.abs(diff)))
    mse = float(np.mean(diff ** 2))
    psnr = float(10 * np.log10(1.0 / mse)) if mse > 0 else np.inf

    _, ssim_map = compute_ssim(
        gt01, pred01, data_range=1.0, full=True
    )
    ssim = float(np.mean(ssim_map[body]))

    return {"MAE": mae, "SSIM": ssim, "PSNR": psnr}


# Average each metric over all slices belonging to a patient.
records = []

for model_name, predictions in preds.items():
    predictions = np.asarray(predictions)

    for i, pid in enumerate(pids):
        metrics = calculate_masked_metrics(
            predictions[i], gt[i], mask[i]
        )
        records.append({
            "patient_id": pid,
            "variant": model_name,
            **metrics
        })

slice_metrics_df = pd.DataFrame(records)

patient_metrics_df = (
    slice_metrics_df
    .groupby(["patient_id", "variant"], as_index=False)
    [["MAE", "SSIM", "PSNR"]]
    .mean()
)

# ------------------------------------------------------------
# 4. Find patients where the new model beats every competitor
# ------------------------------------------------------------
```

```
patient_ids = sorted(set(pids))
strict_winners = []
rank_records = []

for pid in patient_ids:
    table = patient_metrics_df[
        patient_metrics_df["patient_id"] == pid
    ].set_index("variant")

    new = table.loc[NEW_MODEL]
    others = table.loc[COMPETITORS]

    wins_mae = int((new["MAE"] < others["MAE"]).sum())
    wins_ssim = int((new["SSIM"] > others["SSIM"]).sum())
    wins_psnr = int((new["PSNR"] > others["PSNR"]).sum())

    strict = (
        (new["MAE"] < others["MAE"]).all()
        and (new["SSIM"] > others["SSIM"]).all()
        and (new["PSNR"] > others["PSNR"]).all()
    )

    # Separate comparison against Stage-1 only.
    s1 = table.loc[BASELINE]

    beats_stage1 = {
        "MAE": bool(new["MAE"] < s1["MAE"]),
        "SSIM": bool(new["SSIM"] > s1["SSIM"]),
        "PSNR": bool(new["PSNR"] > s1["PSNR"]),
    }

    strict_winners.append((pid, bool(strict)))

    rank_records.append({
        "patient_id": pid,
        "NewModel_MAE": new["MAE"],
        "NewModel_SSIM": new["SSIM"],
        "NewModel_PSNR": new["PSNR"],
        "Stage1_MAE": s1["MAE"],
        "Stage1_SSIM": s1["SSIM"],
        "Stage1_PSNR": s1["PSNR"],
        "MAE_improved_vs_Stage1": beats_stage1["MAE"],
        "SSIM_improved_vs_Stage1": beats_stage1["SSIM"],
        "PSNR_improved_vs_Stage1": beats_stage1["PSNR"],
        "Competitors_beaten_on_MAE": wins_mae,
        "Competitors_beaten_on_SSIM": wins_ssim,
        "Competitors_beaten_on_PSNR": wins_psnr,
        "Strict_win_over_all_variants": bool(strict),
        "Total_metric_wins": wins_mae + wins_ssim + wins_psnr,
    })

ranking_df = pd.DataFrame(rank_records)
```

```python
print("\nPATIENT-LEVEL PERFORMANCE")
print(ranking_df.round(4).to_string(index=False))

# ------------------------------------------------------------
# 5. Select patient 1BA307 plus the strongest other examples
# ------------------------------------------------------------

REQUESTED_PATIENT = "1BA307"

if REQUESTED_PATIENT not in patient_ids:
    print(
        f"\nWARNING: {REQUESTED_PATIENT} is not present in the "
        "current prediction arrays."
    )
    print("Available patients:", patient_ids)

# Start with 1BA307, if available.
selected_patients = (
    [REQUESTED_PATIENT]
    if REQUESTED_PATIENT in patient_ids else []
)

# First include any other patients where the final model strictly
# beats every competitor on all three masked metrics.
other_strict_winners = (
    ranking_df[
        ranking_df["Strict_win_over_all_variants"]
        & (ranking_df["patient_id"] != REQUESTED_PATIENT)
    ]
    .sort_values(
        "Total_metric_wins", ascending=False
    )["patient_id"]
    .tolist()
)

selected_patients.extend(other_strict_winners)

# If no such patient exists, show up to two additional patients with
# the most wins across all three metrics, excluding 1BA307.
if not other_strict_winners:
    alternatives = (
        ranking_df[
            ranking_df["patient_id"] != REQUESTED_PATIENT
        ]
        .sort_values(
            ["Total_metric_wins", "Competitors_beaten_on_SSIM",
             "Competitors_beaten_on_PSNR"],
            ascending=False
        )["patient_id"]
        .tolist()
    )

    selected_patients.extend(alternatives[:2])
```

```python
# Remove duplicates but preserve order.
selected_patients = list(dict.fromkeys(selected_patients))

print("\nPatients selected for visual inspection:", selected_patients)

# ------------------------------------------------------------
# 6. Find a representative middle slice for each patient
# ------------------------------------------------------------

plot_dir = globals().get(
    "DIR_FIG",
    "/kaggle/working/synthrad_ccr_v3/figures"
)
os.makedirs(plot_dir, exist_ok=True)

mri_data = TESTd["mri"]
slice_names = TESTd.get(
    "sname",
    [f"slice_{i:04d}" for i in range(len(pids))]
)

for pid in selected_patients:
    indices = [i for i, patient in enumerate(pids) if patient == pid]

    if not indices:
        continue

    # Select middle slice, not the slice with the highest score.
    idx = indices[len(indices) // 2]

    mri_stack = mri_data[idx]
    if hasattr(mri_stack, "cpu"):
        mri_stack = mri_stack.cpu().numpy()

    # For 2.5D MRI input, display the central slice.
    if mri_stack.ndim == 3:
        offsets = getattr(globals().get("FB", None), "offsets", None)
        center = (
            offsets.index(0)
            if offsets is not None and 0 in offsets
            else mri_stack.shape[0] // 2
        )
        mri_image = mri_stack[center]
    else:
        mri_image = np.asarray(mri_stack).squeeze()

    reference = gt[idx]
    body = mask[idx] > 0.5

    patient_row = ranking_df[
        ranking_df["patient_id"] == pid
    ].iloc[0]

    strict = patient_row["Strict_win_over_all_variants"]
```

```
    # Panels: MRI, ground truth, each model, CCR-v2 error map.
    ncols = len(preds) + 3

    fig, axes = plt.subplots(
        1, ncols,
        figsize=(max(19, 3.0 * ncols), 5.0),
        squeeze=False
    )
    axes = axes[0]

    axes[0].imshow(mri_image, cmap="gray", vmin=-1, vmax=1)
    axes[0].set_title(f"MRI\nPatient {pid}", fontsize=9)
    axes[0].axis("off")

    axes[1].imshow(reference, cmap="gray", vmin=-1, vmax=1)
    axes[1].set_title("Ground-truth CT", fontsize=9)
    axes[1].axis("off")

    for j, (name, prediction) in enumerate(preds.items(), start=2):
        image = np.asarray(prediction)[idx]

        mm = calculate_masked_metrics(image, reference, mask[idx])

        tag = ""
        if name == NEW_MODEL:
            tag = "\nFINAL MODEL"

        axes[j].imshow(image, cmap="gray", vmin=-1, vmax=1)
        axes[j].set_title(
            f"{name}{tag}\n"
            f"MAE={mm['MAE']:.4f}\n"
            f"SSIM={mm['SSIM']:.4f}\n"
            f"PSNR={mm['PSNR']:.2f} dB",
            fontsize=7.5
        )
        axes[j].axis("off")

    # Absolute error map of the final CCR-Diffusion prediction.
    final_image = np.asarray(preds[NEW_MODEL])[idx]
    error_map = (
        np.abs(
            (np.clip(final_image, -1, 1) + 1) / 2
            - (np.clip(reference, -1, 1) + 1) / 2
        ) * body
    )

    last_ax = axes[-1]
    image_plot = last_ax.imshow(
        error_map, cmap="hot", vmin=0, vmax=0.35
    )
    last_ax.set_title("Final model\nmasked absolute error", fontsize=9)
    last_ax.axis("off")
    fig.colorbar(image_plot, ax=last_ax, fraction=0.046, pad=0.04)
```

```
    status = (
        "BEATS ALL COMPETITORS on MAE, SSIM and PSNR"
        if strict
        else "Not a strict all-metric win; inspect patient metrics"
    )

    fig.suptitle(
        f"Patient {pid} | Representative slice: {slice_names[idx]}\n{stat
        fontsize=13
    )

    fig.tight_layout(rect=[0, 0, 1, 0.91])

    safe_pid = re.sub(r"[^A-Za-z0-9_-]", "_", str(pid))
    save_path = os.path.join(
        plot_dir, f"patient_{safe_pid}_all_model_comparison.png"
    )

    fig.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.show()
    plt.close(fig)

    print("Saved:", save_path)

# ------------------------------------------------------------
# 7. Save the patient-level selection report
# ------------------------------------------------------------

report_path = os.path.join(
    plot_dir, "patient_level_all_model_comparison.csv"
)
ranking_df.to_csv(report_path, index=False)

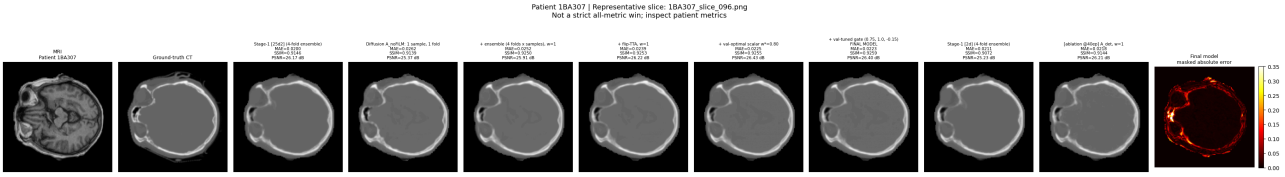
print("\nSaved patient comparison report:", report_path)
print("Finished plotting requested and best available patients.")

if not globals().get("HU_MAPPING_VERIFIED", False):
    print(
        "Note: Any bone-threshold metrics remain provisional until "
        "the PNG-to-HU mapping is verified."
    )
```

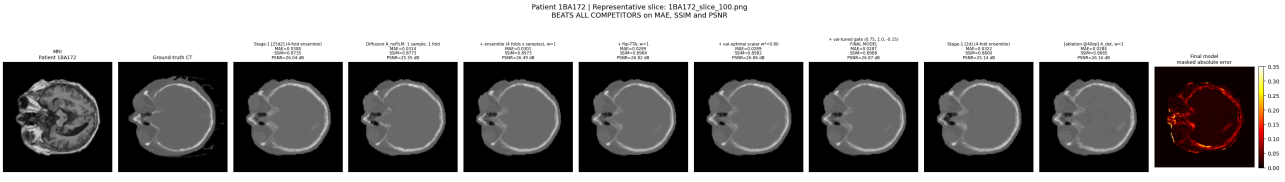
```
Final model: + val-tuned gate (0.75, 1.0, -0.15)
Stage-1 baseline: Stage-1 [25d2] (4-fold ensemble)
Test patients: ['1BA054', '1BA097', '1BA172', '1BA234', '1BA256', '1BA30
7']

PATIENT-LEVEL PERFORMANCE
patient_id  NewModel_MAE  NewModel_SSIM  NewModel_PSNR  Stage1_MAE  Stage
1_SSIM  Stage1_PSNR  MAE_improved_vs_Stage1  SSIM_improved_vs_Stage1  PSN
R_improved_vs_Stage1  Competitors_beaten_on_MAE  Competitors_beaten_on_SS
IM  Competitors_beaten_on_PSNR  Strict_win_over_all_variants  Total_metri
c_wins
    1BA054        0.0272         0.8864        26.2898      0.0306
0.8641      25.1473                    True                      True
True                           7                           6
7                           False                  20
    1BA097        0.0288         0.9080        25.1060      0.0327
0.8851      24.1111                    True                      True
True                           7                           6
6                           False                  19
    1BA172        0.0302         0.8859        25.4487      0.0322
0.8631      24.5190                    True                      True
True                           7                           7
7                            True                  21
    1BA234        0.0239         0.9222        26.0135      0.0265
0.9087      25.0862                    True                      True
True                           5                           6
6                           False                  17
    1BA256        0.0227         0.9040        27.3527      0.0262
0.8838      26.2784                    True                      True
True                           7                           7
7                            True                  21
    1BA307        0.0222         0.9265        28.0642      0.0263
0.9071      26.5316                    True                      True
True                           6                           6
6                           False                  18

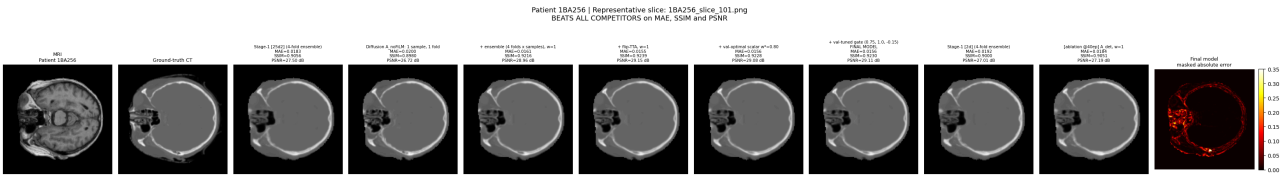
Patients selected for visual inspection: ['1BA307', '1BA172', '1BA256']
```




```
Saved: /kaggle/working/synthrad_ccr_v3/figures/patient_1BA307_all_model_c
omparison.png
```




```
Saved: /kaggle/working/synthrad_ccr_v3/figures/patient_1BA172_all_model_c
omparison.png
```




```
Saved: /kaggle/working/synthrad_ccr_v3/figures/patient_1BA256_all_model_c
omparison.png

Saved patient comparison report: /kaggle/working/synthrad_ccr_v3/figures/
patient_level_all_model_comparison.csv
Finished plotting requested and best available patients.
Note: Any bone-threshold metrics remain provisional until the PNG-to-HU m
apping is verified.
```

In [26…

```python
# ============================================================================
# STEP 12 (UPDATED): PATIENT-WISE WINNERS + 1BA234 + BONE DICE + ZOOM
# Run after FINAL TEST and metric/inference cells.
#
# Outputs:
# 1. Patient-wise metrics for every model.
# 2. Patients where the main model beats ALL competitors on every
#    selected metric.
# 3. Separate detailed visualization for patient 1BA234.
# 4. Readable horizontal metric charts with values and model names.
# 5. Bone Dice (PROVISIONAL), NCC, MAE, SSIM, PSNR and Score.
# 6. Zoom into the region where the main model improves most.
# 7. CSV reports and high-resolution PNGs.
# ============================================================================

import os
import textwrap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from skimage.metrics import structural_similarity as compute_ssim
from IPython.display import display

# -------------------------- 1. Validate ------------------------------

required = ["preds", "t_gt", "t_mask", "t_pid", "TESTd"]
missing = [x for x in required if x not in globals()]
if missing:
    raise RuntimeError(
        f"Missing variables: {missing}. Run final inference/test cells fi
    )

preds = {str(k): np.asarray(v) for k, v in preds.items()}
gt = np.asarray(t_gt)
masks = np.asarray(t_mask)
patient_ids = np.asarray([str(x) for x in t_pid])

if gt.ndim == 4 and gt.shape[1] == 1:
    gt = gt[:, 0]
if masks.ndim == 4 and masks.shape[1] == 1:
```

```
    masks = masks[:, 0]

N = len(patient_ids)
if len(gt) != N or len(masks) != N:
    raise ValueError("t_gt, t_mask and t_pid must contain the same slices

for name, arr in preds.items():
    if len(arr) != N:
        raise ValueError(
            f"Prediction '{name}' has {len(arr)} slices; expected {N}."
        )
    if arr.ndim == 4 and arr.shape[1] == 1:
        preds[name] = arr[:, 0]

# ---------------------- 2. Identify main variant --------------------

main_name = None

if globals().get("final_key") in preds:
    main_name = globals()["final_key"]

if main_name is None:
    preferred_terms = [
        "val-tuned gate",
        "validation-tuned gate",
        "tuned gate",
        "tuned blend",
    ]
    for term in preferred_terms:
        matches = [k for k in preds if term.lower() in k.lower()]
        if matches:
            main_name = matches[-1]
            break

if main_name is None:
    raise RuntimeError(
        "Could not identify the main model automatically. "
        "Set main_name manually to the exact key in preds, for example:\n
        "main_name = '+ val-tuned gate (0.5, 0.75, -0.15)'"
    )

baseline_candidates = [k for k in preds if k.startswith("Stage-1")]
baseline_name = baseline_candidates[0] if baseline_candidates else None

competitors = [k for k in preds if k != main_name]
if not competitors:
    raise RuntimeError("At least one competing prediction variant is requ

# Keep model ordering consistent across all figures.
variant_names = list(preds.keys())

plot_dir = globals().get(
    "DIR_FIG",
    os.path.join(
```

```
        globals().get("DIR_METRICS", "/kaggle/working"),
        "figures"
    )
)
os.makedirs(plot_dir, exist_ok=True)

metrics_dir = globals().get(
    "DIR_METRICS",
    os.path.join(os.path.dirname(plot_dir), "metrics")
)
os.makedirs(metrics_dir, exist_ok=True)

print("=" * 90)
print("PATIENT-WISE ALL-MODEL EVALUATION")
print("Main model:", main_name)
print("Competitors:", len(competitors))
print("Test patients:", len(set(patient_ids)))
print("Output figures:", plot_dir)
print("Output metrics:", metrics_dir)

# -------------------- 3. Metric calculation --------------------------
# Images are assumed normalized to [-1, 1].
# Bone Dice threshold is provisional until PNG-to-HU calibration
# and original image geometry are verified.

HU_MIN_LOCAL = float(globals().get("HU_MIN", -1000.0))
HU_MAX_LOCAL = float(globals().get("HU_MAX", 1500.0))
BONE_HU_THRESHOLD = 300.0

if HU_MAX_LOCAL <= HU_MIN_LOCAL:
    raise ValueError("HU_MAX must be greater than HU_MIN.")

bone_threshold_norm = (
    2.0 * (BONE_HU_THRESHOLD - HU_MIN_LOCAL)
    / (HU_MAX_LOCAL - HU_MIN_LOCAL) - 1.0
)

def calculate_metrics(pred, target, mask):
    p_norm = np.clip(np.asarray(pred).squeeze(), -1, 1)
    g_norm = np.clip(np.asarray(target).squeeze(), -1, 1)
    valid = np.asarray(mask).squeeze() > 0.5

    # Convert normalized values to [0, 1] for common image metrics.
    p = (p_norm + 1.0) / 2.0
    g = (g_norm + 1.0) / 2.0

    if not np.any(valid):
        return {
            "MAE": np.nan, "SSIM": np.nan, "PSNR": np.nan,
            "NCC": np.nan, "Bone_Dice_provisional": np.nan,
            "Score": np.nan
        }

    diff = p[valid] - g[valid]
```

```
    mae = float(np.mean(np.abs(diff)))
    mse = float(np.mean(diff ** 2))
    psnr = float(10 * np.log10(1.0 / max(mse, 1e-12)))

    _, ssim_map = compute_ssim(
        g, p, data_range=1.0, full=True
    )
    ssim = float(np.mean(ssim_map[valid]))

    pv = p[valid].astype(np.float64)
    gv = g[valid].astype(np.float64)
    pv -= pv.mean()
    gv -= gv.mean()
    denom = np.sqrt(np.sum(pv ** 2) * np.sum(gv ** 2))
    ncc = float(np.sum(pv * gv) / denom) if denom > 1e-12 else np.nan

    pred_bone = (p_norm >= bone_threshold_norm) & valid
    gt_bone = (g_norm >= bone_threshold_norm) & valid
    dice_denom = int(pred_bone.sum() + gt_bone.sum())
    bone_dice = (
        float(2 * np.logical_and(pred_bone, gt_bone).sum() / dice_denom)
        if dice_denom > 0 else 1.0
    )

    score = mae + 0.25 * (1.0 - ssim)

    return {
        "MAE": mae,
        "SSIM": ssim,
        "PSNR": psnr,
        "NCC": ncc,
        "Bone_Dice_provisional": bone_dice,
        "Score": score,
    }

# -------------------- 4. Evaluate every slice ------------------------

slice_rows = []
for variant, prediction in preds.items():
    for i in range(N):
        vals = calculate_metrics(prediction[i], gt[i], masks[i])
        slice_rows.append({
            "patient": patient_ids[i],
            "slice_index": i,
            "variant": variant,
            **vals
        })

slice_df = pd.DataFrame(slice_rows)

# Patient macro-level metrics: average slices within each patient first.
patient_df = (
    slice_df.groupby(["patient", "variant"], as_index=False)
    .agg({
```

```
        "MAE": "mean",
        "SSIM": "mean",
        "PSNR": "mean",
        "NCC": "mean",
        "Bone_Dice_provisional": "mean",
        "Score": "mean",
    })
)

patient_df.to_csv(
    os.path.join(metrics_dir, "step12_patient_metrics_all_variants.csv"),
    index=False
)
slice_df.to_csv(
    os.path.join(metrics_dir, "step12_slice_metrics_all_variants.csv"),
    index=False
)

# Overall results use equal weighting per patient, rather than allowing
# patients with more slices to dominate the summary.
overall_df = (
    patient_df.groupby("variant", as_index=False)
    .agg({
        "MAE": "mean",
        "SSIM": "mean",
        "PSNR": "mean",
        "NCC": "mean",
        "Bone_Dice_provisional": "mean",
        "Score": "mean",
    })
)

print("\nOVERALL PATIENT-MACRO RESULTS")
display(overall_df.round(5))

# -------------------- 5. Strict patient winners ----------------------
# Strict win means better on ALL metrics below versus EVERY competitor.
# MAE and Score: lower is better.
# SSIM, PSNR, NCC and Bone Dice: higher is better.

metric_directions = {
    "MAE": "lower",
    "SSIM": "higher",
    "PSNR": "higher",
    "NCC": "higher",
    "Bone_Dice_provisional": "higher",
    "Score": "lower",
}

patient_win_rows = []
strict_winner_patients = []

for patient in sorted(set(patient_ids)):
    subset = patient_df[patient_df["patient"] == patient]
```

```
    indexed = subset.set_index("variant")

    if main_name not in indexed.index:
        continue

    main_vals = indexed.loc[main_name]
    competitor_rows = indexed.drop(index=main_name, errors="ignore")

    wins = {}
    all_metrics_win = True

    for metric, direction in metric_directions.items():
        if metric not in competitor_rows.columns or competitor_rows.empty
            wins[metric] = False
            all_metrics_win = False
            continue

        mv = float(main_vals[metric])
        cv = pd.to_numeric(competitor_rows[metric], errors="coerce").to_n
        cv = cv[np.isfinite(cv)]

        if not np.isfinite(mv) or len(cv) != len(competitor_rows):
            wins[metric] = False
        elif direction == "lower":
            wins[metric] = bool(np.all(mv < cv - 1e-10))
        else:
            wins[metric] = bool(np.all(mv > cv + 1e-10))

        all_metrics_win = all_metrics_win and wins[metric]

    row = {
        "patient": patient,
        "main_variant": main_name,
        **{f"main_{m}": main_vals[m] for m in metric_directions},
        **{f"strict_win_{m}": wins[m] for m in metric_directions},
        "strict_win_all_metrics_all_competitors": all_metrics_win,
    }
    patient_win_rows.append(row)

    if all_metrics_win:
        strict_winner_patients.append(patient)

winner_patient_df = pd.DataFrame(patient_win_rows)
winner_patient_df.to_csv(
    os.path.join(metrics_dir, "step12_strict_patient_winners.csv"),
    index=False
)

print("\nPATIENTS WHERE THE MAIN MODEL BEATS EVERY VARIANT ON EVERY METRI
if strict_winner_patients:
    display(
        winner_patient_df[
            winner_patient_df["strict_win_all_metrics_all_competitors"]
        ].round(5)
```

```
    )
else:
    print(
        "No patient satisfies the strict all-metric rule. "
        "The code will still visualize the strongest per-patient improvem
        "and patient 1BA234 without falsely declaring a winner."
    )

# -------------------- 6. Correctly labeled charts --------------------
# Horizontal bars, wrapped labels and value annotations outside bars
# avoid overlapping model names and values.

chart_specs = [
    ("MAE", "Masked MAE (lower is better)"),
    ("SSIM", "Masked SSIM (higher is better)"),
    ("PSNR", "Masked PSNR in dB (higher is better)"),
    ("NCC", "Masked NCC (higher is better)"),
    ("Bone_Dice_provisional", "Provisional Bone Dice (higher is better)")
    ("Score", "Composite Score (lower is better)"),
]

fig, axes = plt.subplots(
    3, 2, figsize=(17, 21), constrained_layout=True
)
axes = axes.flatten()

chart_df = overall_df.set_index("variant").reindex(variant_names)

for ax, (metric, title) in zip(axes, chart_specs):
    vals = pd.to_numeric(chart_df[metric], errors="coerce")
    valid = vals.notna() & np.isfinite(vals)
    plot_names = vals.index[valid].tolist()
    plot_vals = vals[valid].to_numpy()

    if len(plot_vals) == 0:
        ax.set_title(title + " — no valid values")
        ax.axis("off")
        continue

    direction = metric_directions[metric]
    order = np.argsort(plot_vals)
    if direction == "higher":
        order = order[::-1]

    plot_vals = plot_vals[order]
    plot_names = [plot_names[i] for i in order]
    y = np.arange(len(plot_names))

    bar_colors = [
        "#2ca02c" if name == main_name else
        "#ff7f0e" if name == baseline_name else
        "#7f9db9"
        for name in plot_names
    ]
```

```
    bars = ax.barh(y, plot_vals, color=bar_colors, alpha=0.9)
    ax.set_yticks(y)
    ax.set_yticklabels(
        [textwrap.fill(name, width=27) for name in plot_names],
        fontsize=8
    )
    ax.invert_yaxis()
    ax.set_title(title, fontsize=11)
    ax.grid(axis="x", alpha=0.25)
    ax.set_axisbelow(True)

    low, high = float(np.min(plot_vals)), float(np.max(plot_vals))
    span = max(high - low, abs(high) * 0.05, 0.01)
    ax.set_xlim(min(0, low - 0.03 * span), high + 0.24 * span)

    for bar, val in zip(bars, plot_vals):
        ax.text(
            bar.get_width() + 0.015 * span,
            bar.get_y() + bar.get_height() / 2,
            f"{val:.4f}",
            va="center", ha="left", fontsize=8
        )

fig.suptitle(
    "Overall test performance — equal-weight patient averages\n"
    "Green = main model | Orange = Stage-1 baseline | Blue = other varian
    fontsize=15
)
chart_path = os.path.join(plot_dir, "step12_all_metrics_readable.png")
fig.savefig(chart_path, dpi=180, bbox_inches="tight")
plt.show()
plt.close(fig)
print("Saved:", chart_path)

# -------------------- 7. MRI extraction helper -----------------------

def get_mri_image(index):
    x = TESTd["mri"][index]
    if hasattr(x, "detach"):
        x = x.detach().cpu().numpy()
    x = np.asarray(x).squeeze()

    if x.ndim == 3:
        offsets = getattr(globals().get("FB", None), "offsets", None)
        if offsets is not None and 0 in offsets:
            center = list(offsets).index(0)
        else:
            center = x.shape[0] // 2
        x = x[center]

    return x

# -------------------- 8. Find best zoom region -----------------------
```

```python
# The crop is selected using the largest positive reduction in absolute
# error: competitor error minus main-model error, restricted to the mask.
# It is selected by error difference, not by visual appearance.

def best_improvement_crop(main_pred, competitor_pred, target, body_mask,
                          crop_size=32):
    main_err = np.abs(
        (np.clip(main_pred, -1, 1) + 1) / 2
        - (np.clip(target, -1, 1) + 1) / 2
    )
    comp_err = np.abs(
        (np.clip(competitor_pred, -1, 1) + 1) / 2
        - (np.clip(target, -1, 1) + 1) / 2
    )
    valid = np.asarray(body_mask) > 0.5
    improvement = (comp_err - main_err) * valid

    H, W = improvement.shape
    crop_size = min(crop_size, H, W)
    if crop_size < 1:
        return (0, H, 0, W), np.nan

    best = (-np.inf, 0, 0)
    for y0 in range(H - crop_size + 1):
        for x0 in range(W - crop_size + 1):
            local_mask = valid[y0:y0+crop_size, x0:x0+crop_size]
            if local_mask.sum() < max(4, 0.1 * crop_size * crop_size):
                continue

            local = improvement[
                y0:y0+crop_size, x0:x0+crop_size
            ]
            score = float(local[local_mask].mean())

            if score > best[0]:
                best = (score, y0, x0)

    if not np.isfinite(best[0]):
        return (0, H, 0, W), np.nan

    _, y0, x0 = best
    return (y0, y0 + crop_size, x0, x0 + crop_size), best[0]

# -------------------- 9. Plot each selected patient ------------------
# Show every model for the central slice of each selected patient.
# 1BA234 is always included separately if present in this test cohort.

all_patients = sorted(set(patient_ids))
if "1BA234" not in all_patients:
    print(
        "\nWARNING: Patient 1BA234 is not in t_pid. "
        "It may not be in the current TEST split/inference output."
    )
```

```
patients_to_plot = list(strict_winner_patients)
if "1BA234" in all_patients and "1BA234" not in patients_to_plot:
    patients_to_plot.append("1BA234")

# If no strict winners, also show patients with the largest mean reductio
# in masked MAE versus the strongest MAE competitor.
if not strict_winner_patients:
    gains = []
    for patient in all_patients:
        sub = patient_df[patient_df["patient"] == patient].set_index("var
        if main_name not in sub.index:
            continue
        main_mae = float(sub.loc[main_name, "MAE"])
        other_maes = sub.drop(index=main_name)["MAE"].dropna()
        if len(other_maes):
            gains.append((patient, float(other_maes.min() - main_mae)))

    for patient, _ in sorted(gains, key=lambda x: x[1], reverse=True)[:3]
        if patient not in patients_to_plot:
            patients_to_plot.append(patient)

# Cap extra figures, but never drop 1BA234.
MAX_PATIENT_FIGURES = 8
if len(patients_to_plot) > MAX_PATIENT_FIGURES:
    selected = [p for p in patients_to_plot if p != "1BA234"]
    patients_to_plot = selected[:MAX_PATIENT_FIGURES - 1]
    if "1BA234" in all_patients:
        patients_to_plot.append("1BA234")

def plot_patient(patient):
    indices = np.flatnonzero(patient_ids == patient)
    if len(indices) == 0:
        print("No slices found for", patient)
        return

    # Deterministic middle slice, not chosen based on performance.
    slice_i = int(indices[len(indices) // 2])
    target = np.asarray(gt[slice_i]).squeeze()
    body = np.asarray(masks[slice_i]).squeeze() > 0.5
    mri = get_mri_image(slice_i)
    main_pred = np.asarray(preds[main_name][slice_i]).squeeze()

    # Find the competitor with the greatest whole-mask MAE disadvantage
    # relative to the main model on this representative slice.
    comp_gains = []
    for name in competitors:
        cp = np.asarray(preds[name][slice_i]).squeeze()
        metric_main = calculate_metrics(main_pred, target, body)
        metric_comp = calculate_metrics(cp, target, body)
        comp_gains.append((name, metric_comp["MAE"] - metric_main["MAE"])

    best_comp_name = max(comp_gains, key=lambda x: x[1])[0]
    best_comp_pred = np.asarray(preds[best_comp_name][slice_i]).squeeze()
```

```
    crop, crop_gain = best_improvement_crop(
        main_pred, best_comp_pred, target, body, crop_size=32
    )
    y0, y1, x0, x1 = crop

    # Row 1: input and every prediction; Row 2: masked error maps.
    ncols = len(variant_names) + 2
    fig, axes = plt.subplots(
        3, ncols, figsize=(max(21, 3.0*ncols), 12),
        squeeze=False
    )

    axes[0, 0].imshow(mri, cmap="gray")
    axes[0, 0].set_title(f"MRI\n{patient}", fontsize=9)
    axes[0, 0].axis("off")

    axes[0, 1].imshow(target, cmap="gray", vmin=-1, vmax=1)
    axes[0, 1].set_title("Ground-truth CT", fontsize=9)
    axes[0, 1].axis("off")

    for j, name in enumerate(variant_names, start=2):
        p = np.asarray(preds[name][slice_i]).squeeze()
        m = calculate_metrics(p, target, body)

        axes[0, j].imshow(p, cmap="gray", vmin=-1, vmax=1)
        axes[0, j].set_title(
            f"{textwrap.fill(name, 24)}\n"
            f"MAE {m['MAE']:.4f} | SSIM {m['SSIM']:.4f}\n"
            f"PSNR {m['PSNR']:.2f} dB\n"
            f"Bone Dice* {m['Bone_Dice_provisional']:.3f}",
            fontsize=8
        )
        axes[0, j].axis("off")

    axes[1, 0].imshow(body, cmap="gray")
    axes[1, 0].set_title("Evaluation mask", fontsize=9)
    axes[1, 0].axis("off")
    axes[1, 1].axis("off")
    axes[1, 1].text(
        0.02, 0.95,
        "Masked absolute error\n"
        "Same slice for all models.\n"
        "Lower error is better.\n\n"
        "Bone Dice* is provisional.\n"
        "It is not clinically verified\n"
        "without validated HU calibration.",
        va="top", fontsize=9, wrap=True
    )

    for j, name in enumerate(variant_names, start=2):
        p = np.asarray(preds[name][slice_i]).squeeze()
        err = np.abs(
            (np.clip(p, -1, 1) + 1) / 2
            - (np.clip(target, -1, 1) + 1) / 2
```

```
        ) * body
        axes[1, j].imshow(err, cmap="hot", vmin=0, vmax=0.35)
        axes[1, j].set_title(f"{textwrap.fill(name, 24)}\nError map", fon
        axes[1, j].axis("off")

    # Zoom comparison: GT, main model, strongest MAE competitor,
    # and their absolute-error maps in the best improvement crop.
    zoom_panels = [
        ("GT CT", target),
        (f"Main: {main_name}", main_pred),
        (f"Competitor: {best_comp_name}", best_comp_pred),
    ]

    main_err = np.abs(
        (np.clip(main_pred, -1, 1) + 1) / 2
        - (np.clip(target, -1, 1) + 1) / 2
    )
    comp_err = np.abs(
        (np.clip(best_comp_pred, -1, 1) + 1) / 2
        - (np.clip(target, -1, 1) + 1) / 2
    )
    zoom_panels += [
        ("Main absolute error", main_err * body),
        ("Competitor absolute error", comp_err * body),
        ("Error reduction (positive = main better)", (comp_err-main_err)
    ]

    for j in range(ncols):
        axes[2, j].axis("off")

    # Put the zoom panels into the available columns, starting at column
    for j, (title, image) in enumerate(zoom_panels):
        ax = axes[2, j]
        ax.clear()
        ax.imshow(
            image[y0:y1, x0:x1],
            cmap="gray" if j < 3 else ("hot" if j < 5 else "coolwarm"),
            vmin=(-1 if j < 3 else None),
            vmax=(1 if j < 3 else None),
            interpolation="nearest"
        )
        ax.set_title(textwrap.fill(title, 26), fontsize=8)
        ax.axis("off")

    fig.suptitle(
        f"Patient {patient} — slice {slice_i} | Main: {main_name}\n"
        f"Zoom region: rows {y0}:{y1}, columns {x0}:{x1} | "
        f"Main-vs-competitor mean error improvement={crop_gain:.5f}",
        fontsize=13
    )
    fig.tight_layout(rect=[0, 0, 1, 0.94])

    safe_patient = "".join(c if c.isalnum() or c in "-_" else "_" for c i
    out_path = os.path.join(
```

```
        plot_dir, f"step12_patient_{safe_patient}_all_models_zoom.png"
    )
    fig.savefig(out_path, dpi=180, bbox_inches="tight")
    plt.show()
    plt.close(fig)
    print("Saved patient figure:", out_path)

for patient in patients_to_plot:
    print("\n" + "-" * 80)
    print("VISUALIZATION:", patient)
    print(
        "Strict all-metric winner:",
        bool(
            winner_patient_df.loc[
                winner_patient_df["patient"] == patient,
                "strict_win_all_metrics_all_competitors"
            ].any()
        )
    )
    plot_patient(patient)

# -------------------- 10. Separate 1BA234 table ----------------------

if "1BA234" in all_patients:
    patient_234 = patient_df[patient_df["patient"] == "1BA234"].copy()
    patient_234 = patient_234.set_index("variant").reindex(variant_names)
    patient_234 = patient_234.rename(columns={"index": "variant"})

    print("\n" + "=" * 90)
    print("PATIENT 1BA234 — SEPARATE ALL-VARIANT METRIC TABLE")
    print("=" * 90)
    display(patient_234.round(5))

    patient_234.to_csv(
        os.path.join(metrics_dir, "step12_1BA234_all_variant_metrics.csv"
        index=False
    )

    # Main-model deltas against every other variant on 1BA234.
    row_main = patient_234[patient_234["variant"] == main_name].iloc[0]
    comparison_rows = []

    for _, row in patient_234.iterrows():
        record = {"variant": row["variant"]}
        for metric, direction in metric_directions.items():
            current = float(row[metric])
            main_value = float(row_main[metric])
            if not np.isfinite(current) or not np.isfinite(main_value):
                record[f"{metric}_delta_vs_main"] = np.nan
                record[f"{metric}_main_is_better"] = False
                continue

            # Positive delta always means the main model is better.
            delta = (
```

```
                current - main_value
                if direction == "lower"
                else main_value - current
            )
            record[f"{metric}_delta_vs_main"] = delta
            record[f"{metric}_main_is_better"] = (
                row["variant"] != main_name and delta > 1e-10
            )

        comparison_rows.append(record)

    comp234_df = pd.DataFrame(comparison_rows)
    display(comp234_df.round(5))
    comp234_df.to_csv(
        os.path.join(metrics_dir, "step12_1BA234_main_vs_all_variants.csv
        index=False
    )

print("\n" + "=" * 90)
print("STEP 12 COMPLETE")
print("Main variant:", main_name)
print("Strict winner patients:", strict_winner_patients)
print("Visualized patients:", patients_to_plot)
print("Figures:", plot_dir)
print("CSV reports:", metrics_dir)
print(
    "\nCAUTION: Bone Dice uses a provisional PNG-intensity-to-HU mapping
    f"of {HU_MIN_LOCAL:g} to {HU_MAX_LOCAL:g} HU. The resulting "
    "300-HU threshold is not a validated clinical bone metric unless "
    "the dataset calibration and spatial geometry have been verified."
)
```

```
==========================================================================================
PATIENT-WISE ALL-MODEL EVALUATION
Main model: + val-tuned gate (0.75, 1.0, -0.15)
Competitors: 7
Test patients: 6
Output figures: /kaggle/working/synthrad_ccr_v3/figures
Output metrics: /kaggle/working/synthrad_ccr_v3/metrics

OVERALL PATIENT-MACRO RESULTS
```

	variant	MAE	SSIM	PSNR	NCC	Bone_Dice_provisional	Score
0	+ ensemble (4 folds x samples), w=1	0.02644	0.90276	26.15710	0.92591	0.86680	0.05075
1	+ flip-TTA, w=1	0.02594	0.90576	26.35798	0.92931	0.87173	0.04950
2	+ val-optimal scalar w*=0.80	0.02598	0.90489	26.35917	0.92936	0.87035	0.04976
3	+ val-tuned gate (0.75, 1.0, -0.15)	0.02584	0.90550	26.37915	0.92959	0.87126	0.04947
4	Diffusion A_noFiLM: 1 sample, 1 fold	0.03062	0.88280	24.86052	0.90237	0.84673	0.05992
5	Stage-1 [25d2] (4-fold ensemble)	0.02908	0.88533	25.27893	0.90979	0.84352	0.05775
6	Stage-1 [2d] (4-fold ensemble)	0.02946	0.88154	25.03039	0.90364	0.84088	0.05908
7	[ablation @40ep] A_det, w=1	0.02874	0.88439	25.31024	0.90962	0.84111	0.05765

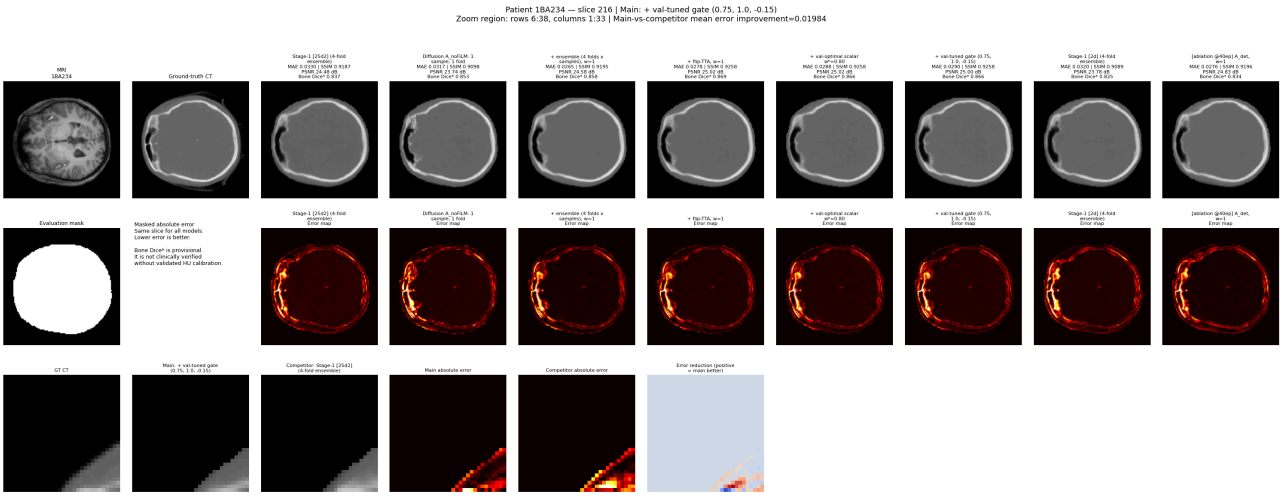
```
PATIENTS WHERE THE MAIN MODEL BEATS EVERY VARIANT ON EVERY METRIC
No patient satisfies the strict all-metric rule. The code will still visu
alize the strongest per-patient improvements and patient 1BA234 without f
alsely declaring a winner.
```



Saved: /kaggle/working/synthrad_ccr_v3/figures/step12_all_metrics_readable.png

---

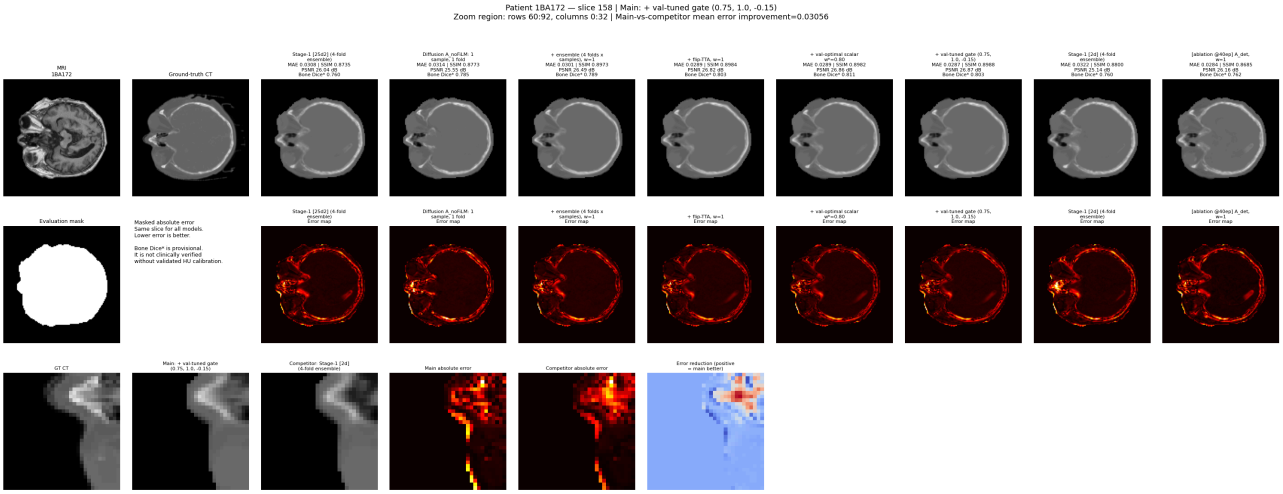
VISUALIZATION: 1BA234
Strict all-metric winner: False



Saved patient figure: /kaggle/working/synthrad_ccr_v3/figures/step12_patient_1BA234_all_models_zoom.png

---

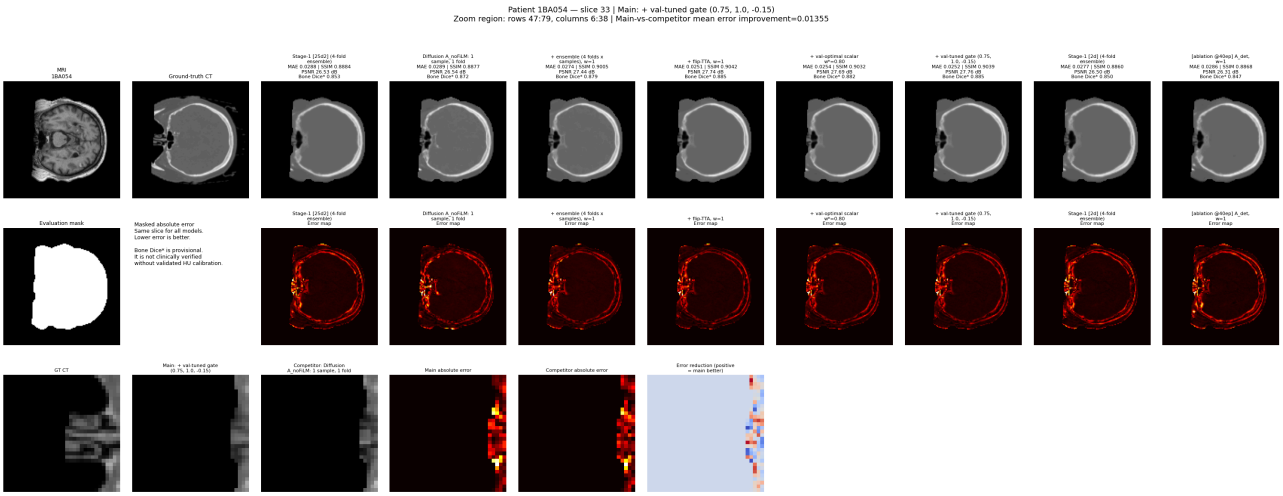
VISUALIZATION: 1BA172
Strict all-metric winner: False



Saved patient figure: /kaggle/working/synthrad_ccr_v3/figures/step12_patient_1BA172_all_models_zoom.png

---

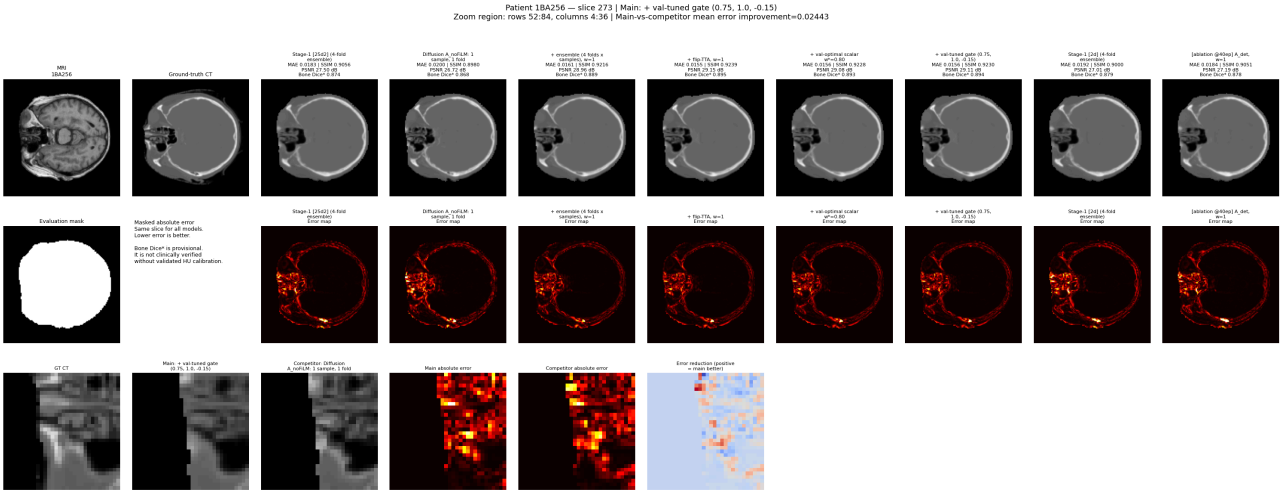
VISUALIZATION: 1BA054
Strict all-metric winner: False



Saved patient figure: /kaggle/working/synthrad_ccr_v3/figures/step12_patient_1BA054_all_models_zoom.png

---

VISUALIZATION: 1BA256
Strict all-metric winner: False



Saved patient figure: /kaggle/working/synthrad_ccr_v3/figures/step12_patient_1BA256_all_models_zoom.png

==========

PATIENT 1BA234 — SEPARATE ALL-VARIANT METRIC TABLE

==========

	variant	patient	MAE	SSIM	PSNR	NCC	Bone_Dice_provisional	Score
0	Stage-1 [25d2] (4-fold ensemble)	1BA234	0.02655	0.90873	25.08615	0.90431	0.84068	0.04937
1	Diffusion A_noFiLM: 1 sample, 1 fold	1BA234	0.02727	0.90632	24.77893	0.89740	0.84876	0.05069
2	+ ensemble (4 folds x samples), w=1	1BA234	0.02314	0.92077	25.96535	0.91966	0.86253	0.04295
3	+ flip-TTA, w=1	1BA234	0.02385	0.92248	26.02164	0.92208	0.86649	0.04323
4	+ val-optimal scalar w*=0.80	1BA234	0.02404	0.92157	25.97069	0.92133	0.86405	0.04365
5	+ val-tuned gate (0.75, 1.0, -0.15)	1BA234	0.02389	0.92217	26.01346	0.92208	0.86524	0.04335
6	Stage-1 [2d] (4-fold ensemble)	1BA234	0.02598	0.90769	24.90337	0.89856	0.83821	0.04906
7	[ablation @40ep] A_det, w=1	1BA234	0.02654	0.90821	25.17858	0.90230	0.83781	0.04949

	variant	MAE_delta_vs_main	MAE_main_is_better	SSIM_delta_vs_main	SSIM_main_is_bette
0	Stage-1 [25d2] (4-fold ensemble)	0.00266	True	0.01345	Tru
1	Diffusion A_noFiLM: 1 sample, 1 fold	0.00338	True	0.01585	Tru
2	+ ensemble (4 folds x samples), w=1	-0.00075	False	0.00140	Tru
3	+ flip-TTA, w=1	-0.00004	False	-0.00031	Fals
4	+ val-optimal scalar w*=0.80	0.00015	True	0.00061	Tru
5	+ val-tuned gate (0.75, 1.0, -0.15)	0.00000	False	0.00000	Fals
6	Stage-1 [2d] (4-fold ensemble)	0.00209	True	0.01449	Tru
7	[ablation @40ep] A_det, w=1	0.00265	True	0.01397	Tru

```
================================================================================
STEP 12 COMPLETE
Main variant: + val-tuned gate (0.75, 1.0, -0.15)
Strict winner patients: []
Visualized patients: ['1BA234', np.str_('1BA172'), np.str_('1BA054'), np.str_('1BA250
Figures: /kaggle/working/synthrad_ccr_v3/figures
CSV reports: /kaggle/working/synthrad_ccr_v3/metrics

CAUTION: Bone Dice uses a provisional PNG-intensity-to-HU mapping of -1000 to 1500 HU
resulting 300-HU threshold is not a validated clinical bone metric unless the dataset
bration and spatial geometry have been verified.
```

In [27…

```python
# %% ====== [11] OPTIONAL: extra seeds for the final config (variance for
seed_rows = []
for sd in EXTRA_SEEDS:
    cfg_s = dict(FINAL_CFG); cfg_s["seed"] = sd
    m_s = get_stage2(f"{FINAL_TAG}_seed{sd}", cfg_s, FINAL_EPOCHS)
    sv = predict_stack(VALd, m_s, cfg_s, FB, P["samples"], P["steps"], fo
    c, r = aggregate(sv); seed_rows.append(summarize_pred(f"seed {sd} (va
```

```
if seed_rows:
    st_ = pd.DataFrame(seed_rows); print(st_.round(5).to_string(index=Fal

# archive small deliverables (no weights)
zp = os.path.join(BASE_OUT_DIR, "ccr_v3_deliverables.zip")
with zipfile.ZipFile(zp, "w", zipfile.ZIP_DEFLATED) as z:
    for root, _, fs in os.walk(BASE_OUT_DIR):
        for f in fs:
            if f.endswith((".csv", ".png", ".json")):
                z.write(os.path.join(root, f), os.path.relpath(os.path.jo
print("Archived:", zp)
```

```
Archived: /kaggle/working/synthrad_ccr_v3/ccr_v3_deliverables.zip
```

Combined CCR-Diffusion v3 + Baseline Evaluation Notebook

The original `ccr-diffusion-v3-ablation` notebook cells and their saved outputs have been retained in their original order. The CCR v3 baselines add-on from `ccr_baselines_v3_preprocessing.ipynb.py` is appended at the end as separate cells.

Safe run order after a Kaggle session restart

1. Run the appended **RESTORE CHECKPOINTS / RESULTS** cell first (select it and press Shift+Enter). It can be run before the earlier notebook cells.
2. Run the original setup/definition cells headed `[0] CONFIG & IMPORTS`, `[1] NUMPY-SIDE UTILITIES`, `[2] DATA`, `[3] MODELS`, `[4] STAGE-1`, and `[5] STAGE-2`. Do **not** click Run All on the whole notebook; the original cells after these headings also include long training and final-test steps.
3. Run the appended **COMPATIBILITY PATCH** cell. This makes `get_stage2()` load a saved best EMA checkpoint directly when the matching `_latest.pth` resume-state file is absent, rather than unnecessarily starting training for inference.
4. Run appended baseline cells `[B0]` through `[B7]` in order. Cell `[B5]` trains/loads additional baselines; those are new experiments and may take substantial GPU time.
5. The existing SynthRAD image dataset discovery is unchanged. The original notebook resolves `DATASET_ROOT` from `/kaggle/input/.../synthrad_png`. The experiment output and restored checkpoints live under `/kaggle/working/synthrad_ccr_v3`.

The restore cell only copies missing files; it does not overwrite existing working files. It searches for the complete checkpoint backup, not only the small CSV/PNG deliverables archive. If only `ccr_v3_deliverables` is mounted, the checkpoint weights cannot be recovered from that archive.

In [19…

```
# ======================================================================
# RESTORE CHECKPOINTS / RESULTS FROM KAGGLE INPUT DATASETS
# IMPORTANT: After a kernel/session restart, run THIS cell first,
# even though it is located at the end of this notebook.
#
# It searches the added Kaggle input datasets for either:
#   - an extracted synthrad_ccr_v3_backup folder, or
#   - a ZIP containing s1/, s2/, metrics/, figures/ and baseline artifact
# Existing files are not overwritten. Missing files are restored to:
# /kaggle/working/synthrad_ccr_v3
#
# The normal SynthRAD dataset discovery in the original notebook is uncha
# ======================================================================

from pathlib import Path
import os, shutil, zipfile, posixpath

RESTORE_TARGET = Path("/kaggle/working/synthrad_ccr_v3")
RESTORE_TARGET.mkdir(parents=True, exist_ok=True)
RESTORE_FOLDERS = {"s1", "s2", "metrics", "figures", "baselines"}
FINAL_BEST_NAME = "s2_final_A_noFiLM_e120_best.pth"

def _has_checkpoint_root(candidate):
    candidate = Path(candidate)
    return (
        (candidate / "s1").is_dir()
        and (candidate / "s2").is_dir()
        and any((candidate / "s2").glob("s2_final_A_noFiLM_e120_best.pth"
    )

def _find_extracted_roots(search_root):
    found = []
    if not Path(search_root).exists():
        return found
    for root, dirs, files in os.walk(search_root):
        # The final best checkpoint is the anchor that distinguishes a mo
        # backup from a deliverables-only archive of PNG/CSV files.
        if FINAL_BEST_NAME in files and Path(root).name == "s2":
            candidate = Path(root).parent
            if _has_checkpoint_root(candidate):
                found.append(candidate)
        # Skip common unrelated large folders where possible.
        dirs[:] = [d for d in dirs if d not in {".git", "__pycache__"}]
    # Prefer the most explicit backup-like root, then the shortest path.
    return sorted(set(found), key=lambda p: (0 if "backup" in str(p).lowe

def _copy_missing_tree(src_root, dst_root):
```

```
    src_root, dst_root = Path(src_root), Path(dst_root)
    copied, skipped = [], []
    for folder in RESTORE_FOLDERS:
        src = src_root / folder
        if not src.is_dir():
            continue
        for src_file in src.rglob("*"):
            if not src_file.is_file():
                continue
            rel = src_file.relative_to(src_root)
            dst_file = dst_root / rel
            if dst_file.exists() and dst_file.stat().st_size > 0:
                skipped.append(str(rel))
                continue
            dst_file.parent.mkdir(parents=True, exist_ok=True)
            shutil.copy2(src_file, dst_file)
            copied.append(str(rel))
    return copied, skipped

restored_files, left_existing = [], []
sources = _find_extracted_roots("/kaggle/input")
sources += [p for p in _find_extracted_roots("/kaggle/working") if p != R
sources = list(dict.fromkeys(p.resolve() for p in sources if p.resolve()

if sources:
    print("Extracted checkpoint backup(s) found:")
    for src in sources:
        print(" -", src)
        copied, skipped = _copy_missing_tree(src, RESTORE_TARGET)
        restored_files.extend(copied)
        left_existing.extend(skipped)
else:
    print("No extracted checkpoint backup with the final best model was f

# If the backup is still a ZIP file, restore only its experiment director
# The archive may contain either direct paths (s1/...) or a wrapper direc
# (for example synthrad_ccr_v3_backup/s1/...).
zip_candidates = []
for search_root in [Path("/kaggle/input"), Path("/kaggle/working")]:
    if search_root.exists():
        for candidate in search_root.rglob("*.zip"):
            if "deliverable" in candidate.name.lower():
                continue
            zip_candidates.append(candidate)

for zpath in sorted(set(zip_candidates), key=lambda p: (0 if "backup" in
    try:
        with zipfile.ZipFile(zpath, "r") as zf:
            names = zf.namelist()
            anchor = None
            prefix = None
            for member in names:
                parts = Path(member).parts
                for j in range(len(parts) - 1):
```

```
                    if parts[j] == "s2" and parts[j + 1] == FINAL_BEST_NA
                        anchor = member
                        prefix = parts[:j]
                        break
                if anchor:
                    break
            if not anchor:
                continue

            print("Checkpoint ZIP found:", zpath)
            for member in names:
                if member.endswith("/"):
                    continue
                parts = Path(member).parts
                if parts[:len(prefix)] != prefix:
                    continue
                rel_parts = parts[len(prefix):]
                if not rel_parts or rel_parts[0] not in RESTORE_FOLDERS:
                    continue
                # Reject unsafe/invalid archive paths.
                if any(part in ("..", "") for part in rel_parts):
                    continue
                dst_file = RESTORE_TARGET.joinpath(*rel_parts)
                if dst_file.exists() and dst_file.stat().st_size > 0:
                    continue
                dst_file.parent.mkdir(parents=True, exist_ok=True)
                with zf.open(member, "r") as source, open(dst_file, "wb")
                    shutil.copyfileobj(source, dest)
                restored_files.append(str(Path(*rel_parts)))
            # A matching full backup ZIP was found; no need to scan other
            break
    except (zipfile.BadZipFile, OSError, PermissionError) as exc:
        print("Skipped unreadable archive:", zpath, "|", exc)

print(f"\nRestored missing files: {len(restored_files)}")
print(f"Existing files preserved: {len(left_existing)}")

must_have = [
    RESTORE_TARGET / "s2" / "s2_final_A_noFiLM_e120_best.pth",
    RESTORE_TARGET / "s1" / "s1_25d2_fold1.pth",
    RESTORE_TARGET / "metrics" / "ablation_stage2_val_e40.csv",
]
for f in must_have:
    print(("OK    " if f.is_file() and f.stat().st_size > 0 else "MISSING

if not (RESTORE_TARGET / "s2" / "s2_final_A_noFiLM_e120_best.pth").is_fil
    print(
        "\nWARNING: the final best checkpoint was not found in the workin
        "or the mounted input backup. A results-only deliverables ZIP is
        "to reconstruct model weights. Do not run expensive cells until t
        "synthrad_ccr_v3_backup archive/folder is attached as a Kaggle in
    )
else:
    print("\nCheckpoint restore/check completed. Output directory:", REST
```

```
Extracted checkpoint backup(s) found:
 - /kaggle/input/datasets/lohithkply/final-study-model-comparisions/synth
rad_ccr_v3_backup
 - /kaggle/input/datasets/lohithkply/final-study-model-comparisions/Synth
RAD_CCR_v3_Complete_Resumed_Results
Checkpoint ZIP found: /kaggle/working/Kaggle_Working_Full_Backup.zip

Restored missing files: 0
Existing files preserved: 131
OK     /kaggle/working/synthrad_ccr_v3/s2/s2_final_A_noFiLM_e120_best.pth
OK     /kaggle/working/synthrad_ccr_v3/s1/s1_25d2_fold1.pth
OK     /kaggle/working/synthrad_ccr_v3/metrics/ablation_stage2_val_e40.cs
v

Checkpoint restore/check completed. Output directory: /kaggle/working/syn
thrad_ccr_v3
```

In [20…

```
# ======================================================================
# CCR v3 -- BASELINES ADD-ON
#   (1) deterministic refiner "A_det" trained for 120 epochs (matched to
#   (2) naive conditional DDPM   (MRI channel-concatenation, direct CT ou
#   (3) pix2pix                  (ResUNet generator + PatchGAN, L1 + LSGA
#   (4) plain ResUNet            (single model, all 48 training patients;
#   + one unified comparison table (val + test), paired statistics, quali
#
# HOW TO RUN
#   1. In your v3 notebook run cells [0]-[5] of ccr_diffusion_v3_ablation
#      (definitions + data preprocessing; they do not train anything).
#   2. Paste each "# %%" block of THIS file as its own cell and run in or
#   Your Stage-1 '25d2' checkpoints and the final diffusion checkpoint ar
#   (/kaggle/working/synthrad_ccr_v3/...). If they are missing they are r
#
# FAIRNESS RULES (state these in the paper)
#   * every learned baseline sees the same input information: MRI slices
#     (the 2D ResUNet gets the centre slice only and is reported as a sep
#   * all models train on the same 48 patients; hyper-parameters / checkp
#     validation patients; the test patients are touched once, in the las
#   * the table reports parameters and network evaluations (NFE) per slic
#   * the pix2pix generator is the same ResUNet as Stage-1, so GAN vs L1
#   * the naive cDDPM uses the same UNet backbone, schedule and v-predict
#     only differences are: no coarse CT, no residual target, no Stage-1
# ======================================================================
```

In [21…

```
# ======================================================================
# COMPATIBILITY PATCH: INFERENCE FROM THE SAVED BEST EMA CHECKPOINT
# Run after the original model-definition cells [0]-[5] have been run,
# but before baseline cell [B0].
#
# Why: the original get_stage2() function starts training when _latest.pt
# is missing, even if the corresponding _best.pth exists. For evaluation,
# a best EMA checkpoint is sufficient. It does not restore optimizer stat
# and therefore is NOT a training-resume checkpoint.
# ======================================================================
```

```
if "get_stage2" not in globals() or "DiffUNet" not in globals() or "_path
    raise RuntimeError(
        "Run the original setup/definition cells [0]-[5] first, then run
    )

_original_get_stage2_for_baselines = get_stage2

def get_stage2(name, cfg, epochs, which="best"):
    latest_path, best_path = _paths(name, epochs)

    # Preserve the original normal path when a latest/resume checkpoint e
    if os.path.isfile(latest_path):
        return _original_get_stage2_for_baselines(name, cfg, epochs, whic

    # If only best EMA weights exist, load them directly for inference/ev
    if os.path.isfile(best_path) and which == "best":
        bundle = get_bundle(cfg["s1"])
        in_ch = 2 + int(cfg["coarse_ch"]) + (len(bundle.offsets) if cfg["
        model = DiffUNet(
            in_ch, cfg["base_ch"], T_DIM, bundle.cond_ch, film=cfg["film"
        ).to(device)
        checkpoint = torch.load(best_path, map_location=device, weights_o
        state = checkpoint.get("ema", checkpoint.get("model"))
        if state is None:
            raise RuntimeError(f"No EMA/model weights found in {best_path
        model.load_state_dict(state)
        print(
            f"[best-checkpoint fallback] Loaded {best_path}. "
            "No training was launched; optimizer/latest state is unavaila
        )
        return model.eval()

    # No usable best checkpoint: retain original behavior (may train a ne
    return _original_get_stage2_for_baselines(name, cfg, epochs, which=wh

print("Compatibility patch installed. Saved best EMA weights can be used
```

```
Compatibility patch installed. Saved best EMA weights can be used when _l
atest.pth is missing.
```

In [22…

```
# %% ====== [B0] CHECKS + CONFIG ====================================
import os, json, time, math, copy, random
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
from torch.utils.data import DataLoader

_need = ["get_split", "SplitDataset", "Stage1ResUNet", "DiffUNet", "sched
         "patient_level", "paired_stats", "predict_stack", "aggregate", "
         "make_cfg", "get_stage2", "ABLATIONS", "S1_VARIANTS", "TRAIN_SNA
         "iter_batches", "seed_everything", "BASE_OUT_DIR", "FINAL_EPOCHS
```

```
        "S1_BASE", "BATCH_SIZE", "NUM_WORKERS", "device", "use_amp", "IM
_missing = [n for n in _need if n not in globals()]
assert not _missing, f"Run cells [0]-[5] of ccr_diffusion_v3_ablation.py

BL_S1 = "25d2"                                  # Stage-1 variant used b
BL_OFFSETS = S1_VARIANTS[BL_S1]                 # (-2,-1,0,1,2)
BL_TO_RUN = ["det", "resunet_25d", "resunet_2d", "pix2pix", "cddpm_v"]

DET_FILM = False                                # match the final model
DET_EPOCHS = FINAL_EPOCHS
DET_SEEDS = (SEED,)                             # e.g. (42, 43, 44, 45)

RESUNET_EPOCHS, RESUNET_LR = 60, 1e-4
P2P_EPOCHS, P2P_LR, P2P_L1 = 100, 2e-4, 100.0
CDDPM_EPOCHS, CDDPM_LR = FINAL_EPOCHS, 2e-4
CDDPM_SAMPLES, CDDPM_STEPS = 16, 50              # x2 flips = 32 samples,
BL_VAL_FREQ = 5                                 # validation (checkpoint
EVAL_VAL, EVAL_TEST = True, True

BL_DIR = os.path.join(BASE_OUT_DIR, "baselines")
os.makedirs(BL_DIR, exist_ok=True)

FINAL_NAME_ = globals().get("FINAL_NAME") or "A_noFiLM"
FINAL_CFG_ = globals().get("FINAL_CFG") or make_cfg(ABLATIONS[FINAL_NAME_
FINAL_TAG_ = globals().get("FINAL_TAG") or (FINAL_NAME_ if ABL_EPOCHS ==

_V0 = get_split("val", VAL_SNAMES, (0,))
BV_GT, BV_MASK, BV_PID = _V0["ct"][:, 0].numpy(), _V0["mask"][:, 0].numpy
print("Baselines to run:", BL_TO_RUN, "| final diffusion:", FINAL_TAG_, F
```

```
Baselines to run: ['det', 'resunet_25d', 'resunet_2d', 'pix2pix', 'cddp
m_v'] | final diffusion: final_A_noFiLM 25d2
```

In [23…

```
# %% ====== [B1] SHARED UTILITIES ======================================
def nparams(m):
    return sum(p.numel() for p in m.parameters()) / 1e6


def _ld(path):
    return torch.load(path, map_location=device, weights_only=False)


def _done(name):
    return os.path.join(BL_DIR, f"{name}.done")


def _mark_done(name):
    open(_done(name), "w").write("ok")


@torch.no_grad()
def predict_direct(model, d, flips=(False,), bs=32):
    """Direct-output networks (ResUNet / pix2pix G). Optional flip-TTA. B
    model.eval(); outs = []
    for mri, ct, mask, sl in iter_batches(d, bs):
```

```
        acc = 0
        for fl in flips:
            y = model(mri.flip(-1) if fl else mri)
            acc = acc + (y.flip(-1) if fl else y)
        y = torch.where(mask > 0.5, (acc / len(flips)).clamp(-1, 1), torc
        outs.append(y[:, 0].cpu().numpy())
    return np.concatenate(outs)


def direct_val_score(model, d_val):
    m, _ = patient_level(predict_direct(model, d_val), BV_GT, BV_MASK, BV
    return m["Score"], m
```

In [24…

```
# %% ====== [B2] PLAIN ResUNet BASELINE (single model, all 48 training pa
def train_resunet(name, offsets, epochs=RESUNET_EPOCHS, lr=RESUNET_LR):
    best_p = os.path.join(BL_DIR, f"{name}_best.pth")
    model = Stage1ResUNet(len(offsets), 1, S1_BASE).to(device)
    if os.path.exists(_done(name)):
        model.load_state_dict(_ld(best_p)["model"]); return model.eval()
    seed_everything(SEED)
    d_tr, d_va = get_split("train", TRAIN_SNAMES, offsets), get_split("va
    loader = DataLoader(SplitDataset(d_tr, True), batch_size=BATCH_SIZE,
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
    sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs, e
    ssim = DifferentiableSSIM().to(device)
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
    ema, best, t0 = EMA(model, 0.998), float("inf"), time.time()
    for ep in range(1, epochs + 1):
        model.train(); run = 0.0
        for b in loader:
            mri, ct, mask = b["mri"].to(device), b["ct"].to(device), b["m
            opt.zero_grad(set_to_none=True)
            with torch.amp.autocast("cuda", enabled=use_amp):
                pred = model(mri)
            pred = pred.float()
            ct_t = torch.where(mask > 0.5, ct, torch.full_like(ct, -1.0))
            pred_m = torch.where(mask > 0.5, pred, torch.full_like(pred,
            loss = _l1(pred, ct, mask) + 0.10 * ssim(pred_m, ct_t, mask=m
            scaler.scale(loss).backward(); scaler.unscale_(opt)
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(opt); scaler.update(); ema.update(model); run +=
        sch.step()
        if ep % BL_VAL_FREQ == 0 or ep == epochs:
            sc, m = direct_val_score(ema.shadow, d_va)
            star = ""
            if sc < best:
                best, star = sc, "  <-- best"
                torch.save({"model": ema.shadow.state_dict(), "epoch": ep
            print(f"[{name}] ep {ep:3d} train {run / len(loader):.4f} | v
                  f"SSIM {m['SSIM']:.4f} PSNR {m['PSNR']:.2f}{star} ({(ti
    _mark_done(name)
    model.load_state_dict(_ld(best_p)["model"])
    return model.eval()
```

In [25…

```
# %% ====== [B3] pix2pix BASELINE (ResUNet generator + PatchGAN, LSGAN +
class PatchD(nn.Module):
    """70x70-style PatchGAN. Input = [MRI stack, CT]."""

    def __init__(self, in_ch, base=64):
        super().__init__()

        def blk(i, o, stride, norm=True):
            L = [nn.Conv2d(i, o, 4, stride, 1)]
            if norm:
                L.append(nn.InstanceNorm2d(o))
            L.append(nn.LeakyReLU(0.2, inplace=True))
            return L

        self.net = nn.Sequential(*blk(in_ch, base, 2, False), *blk(base,
                                 *blk(base * 4, base * 8, 1), nn.Conv2d(b

    def forward(self, x):
        return self.net(x)


def train_pix2pix(name, offsets, epochs=P2P_EPOCHS):
    best_p, last_p = os.path.join(BL_DIR, f"{name}_best.pth"), os.path.jo
    G = Stage1ResUNet(len(offsets), 1, S1_BASE).to(device)
    if os.path.exists(_done(name)):
        G.load_state_dict(_ld(best_p)["G"]); return G.eval()
    seed_everything(SEED)
    D = PatchD(len(offsets) + 1).to(device)
    d_tr, d_va = get_split("train", TRAIN_SNAMES, offsets), get_split("va
    loader = DataLoader(SplitDataset(d_tr, True), batch_size=BATCH_SIZE,
    optG = torch.optim.Adam(G.parameters(), lr=P2P_LR, betas=(0.5, 0.999)
    optD = torch.optim.Adam(D.parameters(), lr=P2P_LR, betas=(0.5, 0.999)
    start, best = 0, float("inf")
    if os.path.exists(last_p):
        ck = _ld(last_p)
        G.load_state_dict(ck["G"]); D.load_state_dict(ck["D"])
        optG.load_state_dict(ck["optG"]); optD.load_state_dict(ck["optD"]
        print(f"[resume] {name} from epoch {start}")
    half, t0 = epochs // 2, time.time()
    for ep in range(start + 1, epochs + 1):
        lr = P2P_LR if ep <= half else P2P_LR * (epochs - ep + 1) / (epoc
        for o in (optG, optD):
            for g in o.param_groups:
                g["lr"] = lr
        G.train(); D.train(); rl = np.zeros(3)
        for b in loader:
            mri, ct, mask = b["mri"].to(device), b["ct"].to(device), b["m
            ct_t = torch.where(mask > 0.5, ct, torch.full_like(ct, -1.0))
            fake = G(mri)
            fake_m = torch.where(mask > 0.5, fake, torch.full_like(fake,
            optD.zero_grad(set_to_none=True)
            d_real, d_fake = D(torch.cat([mri, ct_t], 1)), D(torch.cat([m
            lossD = 0.5 * (F.mse_loss(d_real, torch.ones_like(d_real)) +
            lossD.backward(); optD.step()
```

```
            optG.zero_grad(set_to_none=True)
            d_g = D(torch.cat([mri, fake_m], 1))
            l_gan, l_l1 = F.mse_loss(d_g, torch.ones_like(d_g)), _l1(fake
            lossG = l_gan + P2P_L1 * l_l1
            lossG.backward(); optG.step()
            rl += [lossD.item(), l_gan.item(), l_l1.item()]
        rl /= len(loader)
        msg = f"[{name}] ep {ep:3d} D {rl[0]:.3f} G_gan {rl[1]:.3f} L1 {r
        if ep % BL_VAL_FREQ == 0 or ep == epochs:
            sc, m = direct_val_score(G, d_va)
            star = ""
            if sc < best:
                best, star = sc, "  <-- best"
                torch.save({"G": G.state_dict(), "epoch": ep, "metrics":
            msg += f" | val Score {sc:.4f} MAE {m['MAE']:.4f} SSIM {m['SS
        print(msg)
        torch.save({"G": G.state_dict(), "D": D.state_dict(), "optG": opt
                    "epoch": ep, "best": best}, last_p)
    _mark_done(name)
    G.load_state_dict(_ld(best_p)["G"])
    return G.eval()
```

In [26…

```
# %% ====== [B4] NAIVE conditional DDPM (MRI concat -> direct CT) =======
@torch.no_grad()
def sample_cddpm(model, pred, mri, mask, steps=50, generator=None):
    """DDIM (eta=0). The sampled variable is the CT itself, in [-1,1]."""
    model.eval()
    b, dev = mri.size(0), mri.device
    z = torch.randn((b, 1, IMAGE_SIZE, IMAGE_SIZE), device=dev, generator
    ts = sorted(set(torch.linspace(0, sched.T - 1, steps).long().tolist()
    ab = sched.alphas_cumprod
    x0 = z
    for i, tv in enumerate(ts):
        t = torch.full((b,), tv, device=dev, dtype=torch.long)
        with torch.amp.autocast("cuda", enabled=use_amp):
            out = model(torch.cat([z, mask, mri], 1), t, None)
        out = out.float()
        a = ab[tv]
        a_prev = ab[ts[i + 1]] if i + 1 < len(ts) else torch.ones((), dev
        if pred == "v":
            x0 = a.sqrt() * z - (1 - a).sqrt() * out
            eps = (1 - a).sqrt() * z + a.sqrt() * out
        else:
            eps = out
            x0 = (z - (1 - a).sqrt() * eps) / (a.sqrt() + 1e-8)
        x0 = x0.clamp(-1, 1)
        z = a_prev.sqrt() * x0 + (1 - a_prev).sqrt() * eps
    return x0


@torch.no_grad()
def predict_cddpm(model, pred, d, n_samples, flips, steps, bs=16, seed=SE
    """Returns (single sample, mean over n_samples x flips). Background s
    g = torch.Generator(device=device).manual_seed(seed)
```

```
    single, mean = [], []
    for mri, ct, mask, sl in iter_batches(d, bs):
        acc, cnt, first = 0, 0, None
        for fl in flips:
            m_in, mk = (mri.flip(-1), mask.flip(-1)) if fl else (mri, mas
            for _ in range(n_samples):
                x = sample_cddpm(model, pred, m_in, mk, steps, g)
                x = x.flip(-1) if fl else x
                first = x.clone() if first is None else first
                acc = acc + x; cnt += 1
        bg = torch.full_like(first, -1.0)
        single.append(torch.where(mask > 0.5, first, bg)[:, 0].cpu().nump
        mean.append(torch.where(mask > 0.5, acc / cnt, bg)[:, 0].cpu().nu
    return np.concatenate(single), np.concatenate(mean)


def train_cddpm(name, pred, offsets, epochs=CDDPM_EPOCHS):
    best_p, last_p = os.path.join(BL_DIR, f"{name}_best.pth"), os.path.jo
    in_ch = 2 + len(offsets)
    model = DiffUNet(in_ch, 48, T_DIM, (32, 64, 128, 256), film=False).to
    if os.path.exists(_done(name)):
        model.load_state_dict(_ld(best_p)["ema"]); return model.eval()
    seed_everything(SEED)
    d_tr, d_va = get_split("train", TRAIN_SNAMES, offsets), get_split("va
    loader = DataLoader(SplitDataset(d_tr, True), batch_size=BATCH_SIZE,
                        drop_last=True, pin_memory=use_amp)
    ema = EMA(model, 0.999)
    opt = torch.optim.AdamW(model.parameters(), lr=CDDPM_LR, weight_decay
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
    total, start, best, step = epochs * len(loader), 0, float("inf"), 0
    if os.path.exists(last_p):
        ck = _ld(last_p)
        model.load_state_dict(ck["model"]); ema.shadow.load_state_dict(ck
        opt.load_state_dict(ck["opt"]); scaler.load_state_dict(ck["scaler
        step = start * len(loader); print(f"[resume] {name} from epoch {s

    def lr_at(s):
        return (1e-6 + (CDDPM_LR - 1e-6) * 0.5 * (1 + math.cos(math.pi *

    t0 = time.time()
    for ep in range(start + 1, epochs + 1):
        model.train(); run = 0.0
        for b in tqdm(loader, desc=f"{name} {ep}/{epochs}", leave=False):
            mri, ct, mask = b["mri"].to(device), b["ct"].to(device), b["m
            x0 = torch.where(mask > 0.5, ct, torch.full_like(ct, -1.0))
            t = torch.randint(0, T_STEPS, (x0.size(0),), device=device)
            noise = torch.randn_like(x0)
            xt = sched.q_sample(x0, t, noise)
            tgt = sched.v_target(x0, noise, t) if pred == "v" else noise
            for g_ in opt.param_groups:
                g_["lr"] = lr_at(step)
            opt.zero_grad(set_to_none=True)
            with torch.amp.autocast("cuda", enabled=use_amp):
                out = model(torch.cat([xt, mask, mri], 1), t, None)
```

```
            out = out.float()
            loss = ((((out - tgt) ** 2) * mask).sum((1, 2, 3)) / (mask.su
            scaler.scale(loss).backward(); scaler.unscale_(opt)
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(opt); scaler.update(); ema.update(model); step +=
        msg = f"[{name}] ep {ep:3d} loss {run / len(loader):.4f}"
        if ep % 10 == 0 or ep == epochs:
            single, _ = predict_cddpm(ema.shadow, pred, d_va, 1, (False,)
            m, _ = patient_level(single, BV_GT, BV_MASK, BV_PID)
            star = ""
            if m["Score"] < best:
                best, star = m["Score"], "  <-- best"
                torch.save({"ema": ema.shadow.state_dict(), "epoch": ep,
            msg += (f" | val(1 sample) Score {m['Score']:.4f} MAE {m['MAE
                    f"PSNR {m['PSNR']:.2f}{star} ({(time.time() - t0) / 6
        print(msg)
        torch.save({"model": model.state_dict(), "ema": ema.shadow.state_
                    "scaler": scaler.state_dict(), "epoch": ep, "best": b
    _mark_done(name)
    model.load_state_dict(_ld(best_p)["ema"])
    return model.eval()
```

In [27…

```
# %% ====== [B5] TRAIN / LOAD EVERYTHING (resumable; run in stages if nee
BL_MODELS = {}                                      # name -> (model, offse
if "resunet_2d" in BL_TO_RUN:
    BL_MODELS["resunet_2d"] = (train_resunet("resunet_2d", (0,)), (0,))
if "resunet_25d" in BL_TO_RUN:
    BL_MODELS["resunet_25d"] = (train_resunet("resunet_25d", BL_OFFSETS),
if "pix2pix" in BL_TO_RUN:
    BL_MODELS["pix2pix"] = (train_pix2pix("pix2pix_25d", BL_OFFSETS), BL_
for _tag, _pred in (("cddpm_v", "v"), ("cddpm_eps", "eps")):
    if _tag in BL_TO_RUN:
        BL_MODELS[_tag] = (train_cddpm(_tag, _pred, BL_OFFSETS), BL_OFFSE

DET_MODELS = []                                     # deterministic refiner
if "det" in BL_TO_RUN:
    for _sd in DET_SEEDS:
        _cfg = make_cfg({"mode": "det", "film": DET_FILM, "seed": _sd}, B
        _name = f"det_{'FiLM' if DET_FILM else 'noFiLM'}" + ("" if _sd ==
        DET_MODELS.append((_cfg, get_stage2(_name, _cfg, DET_EPOCHS)))
print("Ready:", list(BL_MODELS), f"| det models: {len(DET_MODELS)}")
```

```
Ready: ['resunet_2d', 'resunet_25d', 'pix2pix', 'cddpm_v'] | det models:
1
```

In [28…

```
import os
import torch

baseline_dir = "/kaggle/working/synthrad_ccr_v3/baselines"
latest_path = os.path.join(baseline_dir, "cddpm_v_latest.pth")

if os.path.exists(latest_path):
    ck = torch.load(latest_path, map_location="cpu", weights_only=False)
    print("cDDPM v-prediction checkpoint found.")
```

```
    print("Last saved epoch:", ck.get("epoch", "unknown"))
    print("Target epochs:", CDDPM_EPOCHS)
    print("Completion marker exists:", os.path.exists(
        os.path.join(baseline_dir, "cddpm_v.done")
    ))
else:
    print("Latest cDDPM checkpoint is missing. Do not start training yet.
```

```
cDDPM v-prediction checkpoint found.
Last saved epoch: 120
Target epochs: 120
Completion marker exists: True
```

In [29…

```
import os

required_variables = [
    "get_split", "Stage1ResUNet", "DiffUNet", "predict_stack",
    "BL_MODELS", "DET_MODELS", "BL_DIR", "EVAL_VAL",
    "VAL_SNAMES", "TEST_SNAMES"
]

missing = [name for name in required_variables if name not in globals()]

print("Missing Python variables:", missing)
print("Baseline directory exists:", os.path.isdir(
    "/kaggle/working/synthrad_ccr_v3/baselines"
))

baseline_dir = "/kaggle/working/synthrad_ccr_v3/baselines"

for name in [
    "resunet_2d.done",
    "resunet_25d.done",
    "pix2pix_25d.done",
    "cddpm_v.done",
    "cddpm_v_latest.pth",
    "cddpm_v_best.pth",
]:
    path = os.path.join(baseline_dir, name)
    print(("FOUND " if os.path.exists(path) else "MISSING "), name)
```

```
Missing Python variables: []
Baseline directory exists: True
FOUND   resunet_2d.done
FOUND   resunet_25d.done
FOUND   pix2pix_25d.done
FOUND   cddpm_v.done
FOUND   cddpm_v_latest.pth
FOUND   cddpm_v_best.pth
```

In [30…

```
# %% ====== [B6] UNIFIED COMPARISON: collect predictions, metrics, cost =
BL_LABEL = {"resunet_2d": "ResUNet 2D (single model)", "resunet_25d": "Re
            "pix2pix": "pix2pix (ResUNet G + PatchGAN, 2.5D)", "cddpm_v":
            "cddpm_eps": "Naive cDDPM (direct CT, eps-pred)"}
S1_BUNDLE = get_bundle(BL_S1)
```

```
S1_PARAMS = nparams(S1_BUNDLE.models[1])


def _final_diff_params():
    m = globals().get("final_model")
    if m is None:
        m = get_stage2(FINAL_TAG_, FINAL_CFG_, FINAL_EPOCHS)
    return nparams(m)


def _ours(split, mask):
    """Final diffusion model: (1 sample, 1 fold) and (4 folds x 4 samples
    cache = os.path.join(BL_DIR, f"ours_{split}.npz")
    if os.path.exists(cache):
        z = np.load(cache); return z["single"], z["tta"]
    if split == "test" and "preds" in globals():
        k1 = [k for k in preds if k.startswith("Diffusion") and "1 sample
        k2 = [k for k in preds if k.startswith("+ flip-TTA")]
        if k1 and k2:
            return preds[k1[0]], preds[k2[0]]
    if split == "val" and "val_stack" in globals():
        c1, r1 = aggregate(val_stack, fidx=[0], lidx=[0], sidx=[0]); cT,
        return finalize(c1, r1, mask), finalize(cT, rT, mask)
    m = globals().get("final_model") or get_stage2(FINAL_TAG_, FINAL_CFG_
    b = get_bundle(FINAL_CFG_["s1"])
    d = get_split(split, VAL_SNAMES if split == "val" else TEST_SNAMES, b
    st = predict_stack(d, m, FINAL_CFG_, b, 4, 50, folds=(1, 2, 3, 4), fl
    c1, r1 = aggregate(st, fidx=[0], lidx=[0], sidx=[0]); cT, rT = aggreg
    single, tta = finalize(c1, r1, mask), finalize(cT, rT, mask)
    np.savez(cache, single=single, tta=tta)
    return single, tta


def collect(split):
    names = VAL_SNAMES if split == "val" else TEST_SNAMES
    d0 = get_split(split, names, (0,))
    gt, mask, pid = d0["ct"][:, 0].numpy(), d0["mask"][:, 0].numpy(), d0[
    rows = {}                                                  # label -> (pr

    for tag, (model, offs) in BL_MODELS.items():
        d = get_split(split, names, offs)
        if tag.startswith("cddpm"):
            pred_t = "v" if tag == "cddpm_v" else "eps"
            single, mean = predict_cddpm(model, pred_t, d, CDDPM_SAMPLES,
            rows[f"{BL_LABEL[tag]}: 1 sample"] = (single, CDDPM_STEPS, np
            rows[f"{BL_LABEL[tag]}: {2 * CDDPM_SAMPLES}-sample mean + TTA
        else:
            rows[BL_LABEL[tag]] = (predict_direct(model, d), 1, nparams(m
            rows[BL_LABEL[tag] + " + flip-TTA"] = (predict_direct(model,

    rows["Stage-1 2.5D, 4-fold ensemble (ours, stage 1)"] = (
        finalize(stage1_ensemble(S1_BUNDLE, get_split(split, names, S1_BU

    if DET_MODELS:
```

```
        c0s, r0s, c1s, r1s = [], [], [], []
        for cfg, model in DET_MODELS:
            b = get_bundle(cfg["s1"]); d = get_split(split, names, b.offs
            st = predict_stack(d, model, cfg, b, 1, 1, folds=(1, 2, 3, 4)
            c0, r0 = aggregate(st, lidx=[0]); c1, r1 = aggregate(st)
            c0s.append(c0); r0s.append(r0); c1s.append(c1); r1s.append(r1
        k = len(DET_MODELS); pm = 4 * S1_PARAMS + sum(nparams(m) for _, m
        tag_ = f"Deterministic refiner ({DET_EPOCHS} ep, {'FiLM' if DET_F
        rows[tag_ + ", 4 folds"] = (finalize(np.mean(c0s, 0), np.mean(r0s
        rows[tag_ + ", 4 folds + flip-TTA"] = (finalize(np.mean(c1s, 0),

    single, tta = _ours(split, mask)
    dp = _final_diff_params()
    rows["Ours: diffusion, 1 sample, 1 fold"] = (single, 50 + 1, S1_PARAM
    rows["Ours: diffusion, 4 folds x 4 samples + flip-TTA (w=1)"] = (tta,
    return rows, gt, mask, pid


def table(rows, gt, mask, pid):
    out = []
    for label, (p, nfe, par) in rows.items():
        m, _ = patient_level(p, gt, mask, pid)
        out.append({"method": label, "MAE": m["MAE"], "RMSE": m["RMSE"],
                    "Global_SSIM": m["Global_SSIM"], "Global_PSNR": m["Gl
                    "NFE/slice": nfe, "Params(M)": par})
    return pd.DataFrame(out)


if EVAL_VAL:
    rows_v, gt_v, mk_v, pid_v = collect("val")
    tab_v = table(rows_v, gt_v, mk_v, pid_v)
    print("\nVALIDATION (6 patients)"); print(tab_v.round(4).to_string(in
    tab_v.to_csv(os.path.join(DIR_METRICS, "baselines_val.csv"), index=Fa
```

```
VALIDATION (6 patients)
                                                    method    MAE   RMSE
SSIM    PSNR  Global_SSIM  Global_PSNR  Score  NFE/slice  Params(M)
                                ResUNet 2D (single model) 0.0365 0.0649
0.8596 24.0263       0.8225      22.7564 0.0716          1     4.6243
                     ResUNet 2D (single model) + flip-TTA 0.0359 0.0635
0.8632 24.2151       0.8241      22.8221 0.0701          2     4.6243
                               ResUNet 2.5D (single model) 0.0367 0.0645
0.8620 24.0875       0.8235      22.7761 0.0712          1     4.6256
                     ResUNet 2.5D (single model) + flip-TTA 0.0357 0.0624
0.8669 24.3870       0.8258      22.8716 0.0690          2     4.6256
                      pix2pix (ResUNet G + PatchGAN, 2.5D) 0.0352 0.0638
0.8662 24.2094       0.8256      22.8297 0.0687          1     4.6256
           pix2pix (ResUNet G + PatchGAN, 2.5D) + flip-TTA 0.0339 0.0607
0.8743 24.6417       0.8293      22.9696 0.0653          2     4.6256
                 Naive cDDPM (direct CT, v-pred): 1 sample 0.0520 0.0730
0.8468 23.0080       0.8183      22.3359 0.0903         50    13.3468
     Naive cDDPM (direct CT, v-pred): 32-sample mean + TTA 0.0497 0.0672
0.8779 23.7401       0.8322      22.5993 0.0802       1600    13.3468
             Stage-1 2.5D, 4-fold ensemble (ours, stage 1) 0.0363 0.0627
0.8621 24.3217       0.8239      22.8538 0.0708          4    18.5023
          Deterministic refiner (120 ep, noFiLM), 4 folds 0.0336 0.0592
0.8769 24.8380       0.8303      23.0208 0.0644          4    31.8496
Deterministic refiner (120 ep, noFiLM), 4 folds + flip-TTA 0.0329 0.0574
0.8811 25.1173       0.8324      23.0997 0.0626          8    31.8496
                         Ours: diffusion, 1 sample, 1 fold 0.0378 0.0666
0.8606 23.8127       0.8228      22.6870 0.0726         51    17.9728
     Ours: diffusion, 4 folds x 4 samples + flip-TTA (w=1) 0.0332 0.0568
0.8850 25.2262       0.8340      23.1269 0.0620       1608    31.8496
```

In [31…

```python
# %% ====== [B7] FINAL TEST COMPARISON (run ONCE) ======================
if EVAL_TEST:
    rows_t, gt_t, mk_t, pid_t = collect("test")
    tab_t = table(rows_t, gt_t, mk_t, pid_t)
    print("\nTEST (6 held-out patients)"); print(tab_t.round(4).to_string
    tab_t.to_csv(os.path.join(DIR_METRICS, "baselines_test.csv"), index=F

    ours_key = "Ours: diffusion, 4 folds x 4 samples + flip-TTA (w=1)"
    stats = {}
    for label in rows_t:
        if label == ours_key or label.startswith("Ours"):
            continue
        s = paired_stats(rows_t[label][0], rows_t[ours_key][0], gt_t, mk_
        stats[label] = s
        print(f"\n=== Ours minus [{label}] (per-patient paired, n=6) ==="
        print(s[s["metric"].isin(["mae", "ssim", "psnr"])].round(4).to_st
    pd.concat(stats).to_csv(os.path.join(DIR_METRICS, "baselines_test_pai

    per = {label: patient_level(p, gt_t, mk_t, pid_t)[1].set_index("patie
           for label, (p, _, _) in rows_t.items()}
    pd.concat(per, axis=1).to_csv(os.path.join(DIR_METRICS, "baselines_te

    # qualitative: middle slice of the first test patient
    def _pick(sub, must=None):
```

```
        c = [k for k in rows_t if sub in k and (must is None or must in k
        return c[-1] if c else None

    keys = [k for k in (_pick("ResUNet 2.5D", "TTA"), _pick("pix2pix", "T
                        _pick("Naive cDDPM (direct CT, v-pred)", "mean"),
                        _pick("Ours: diffusion, 4 folds")) if k]
    pids_u = sorted(set(pid_t)); idx = [i for i, p in enumerate(pid_t) if
    d0 = get_split("test", TEST_SNAMES, (0,)); mk = mk_t[i] > 0.5
    fig, ax = plt.subplots(2, len(keys) + 2, figsize=(3.2 * (len(keys) +
    ax[0, 0].imshow(d0["mri"][i, 0].numpy(), cmap="gray", vmin=-1, vmax=1
    ax[0, 1].imshow(gt_t[i], cmap="gray", vmin=-1, vmax=1); ax[0, 1].set_
    for j, k in enumerate(keys):
        ax[0, j + 2].imshow(rows_t[k][0][i], cmap="gray", vmin=-1, vmax=1
        ax[1, j + 2].imshow(np.abs(rows_t[k][0][i] - gt_t[i]) * mk, cmap=
        ax[1, j + 2].set_title(f"|err| MAE {np.abs(rows_t[k][0][i] - gt_t
    for a in ax.ravel():
        a.axis("off")
    plt.tight_layout(); plt.savefig(os.path.join(DIR_FIG, "baselines_qual
```

```
TEST (6 held-out patients)
                                                  method    MAE   RMSE
SSIM    PSNR  Global_SSIM  Global_PSNR  Score  NFE/slice  Params(M)
                                ResUNet 2D (single model) 0.0299 0.0587
0.8823 24.8844       0.8500      23.6131 0.0593          1     4.6243
                     ResUNet 2D (single model) + flip-TTA 0.0291 0.0570
0.8860 25.1401       0.8518      23.7016 0.0575          2     4.6243
                               ResUNet 2.5D (single model) 0.0296 0.0572
0.8863 25.1092       0.8518      23.6905 0.0581          1     4.6256
                    ResUNet 2.5D (single model) + flip-TTA 0.0289 0.0558
0.8896 25.3107       0.8533      23.7548 0.0565          2     4.6256
                      pix2pix (ResUNet G + PatchGAN, 2.5D) 0.0271 0.0556
0.8927 25.3713       0.8545      23.7827 0.0539          1     4.6256
           pix2pix (ResUNet G + PatchGAN, 2.5D) + flip-TTA 0.0260 0.0529
0.8988 25.8035       0.8572      23.9212 0.0513          2     4.6256
                 Naive cDDPM (direct CT, v-pred): 1 sample 0.0477 0.0665
0.8670 23.7853       0.8452      23.2423 0.0810         50    13.3468
     Naive cDDPM (direct CT, v-pred): 32-sample mean + TTA 0.0456 0.0610
0.8993 24.5342       0.8594      23.5350 0.0708       1600    13.3468
             Stage-1 2.5D, 4-fold ensemble (ours, stage 1) 0.0291 0.0560
0.8853 25.2789       0.8514      23.7664 0.0577          4    18.5023
          Deterministic refiner (120 ep, noFiLM), 4 folds 0.0270 0.0522
0.8979 25.9168       0.8570      23.9626 0.0525          4    31.8496
Deterministic refiner (120 ep, noFiLM), 4 folds + flip-TTA 0.0261 0.0501
0.9026 26.2666       0.8592      24.0698 0.0504          8    31.8496
                         Ours: diffusion, 1 sample, 1 fold 0.0306 0.0590
0.8828 24.8605       0.8502      23.6300 0.0599         51    17.9728
     Ours: diffusion, 4 folds x 4 samples + flip-TTA (w=1) 0.0259 0.0496
0.9058 26.3580       0.8604      24.1027 0.0495       1608    31.8496

=== Ours minus [ResUNet 2D (single model)] (per-patient paired, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0039  -0.0049  -0.0028      0.0312                  6
6
  ssim          0.0234   0.0180   0.0276      0.0312                  6
6
  psnr          1.4736   1.2441   1.7121      0.0312                  6
6

=== Ours minus [ResUNet 2D (single model) + flip-TTA] (per-patient paire
d, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0031  -0.0039  -0.0022      0.0312                  6
6
  ssim          0.0197   0.0144   0.0237      0.0312                  6
6
  psnr          1.2179   0.9842   1.4508      0.0312                  6
6

=== Ours minus [ResUNet 2.5D (single model)] (per-patient paired, n=6)
===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
```

```
   mae         -0.0037  -0.0043  -0.0032      0.0312                  6
6
  ssim          0.0195   0.0172   0.0217      0.0312                  6
6
  psnr          1.2488   1.0887   1.4184      0.0312                  6
6

=== Ours minus [ResUNet 2.5D (single model) + flip-TTA] (per-patient pair
ed, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0030  -0.0036  -0.0024      0.0312                  6
6
  ssim          0.0162   0.0142   0.0183      0.0312                  6
6
  psnr          1.0473   0.8766   1.2391      0.0312                  6
6

=== Ours minus [pix2pix (ResUNet G + PatchGAN, 2.5D)] (per-patient paire
d, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0012  -0.0030   0.0007      0.5625                  3
6
  ssim          0.0130   0.0076   0.0197      0.0312                  6
6
  psnr          0.9867   0.6365   1.3551      0.0312                  6
6

=== Ours minus [pix2pix (ResUNet G + PatchGAN, 2.5D) + flip-TTA] (per-pat
ient paired, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0001  -0.0018   0.0016      1.0000                  3
6
  ssim          0.0070   0.0026   0.0124      0.0625                  5
6
  psnr          0.5545   0.2498   0.8749      0.0625                  5
6

=== Ours minus [Naive cDDPM (direct CT, v-pred): 1 sample] (per-patient p
aired, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0218  -0.0263  -0.0169      0.0312                  6
6
  ssim          0.0388   0.0319   0.0452      0.0312                  6
6
  psnr          2.5727   1.9402   3.1203      0.0312                  6
6

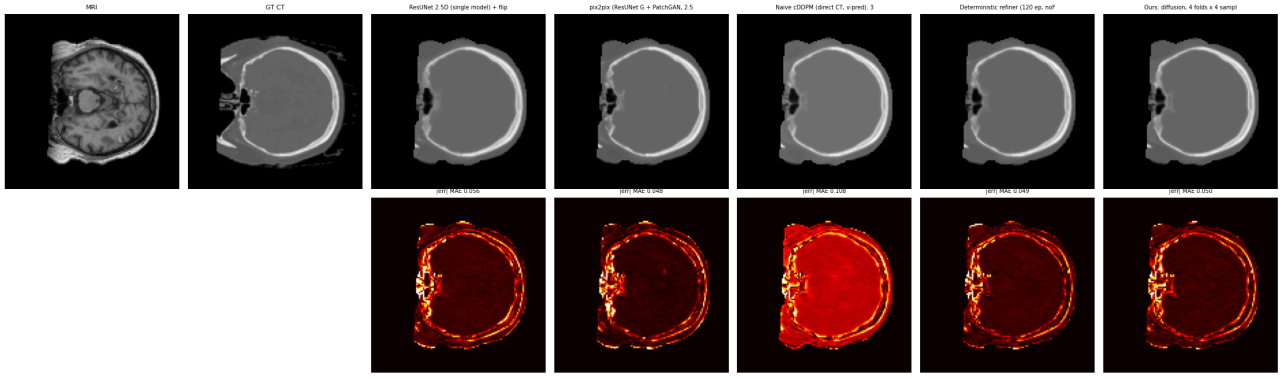
=== Ours minus [Naive cDDPM (direct CT, v-pred): 32-sample mean + TTA] (p
er-patient paired, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
```

```
   mae         -0.0196  -0.0243  -0.0148      0.0312                   6
6
  ssim          0.0065   0.0023   0.0097      0.0625                   5
6
  psnr          1.8238   1.1670   2.3752      0.0312                   6
6

=== Ours minus [Stage-1 2.5D, 4-fold ensemble (ours, stage 1)] (per-patie
nt paired, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0031  -0.0038  -0.0024      0.0312                   6
6
  ssim          0.0204   0.0175   0.0226      0.0312                   6
6
  psnr          1.0791   0.9428   1.2818      0.0312                   6
6

=== Ours minus [Deterministic refiner (120 ep, noFiLM), 4 folds] (per-pat
ient paired, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0010  -0.0021  -0.0001      0.2188                   4
6
  ssim          0.0078   0.0064   0.0094      0.0312                   6
6
  psnr          0.4411   0.2742   0.6145      0.0312                   6
6

=== Ours minus [Deterministic refiner (120 ep, noFiLM), 4 folds + flip-TT
A] (per-patient paired, n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0001  -0.0009   0.0006      1.0000                   3
6
  ssim          0.0031   0.0020   0.0043      0.0312                   6
6
  psnr          0.0914  -0.0017   0.2106      0.2188                   4
6
```



In [28…

```
# ============================================================
# ZIP THE ENTIRE KAGGLE /kaggle/working DIRECTORY
```

```python
# Creates a ZIP with progress monitoring and a download link.
# ================================================================

import os
import zipfile
from pathlib import Path
from tqdm.auto import tqdm
from IPython.display import display, FileLink

WORKING_DIR = Path("/kaggle/working")
ZIP_PATH = WORKING_DIR / "Kaggle_Working_Full_Backup.zip"

# Set True to avoid including older ZIP archives and making the
# new backup unnecessarily large. Set False to include them.
EXCLUDE_OLD_ZIPS = True

# ------------------------------------------------
# 1. Collect files
# ------------------------------------------------
files_to_zip = []

for root, dirs, filenames in os.walk(WORKING_DIR):
    # Do not include the archive being created.
    for filename in filenames:
        path = Path(root) / filename

        if path.resolve() == ZIP_PATH.resolve():
            continue

        if EXCLUDE_OLD_ZIPS and path.suffix.lower() == ".zip":
            continue

        files_to_zip.append(path)

if not files_to_zip:
    raise RuntimeError(f"No files found inside {WORKING_DIR}")

total_bytes = sum(p.stat().st_size for p in files_to_zip)

print(f"Working directory: {WORKING_DIR}")
print(f"Files to archive: {len(files_to_zip):,}")
print(f"Total source size: {total_bytes / (1024**3):.2f} GiB")
print(f"ZIP output: {ZIP_PATH}")

# ------------------------------------------------
# 2. Create ZIP with progress bar
# ZIP_STORED is used for faster archiving of large
# model checkpoints and already-compressed images.
# ------------------------------------------------
with zipfile.ZipFile(
    ZIP_PATH,
    mode="w",
    compression=zipfile.ZIP_STORED,
    allowZip64=True
```

```
) as archive:

    for path in tqdm(files_to_zip, desc="Creating backup ZIP", unit="file
        archive.write(
            path,
            arcname=str(path.relative_to(WORKING_DIR))
        )

# ------------------------------------------------
# 3. Verify archive integrity
# ------------------------------------------------
print("\nVerifying ZIP archive...")

with zipfile.ZipFile(ZIP_PATH, "r") as archive:
    bad_file = archive.testzip()
    archived_count = len(archive.namelist())

if bad_file is not None:
    raise RuntimeError(f"ZIP verification failed at: {bad_file}")

print("ZIP verification successful.")
print(f"Archived entries: {archived_count:,}")
print(f"ZIP size: {ZIP_PATH.stat().st_size / (1024**3):.2f} GiB")

# ------------------------------------------------
# 4. Display download link
# ------------------------------------------------
print("\nClick the link below to download your backup:")
display(FileLink(str(ZIP_PATH)))
```

```
Working directory: /kaggle/working
Files to archive: 111
Total source size: 3.95 GiB
ZIP output: /kaggle/working/Kaggle_Working_Full_Backup.zip
Creating backup ZIP:   0%|          | 0/111 [00:00<?, ?file/s]
Verifying ZIP archive...
ZIP verification successful.
Archived entries: 111
ZIP size: 3.95 GiB

Click the link below to download your backup:
```

/kaggle/working/Kaggle_Working_Full_Backup.zip

In [29…

```
from pathlib import Path
from IPython.display import display, FileLink, Javascript
import os

# ================================================
# FIND THE ZIP FILE
# ================================================

WORKING_DIR = Path("/kaggle/working")

# Preferred ZIP filename
```

```
ZIP_PATH = WORKING_DIR / "Kaggle_Working_Full_Backup.zip"

# If the preferred ZIP doesn't exist, select the most recent ZIP.
if not ZIP_PATH.exists():
    zip_files = sorted(
        WORKING_DIR.glob("*.zip"),
        key=lambda p: p.stat().st_mtime,
        reverse=True
    )

    if not zip_files:
        raise FileNotFoundError(
            "No ZIP file found in /kaggle/working. Create the ZIP first."
        )

    ZIP_PATH = zip_files[0]

print("ZIP file:", ZIP_PATH)
print(f"Size: {ZIP_PATH.stat().st_size / (1024**3):.2f} GiB")

# ============================================================
# DISPLAY DOWNLOAD LINK
# ============================================================

display(FileLink(str(ZIP_PATH)))

# ============================================================
# ATTEMPT AUTOMATIC BROWSER DOWNLOAD
# ============================================================

filename = ZIP_PATH.name

display(Javascript("""
(() => {
    const filename = %s;
    const links = Array.from(document.querySelectorAll('a'));

    const link = links.find(a =>
        a.href.includes(filename) ||
        a.textContent.includes(filename)
    );

    if (link) {
        link.click();
        console.log("Download initiated:", filename);
    } else {
        console.log(
            "Automatic download was blocked. Click the ZIP link shown abo
        );
    }
})();
""" % repr(filename)))
```

```
ZIP file: /kaggle/working/Kaggle_Working_Full_Backup.zip
Size: 3.95 GiB
```

/kaggle/working/Kaggle_Working_Full_Backup.zip

In [ ]:

APPENDED ADD-ON 1: CCR Paper Suite (P0–P9)

The original `datasets-comp.ipynb` cells and outputs above are retained. The cells below are appended from `ccr_paper_suite.ipynb` in their source order.

In [34…

```
# ======================================================================
# CCR PAPER SUITE -- every remaining test needed for the paper, in one fi
# (add-on: run in the SAME notebook session, after v3 cells [0]-[5] and b
#
#  CHEAP (no training; minutes)
#   [P1] reproducibility record (versions, seeds, splits, configs)
#   [P2] PNG -> HU calibration from the original NIfTI volumes (+ verifie
#   [P3] error analysis: slice-position thirds, bone vs soft-tissue error
#   [P4] cost table: seconds / slice, peak GPU memory, NFE
#   [P5] uncertainty vs error (does diffusion spread beat deterministic-e
#   [P6] full statistics for the main table: paired differences, bootstra
#  HEAVY (training; switch on in RUN, hours each)
#   [P7] capacity-matched baselines: 4-model pix2pix ensemble + ResUNet t
#   [P8] extra seeds of the final diffusion model and the deterministic r
#   [P9] extra disjoint patient-level splits (generalisation beyond the 6
#
# RULES
#   * everything is selected on VALIDATION patients only; test patients a
#   * disclose in the paper that the original 6 test patients were reused
#   * all HU numbers stay "provisional" until [P2] reports a verified map
# ======================================================================
```

In [35…

```
# %% ====== [P0] CHECKS + CONFIG ====================================
import os, glob, re, sys, time, json, math, copy, platform, warnings
import numpy as np
import pandas as pd
import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from PIL import Image
from tqdm.auto import tqdm
from torch.utils.data import DataLoader, Subset
from scipy.stats import spearmanr, wilcoxon

_need = ["get_split", "SplitDataset", "iter_batches", "Stage1ResUNet", "D
```

```
          "build_input", "quick_validate", "patient_level", "predict_stack
          "stage1_ensemble", "make_cfg", "get_stage2", "ABLATIONS", "_l1",
          "PATIENT_SORTED", "PID_TO_FOLD", "SPLITS", "BUNDLES", "FOLDS", "
          "TRAIN_SNAMES", "VAL_SNAMES", "TEST_SNAMES", "DIR_S1", "DIR_S2",
          "N_FOLDS", "S1_BASE", "CT_DIR", "FINAL_EPOCHS", "ABL_EPOCHS", "T
          "use_amp", "SEED", "S2_LR", "S2_WD", "S2_WARMUP", "EMA_DECAY", "
          "PatchD", "_ld", "_done", "nparams", "predict_direct", "direct_v
          "BL_S1", "BL_OFFSETS", "P2P_EPOCHS", "P2P_LR", "P2P_L1"]
_missing = [n for n in _need if n not in globals()]
assert not _missing, f"Run v3 cells [0]-[5] and baseline cells [B0]-[B7]

RUN = dict(repro=True, hu=True, error_analysis=True, cost=True, uncertain
           matched_baselines=False, extra_seeds=False, extra_splits=False

EXTRA_SEEDS, SEED_EVAL_VAL = (43, 44), False
ENS_P2P_EPOCHS, MATCHED_RESUNET_EPOCHS = P2P_EPOCHS, P2P_EPOCHS
CV_SPLITS, CV_EPOCHS, CV_RESUNET_EPOCHS, CV_P2P_EPOCHS = 2, 60, 40, 60
CV_SAMPLES, CV_STEPS, CV_INCLUDE_PIX2PIX = 2, 25, True
NIFTI_ROOT, HU_N_PATIENTS = None, 3
BONE_HU = 300.0

FINAL_NAME_ = globals().get("FINAL_NAME_") or globals().get("FINAL_NAME")
FINAL_CFG_ = globals().get("FINAL_CFG_") or globals().get("FINAL_CFG") or
FINAL_TAG_ = globals().get("FINAL_TAG_") or globals().get("FINAL_TAG") or
FINAL_PROTO = globals().get("FINAL_PROTOCOL") or dict(folds=(1, 2, 3, 4),
SUITE_DIR = os.path.join(BASE_OUT_DIR, "paper_suite")
os.makedirs(SUITE_DIR, exist_ok=True)


def _pick(rows, sub, must=None, mustnot=None):
    c = [k for k in rows if sub in k and (must is None or must in k) and
    return c[-1] if c else None


def method_keys(rows):
    """Short names -> labels in rows_t / rows_v (strongest version of eac
    k = {"Stage-1 (4-fold ens.)": _pick(rows, "Stage-1 2.5D, 4-fold ensem
         "ResUNet 2.5D + TTA": _pick(rows, "ResUNet 2.5D", "TTA"),
         "pix2pix + TTA": _pick(rows, "pix2pix", "flip-TTA", "ensemble"),
         "Naive cDDPM (32 samples + TTA)": _pick(rows, "Naive cDDPM", "me
         "Det. refiner + TTA": _pick(rows, "Deterministic refiner", "flip
         "Ours (diffusion) + TTA": _pick(rows, "Ours: diffusion, 4 folds"
    return {a: b for a, b in k.items() if b}


def _arrays(split):
    names = VAL_SNAMES if split == "val" else TEST_SNAMES
    d0 = get_split(split, names, (0,))
    return d0["ct"][:, 0].numpy(), d0["mask"][:, 0].numpy(), d0["pid"], d
```

In [36…

```
# %% ====== [P1] REPRODUCIBILITY RECORD ================================
def save_repro():
    import numpy, pandas, scipy, skimage
    info = {"time_utc": time.strftime("%Y-%m-%d %H:%M:%S", time.gmtime())
```

```
            "platform": platform.platform(), "torch": torch.__version__,
            "gpu": torch.cuda.get_device_name(0) if torch.cuda.is_availab
            "numpy": numpy.__version__, "pandas": pandas.__version__, "sc
            "skimage": skimage.__version__, "opencv": cv2.__version__, "s
            "split": {"train": TRAIN_PIDS, "val": VAL_PIDS, "test": TEST_
            "n_slices": {"train": len(TRAIN_SNAMES), "val": len(VAL_SNAME
            "image_size": globals().get("IMAGE_SIZE"), "final_config": FI
            "final_protocol": FINAL_PROTO, "stage1": {"base": S1_BASE, "e
                                                      "lr": globals().get
                                                      "variant": BL_S1, "
            "stage2": {"lr": S2_LR, "wd": S2_WD, "warmup": S2_WARMUP, "em
                       "ssim_cutoff": globals().get("SSIM_T_CUTOFF")},
            "baselines": {"resunet_epochs": globals().get("RESUNET_EPOCHS
                          "pix2pix_l1": P2P_L1, "cddpm_epochs": globals()
            "hu_window_assumed": [HU_MIN, HU_MAX], "hu_verified": bool(gl
    path = os.path.join(SUITE_DIR, "reproducibility.json")
    json.dump(info, open(path, "w"), indent=2, default=str)
    print("saved", path)
    return info


if RUN["repro"]:
    REPRO = save_repro()
```

```
saved /kaggle/working/synthrad_ccr_v3/paper_suite/reproducibility.json
```

In [37…

```
# %% ====== [P2] PNG -> HU CALIBRATION (needs nibabel + the original NIfT
def _dihedral(a):
    out = []
    for k in range(4):
        r = np.rot90(a, k)
        out += [r, np.fliplr(r)]
    return out


def _small(a, n):
    return cv2.resize(np.ascontiguousarray(a, dtype=np.float32), (n, n),


def _corr(a, b):
    a, b = a - a.mean(), b - b.mean()
    d = np.sqrt((a * a).sum() * (b * b).sum())
    return float((a * b).sum() / d) if d > 0 else -1.0


def calibrate_png_to_hu(vol, pngs, match_size=64):
    """vol: 3-D HU volume; pngs: list of 2-D uint8 CT slices from the sam
    Finds axial axis / orientation / slice index by correlation, then fit
    Returns window [lo, hi] (HU values mapped to 0 and 255), R^2 and the
    vol = np.asarray(vol, np.float32)
    ps = [_small(p, match_size) for p in pngs]
    best_c, best = -2.0, None
    for axis in range(3):
        v = np.moveaxis(vol, axis, 0)
        for z in range(v.shape[0]):
```

```
            s = _small(np.clip(v[z], -1000, 3000), match_size)
            for o, cand in enumerate(_dihedral(s)):
                c = _corr(cand, ps[0])
                if c > best_c:
                    best_c, best = c, (axis, z, o)
    axis, z0, o = best
    v = np.moveaxis(vol, axis, 0)
    zs = [z0]
    for p_small in ps[1:]:
        sc = [_corr(_dihedral(_small(np.clip(v[z], -1000, 3000), match_si
        zs.append(int(np.argmax(sc)))
    xs, ys = [], []
    for p, z in zip(pngs, zs):
        hu = _dihedral(v[z])[o]
        hu = cv2.resize(np.ascontiguousarray(hu, np.float32), (p.shape[1]
        keep = (p > 3) & (p < 252)
        xs.append(hu[keep]); ys.append(p[keep].astype(np.float32))
    x, y = np.concatenate(xs), np.concatenate(ys)
    a, b = np.polyfit(x, y, 1)
    r2 = 1 - ((y - (a * x + b)) ** 2).sum() / max(((y - y.mean()) ** 2).s
    return {"lo": float(-b / a), "hi": float((255 - b) / a), "r2": float(
            "orientation": int(o), "z": zs, "align_corr": float(best_c)}


def _find_ct_nifti(pid, files):
    c = []
    for f in files:
        fl, base = f.lower(), os.path.basename(f).lower()
        if pid.lower() in fl and re.search(r"(^|[_\-.])ct([_\-.]|\.nii)",
            c.append(f)
    return sorted(c, key=len)[0] if c else None


def run_hu_calibration():
    global HU_MIN, HU_MAX, HU_SPAN, HU_MAPPING_VERIFIED
    try:
        import nibabel as nib
    except Exception:
        print("nibabel is not installed -> HU calibration skipped (pip in
    root = NIFTI_ROOT or "/kaggle/input"
    files = glob.glob(os.path.join(root, "**", "*.nii*"), recursive=True)
    print(f"NIfTI files found under {root}: {len(files)}")
    rows = []
    for pid in TRAIN_PIDS[:HU_N_PATIENTS]:
        f = _find_ct_nifti(pid, files)
        if f is None:
            print(f"  {pid}: no CT NIfTI found"); continue
        vol = nib.load(f).get_fdata()
        names = PATIENT_SORTED[pid]
        pick = [names[int(i)] for i in np.linspace(len(names) * 0.25, len
        pngs = [np.array(Image.open(os.path.join(CT_DIR, s)).convert("L")
        r = calibrate_png_to_hu(vol, pngs)
        r.update({"patient": pid, "file": os.path.relpath(f, root), "volu
        rows.append(r); print(f"  {pid}: window [{r['lo']:.0f}, {r['hi']:
```

```
    if not rows:
        print("No CT volumes matched. Set NIFTI_ROOT to the folder that h
    df = pd.DataFrame(rows); df.to_csv(os.path.join(SUITE_DIR, "hu_calibr
    lo, hi = float(df["lo"].median()), float(df["hi"].median())
    ok = bool((df["r2"] > 0.95).all() and (df["align_corr"] > 0.9).all()
              and df["lo"].std(ddof=0) < 0.05 * abs(hi - lo) and df["hi"]
    print(f"\nMedian window: [{lo:.0f}, {hi:.0f}] HU (assumed [{HU_MIN:.0
    if ok:
        HU_MIN, HU_MAX, HU_SPAN, HU_MAPPING_VERIFIED = lo, hi, hi - lo, T
        print("HU window updated; HU_MAPPING_VERIFIED = True (HU MAE and
    else:
        print("Mapping NOT verified (low R2, inconsistent windows or fail
    return df


def bone_dice_hu(pred, gt, mask, thr_hu=BONE_HU):
    thr = 2 * (thr_hu - HU_MIN) / HU_SPAN - 1
    m = mask > 0.5
    p, g = (pred > thr) & m, (gt > thr) & m
    d = p.sum() + g.sum()
    return 1.0 if d == 0 else float(2 * (p & g).sum() / d)


if RUN["hu"]:
    HU_RESULT = run_hu_calibration()
```

```
NIfTI files found under /kaggle/input: 1080
  1BA001: window [-996, 1521] HU  R2 0.9959  align corr 1.000
  1BA005: window [-983, 1643] HU  R2 0.9593  align corr 1.000
  1BA012: window [-995, 1518] HU  R2 0.9988  align corr 1.000

Median window: [-995, 1521] HU (assumed [-1000, 1500]) | consistent & wel
l fitted: True
HU window updated; HU_MAPPING_VERIFIED = True (HU MAE and bone Dice below
are now calibrated).
```

In [38…

```
# %% ====== [P3] ERROR ANALYSIS (slice position, bone vs soft tissue, wor
def slice_index(sname):
    m = re.search(r"slice_(\d+)", sname)
    return int(m.group(1)) if m else 0


def _pmean(v, pid_arr):
    vals = []
    for p in np.unique(pid_arr):
        x = v[pid_arr == p]; x = x[~np.isnan(x)]
        if len(x):
            vals.append(x.mean())
    return float(np.mean(vals)) if vals else float("nan")


def error_analysis_core(preds, gt, mask, pid, sname, thr, span=1.0):
    """preds: label -> [N,H,W]. Errors in [0,1] intensity units (x span =
    N, pid_arr, m = len(pid), np.array(pid), mask > 0.5
    idx = np.array([slice_index(s) for s in sname])
```

```
    third = np.zeros(N, int)
    for p in np.unique(pid_arr):
        ii = np.where(pid_arr == p)[0]; order = ii[np.argsort(idx[ii])]
        third[order] = np.digitize(np.arange(len(order)) / max(len(order)
    bone, soft = (gt > thr) & m, m & ~((gt > thr) & m)
    g01 = (np.clip(gt, -1, 1) + 1) / 2
    rows = []
    for label, p in preds.items():
        err = np.abs((np.clip(p, -1, 1) + 1) / 2 - g01) * span
        sl = np.array([err[i][m[i]].mean() for i in range(N)])
        reg = lambda mk: np.array([err[i][mk[i]].mean() if mk[i].sum() >
        pooled = err[m]
        rows.append({"method": label, "MAE": _pmean(sl, pid_arr), "MAE_bo
                     "MAE_soft": _pmean(reg(soft), pid_arr),
                     "MAE_first_third": _pmean(np.where(third == 0, sl, n
                     "MAE_mid_third": _pmean(np.where(third == 1, sl, np.
                     "MAE_last_third": _pmean(np.where(third == 2, sl, np
                     "worst10pct_slices_MAE": float(np.sort(sl)[-max(1, i
                     "p95_abs_err": float(np.percentile(pooled, 95)), "p9
    return pd.DataFrame(rows)


def run_error_analysis(rows, split="test"):
    keys = method_keys(rows)
    gt, mask, pid, sn = _arrays(split)
    thr = 2 * (BONE_HU - HU_MIN) / HU_SPAN - 1
    verified = bool(globals().get("HU_MAPPING_VERIFIED", False))
    df = error_analysis_core({k: rows[v][0] for k, v in keys.items()}, gt
    unit = "HU" if verified else "[0,1] intensity (HU provisional)"
    print(f"\nERROR ANALYSIS ({split}) in {unit}; bone = GT > {BONE_HU:.0
    print("(first/mid/last third = position in the slice-index order, not
    print(df.round(4 if not verified else 1).to_string(index=False))
    df.to_csv(os.path.join(SUITE_DIR, f"error_analysis_{split}.csv"), ind
    return df


if RUN["error_analysis"]:
    assert "rows_t" in globals(), "Run baseline cells [B6]-[B7] first (ro
    ERR_TEST = run_error_analysis(rows_t, "test")
    if "rows_v" in globals():
        ERR_VAL = run_error_analysis(rows_v, "val")
```

```
ERROR ANALYSIS (test) in HU; bone = GT > 300 HU-equivalent threshold (ver
ified)
(first/mid/last third = position in the slice-index order, not an anatomi
cal claim)
                        method   MAE  MAE_bone  MAE_soft  MAE_first_third
MAE_mid_third  MAE_last_third  worst10pct_slices_MAE  p95_abs_err  p99_ab
s_err
         Stage-1 (4-fold ens.)  73.2     171.9      57.8            94.9
66.3           58.7                   118.4        313.3        608.0
            ResUNet 2.5D + TTA  72.8     171.8      57.4            92.0
67.1           59.5                   118.8        307.5        613.8
                 pix2pix + TTA  65.5     164.8      50.1            85.8
59.4           51.6                   111.1        289.1        593.8
Naive cDDPM (32 samples + TTA) 114.6     150.9     109.1           138.2
104.2          101.8                   170.9        301.3        571.8
            Det. refiner + TTA  65.6     153.6      51.9            81.2
60.2           55.7                   106.0        273.5        550.4
        Ours (diffusion) + TTA  65.3     152.3      51.7            84.5
60.1           51.6                   106.9        271.1        548.4

ERROR ANALYSIS (val) in HU; bone = GT > 300 HU-equivalent threshold (veri
fied)
(first/mid/last third = position in the slice-index order, not an anatomi
cal claim)
                        method   MAE  MAE_bone  MAE_soft  MAE_first_third
MAE_mid_third  MAE_last_third  worst10pct_slices_MAE  p95_abs_err  p99_ab
s_err
         Stage-1 (4-fold ens.)  91.3     194.3      74.2           113.2
78.0           82.7                   174.9        354.4        645.2
            ResUNet 2.5D + TTA  89.8     193.4      72.8           112.9
78.1           78.5                   170.2        352.6        648.4
                 pix2pix + TTA  85.2     190.0      67.8           108.7
73.1           73.7                   163.0        339.2        645.7
Naive cDDPM (32 samples + TTA) 124.9     171.7     118.4           140.5
113.8          120.5                   231.3        345.9        620.5
            Det. refiner + TTA  82.8     174.3      67.8           100.6
74.2           73.7                   164.3        318.6        598.1
        Ours (diffusion) + TTA  83.6     173.0      68.9           102.6
75.0           73.3                   167.9        315.6        594.4
```

In [39…

```python
# Keep optional, expensive experiments disabled for now.
RUN["cost"] = False
RUN["uncertainty"] = False
RUN["matched_baselines"] = False
RUN["extra_seeds"] = False
RUN["extra_splits"] = False
```

In [40…

```python
# %% ====== [P4] COST TABLE (time per slice, peak GPU memory, NFE) ======

def _sub(d, n):
    return {
        k: (v[:n] if (torch.is_tensor(v) or isinstance(v, list)) else v)
        for k, v in d.items()
    }
```

```
def _time(fn):
    if device.type == "cuda":
        torch.cuda.synchronize()
        torch.cuda.reset_peak_memory_stats()

    t0 = time.time()
    fn()

    if device.type == "cuda":
        torch.cuda.synchronize()

    elapsed = time.time() - t0
    peak_mem = (
        torch.cuda.max_memory_allocated() / (2 ** 30)
        if device.type == "cuda"
        else float("nan")
    )

    return elapsed, peak_mem


def run_cost_table(n=16):
    print("=" * 80)
    print("STARTING COST BENCHMARK")
    print(f"Validation slices: {n}")
    print(f"Device: {torch.cuda.get_device_name(0) if torch.cuda.is_avail
    print("=" * 80)

    b = get_bundle(BL_S1)
    d = _sub(get_split("val", VAL_SNAMES, b.offsets), n)

    items = []

    # ------------------------------------------------
    # ResUNet and Pix2Pix direct-inference models
    # Capture each model in the lambda's default argument.
    # ------------------------------------------------

    if "BL_MODELS" in globals():

        if "resunet_25d" in BL_MODELS:
            model = BL_MODELS["resunet_25d"][0]

            items += [
                (
                    "ResUNet 2.5D",
                    lambda model=model, data=d:
                        predict_direct(model, data),
                    1
                ),
                (
                    "ResUNet 2.5D + TTA",
```

```
                    lambda model=model, data=d:
                        predict_direct(model, data, flips=(False, True)),
                    2
                ),
            ]

        if "pix2pix" in BL_MODELS:
            model = BL_MODELS["pix2pix"][0]

            items.append(
                (
                    "pix2pix + TTA",
                    lambda model=model, data=d:
                        predict_direct(model, data, flips=(False, True)),
                    2
                )
            )

    # ------------------------------------------------
    # Stage-1 ensemble
    # ------------------------------------------------

    items.append(
        (
            "Stage-1 4-fold ensemble",
            lambda bundle=b, data=d:
                stage1_ensemble(bundle, data),
            4
        )
    )

    # ------------------------------------------------
    # Deterministic refiner
    # ------------------------------------------------

    if "DET_MODELS" in globals() and DET_MODELS:
        cfg_det, model_det = DET_MODELS[0]

        items.append(
            (
                "Det. refiner, 4 folds + TTA",
                lambda model=model_det, cfg=cfg_det, bundle=b, data=d:
                    predict_stack(
                        data, model, cfg, bundle, 1, 1,
                        folds=(1, 2, 3, 4),
                        flips=(False, True)
                    ),
                8
            )
        )

    # ------------------------------------------------
    # Final CCR diffusion model
    # ------------------------------------------------
```

```
fm = globals().get("final_model")

if fm is None:
    fm = get_stage2(
        FINAL_TAG_,
        FINAL_CFG_,
        FINAL_EPOCHS
    )

final_cfg = FINAL_CFG_
final_bundle = get_bundle(final_cfg["s1"])

items += [
    (
        "Ours: 1 sample, 1 fold",
        lambda model=fm, cfg=final_cfg, bundle=final_bundle, data=d:
            predict_stack(
                data, model, cfg, bundle,
                1, 50,
                folds=(1,)
            ),
        51
    ),
    (
        "Ours: full (4 folds x 4 samples x 2 flips)",
        lambda model=fm, cfg=final_cfg, bundle=final_bundle, data=d:
            predict_stack(
                data, model, cfg, bundle,
                FINAL_PROTO["samples"],
                FINAL_PROTO["steps"],
                folds=FINAL_PROTO["folds"],
                flips=FINAL_PROTO["flips"]
            ),
        1608
    ),
]

# ------------------------------------------------
# Run warm-up and timing
# ------------------------------------------------

rows = []

total = len(items)

for idx, (name, fn, nfe) in enumerate(items, start=1):
    print(f"\n[{idx}/{total}] Benchmarking: {name}")

    # Warm-up pass, as in the original cost table.
    print("  Warm-up inference...")
    fn()

    print("  Measuring inference time and peak GPU memory...")
```

```
        elapsed, mem = _time(fn)

        seconds_per_slice = elapsed / n

        rows.append({
            "method": name,
            "NFE/slice": nfe,
            "sec/slice": seconds_per_slice,
            "peak_GPU_GB": mem
        })

        print(f"  Elapsed time: {elapsed:.2f} seconds")
        print(f"  Time per slice: {seconds_per_slice:.4f} seconds")
        print(f"  Peak GPU memory: {mem:.3f} GB" if np.isfinite(mem)
              else "  Peak GPU memory: unavailable")

        # Save after every completed model so results survive interruptio
        df = pd.DataFrame(rows)
        output_path = os.path.join(SUITE_DIR, "cost_table.csv")
        df.to_csv(output_path, index=False)

    print("\n" + "=" * 80)
    print("COST BENCHMARK COMPLETE")
    print("=" * 80)

    print(
        f"\nCOST (batch of {n} validation slices, "
        f"{torch.cuda.get_device_name(0) if torch.cuda.is_available() els
    )

    print(df.round(4).to_string(index=False))
    print(f"\nSaved cost table: {output_path}")

    return df


if RUN["cost"]:
    COST = run_cost_table()
else:
    print("Cost benchmark skipped because RUN['cost'] is False.")
```

```
Cost benchmark skipped because RUN['cost'] is False.
```

In [41…

```
# %% ====== [P5] UNCERTAINTY vs ERROR ==================================
def members_from_stack(stack):
    return np.clip(stack["coarse"][:, :, None] + stack["res"], -1, 1)


def unc_vs_err(members, gt, mask, pid, n_sub=300000, seed=0):
    mean, std = members.mean((0, 1, 2)), members.std((0, 1, 2))
    err = np.abs((mean + 1) / 2 - (np.clip(gt, -1, 1) + 1) / 2)
    m = mask > 0.5
    u, e = std[m], err[m]
    rng = np.random.RandomState(seed)
    ii = rng.choice(len(u), size=min(n_sub, len(u)), replace=False)
```

```
    per_patient = []
    for p in sorted(set(pid)):
        idx = [i for i, q in enumerate(pid) if q == p]
        up, ep = std[idx][m[idx]], err[idx][m[idx]]
        jj = rng.choice(len(up), size=min(60000, len(up)), replace=False)
        per_patient.append(float(spearmanr(up[jj], ep[jj])[0]))
    edges = np.quantile(u, np.linspace(0, 1, 11))
    b = np.clip(np.digitize(u, edges[1:-1]), 0, 9)
    dec = np.array([e[b == k].mean() for k in range(10)])
    return {"spearman_pooled": float(spearmanr(u[ii], e[ii])[0]), "spearm
            "spearman_per_patient_min": float(np.min(per_patient)), "deci
            "top_over_bottom_decile": float(dec[-1] / max(dec[0], 1e-12))


def _diff_stack(split):
    nm = "val_stack" if split == "val" else "test_stack"
    if nm in globals():
        return globals()[nm]
    m = globals().get("final_model") or get_stage2(FINAL_TAG_, FINAL_CFG_
    b = get_bundle(FINAL_CFG_["s1"]); names = VAL_SNAMES if split == "val
    return predict_stack(get_split(split, names, b.offsets), m, FINAL_CFG
                         folds=FINAL_PROTO["folds"], flips=FINAL_PROTO["f


def _det_stack(split):
    cfg = make_cfg({"mode": "det", "film": False, "seed": SEED}, BL_S1)
    m = DET_MODELS[0][1] if "DET_MODELS" in globals() and DET_MODELS else
    b = get_bundle(cfg["s1"]); names = VAL_SNAMES if split == "val" else
    return predict_stack(get_split(split, names, b.offsets), m, cfg, b, 1


def run_uncertainty(split):
    gt, mask, pid, _ = _arrays(split)
    sd, st = _diff_stack(split), _det_stack(split)
    src = {"Deterministic refiner: 8 members": members_from_stack(st),
           "Diffusion: 8 members (1 sample per fold x flip)": members_fro
           "Diffusion: 32 members": members_from_stack(sd)}
    rows, curves = [], {}
    for name, mem in src.items():
        r = unc_vs_err(mem, gt, mask, pid); curves[name] = r["decile_mae"
        rows.append({"source": name, **{k: v for k, v in r.items() if k !
    df = pd.DataFrame(rows)
    print(f"\nUNCERTAINTY vs ERROR ({split}); higher Spearman = spread tr
    print(df.round(4).to_string(index=False)); df.to_csv(os.path.join(SUI
    fig, ax = plt.subplots(figsize=(6.5, 4.2))
    for name, dec in curves.items():
        ax.plot(range(1, 11), dec, marker="o", label=name)
    ax.set_xlabel("uncertainty decile (1 = most certain)"); ax.set_ylabel
    ax.set_title(f"Does the spread predict error? ({split})"); ax.grid(al
    plt.tight_layout(); plt.savefig(os.path.join(SUITE_DIR, f"uncertainty
    return df


if RUN["uncertainty"]:
```

```
    UNC_VAL = run_uncertainty("val")
    UNC_TEST = run_uncertainty("test")
```

In [42…

```
# %% ====== [P6] FULL STATISTICS FOR THE MAIN TABLE ====================
def holm(p):
    p = np.asarray(p, float); adj = np.full(len(p), np.nan)
    ok = ~np.isnan(p); idx = np.where(ok)[0]; order = idx[np.argsort(p[id
    for rank, i in enumerate(order):
        run = max(run, (len(order) - rank) * p[i]); adj[i] = min(1.0, run
    return adj


def paired_full(pred_a, pred_b, gt, mask, pid, n_boot=10000, seed=0):
    """b minus a per patient: mean diff, bootstrap CI, Wilcoxon p, Cohen'
    _, da = patient_level(pred_a, gt, mask, pid); _, db = patient_level(p
    da = da.set_index("patient_id"); db = db.set_index("patient_id").loc[
    rng, out = np.random.RandomState(seed), []
    for met, sign in (("mae", -1), ("rmse", -1), ("ssim", 1), ("psnr", 1)
        d = (db[met] - da[met]).values
        boots = rng.choice(d, size=(n_boot, len(d)), replace=True).mean(1
        lo, hi = np.percentile(boots, [2.5, 97.5])
        p = float(wilcoxon(d).pvalue) if np.any(d != 0) else float("nan")
        out.append({"metric": met, "mean_diff(b-a)": d.mean(), "CI95_lo":
                    "cohens_dz": float(d.mean() / d.std(ddof=1)) if len(d
                    "patients_improved": int((sign * d > 0).sum()), "n":
    return pd.DataFrame(out)


def main_stats(rows, split="test"):
    gt, mask, pid, _ = _arrays(split)
    keys = method_keys(rows); ours = keys.pop("Ours (diffusion) + TTA")
    tabs = []
    for short, lab in keys.items():
        t = paired_full(rows[lab][0], rows[ours][0], gt, mask, pid); t.in
    df = pd.concat(tabs, ignore_index=True)
    df["wilcoxon_p_holm"] = np.nan
    for met in df["metric"].unique():                                    #
        sel = df["metric"] == met; df.loc[sel, "wilcoxon_p_holm"] = holm(
    print(f"\nPAIRED STATISTICS ({split}): Ours minus baseline. With n={d
          f"Wilcoxon p is {2 / 2 ** df['n'].iloc[0]:.4f}")
    print(df.round(4).to_string(index=False)); df.to_csv(os.path.join(SUI
    return df


if RUN["main_stats"]:
    assert "rows_t" in globals(), "Run baseline cells [B6]-[B7] first (ro
    STATS_TEST = main_stats(rows_t, "test")
```

```
PAIRED STATISTICS (test): Ours minus baseline. With n=6 patients the smal
lest possible Wilcoxon p is 0.0312
                    baseline metric  mean_diff(b-a)  CI95_lo  CI95_hi
wilcoxon_p  cohens_dz  patients_improved  n  wilcoxon_p_holm
         Stage-1 (4-fold ens.)    mae         -0.0031  -0.0038  -0.0024
0.0312    -3.4125                  6  6          0.1562
         Stage-1 (4-fold ens.)   rmse         -0.0064  -0.0069  -0.0059
0.0312    -8.9956                  6  6          0.1562
         Stage-1 (4-fold ens.)   ssim          0.0204   0.0175   0.0226
0.0312     5.7032                  6  6          0.1562
         Stage-1 (4-fold ens.)   psnr          1.0791   0.9428   1.2818
0.0312     4.4239                  6  6          0.1562
           ResUNet 2.5D + TTA    mae         -0.0030  -0.0036  -0.0024
0.0312    -3.4011                  6  6          0.1562
           ResUNet 2.5D + TTA   rmse         -0.0062  -0.0069  -0.0055
0.0312    -6.3614                  6  6          0.1562
           ResUNet 2.5D + TTA   ssim          0.0162   0.0142   0.0183
0.0312     5.7230                  6  6          0.1562
           ResUNet 2.5D + TTA   psnr          1.0473   0.8766   1.2391
0.0312     4.2137                  6  6          0.1562
                 pix2pix + TTA    mae         -0.0001  -0.0018   0.0016
1.0000    -0.0328                  3  6          1.0000
                 pix2pix + TTA   rmse         -0.0033  -0.0052  -0.0016
0.0625    -1.3300                  5  6          0.1562
                 pix2pix + TTA   ssim          0.0070   0.0026   0.0124
0.0625     1.0590                  5  6          0.1562
                 pix2pix + TTA   psnr          0.5545   0.2498   0.8749
0.0625     1.2611                  5  6          0.1562
Naive cDDPM (32 samples + TTA)    mae         -0.0196  -0.0243  -0.0148
0.0312    -2.9591                  6  6          0.1562
Naive cDDPM (32 samples + TTA)   rmse         -0.0114  -0.0152  -0.0072
0.0312    -2.1061                  6  6          0.1562
Naive cDDPM (32 samples + TTA)   ssim          0.0065   0.0023   0.0097
0.0625     1.2749                  5  6          0.1562
Naive cDDPM (32 samples + TTA)   psnr          1.8238   1.1670   2.3752
0.0312     2.1855                  6  6          0.1562
           Det. refiner + TTA    mae         -0.0001  -0.0009   0.0006
1.0000    -0.1323                  3  6          1.0000
           Det. refiner + TTA   rmse         -0.0005  -0.0010  -0.0001
0.1562    -0.8480                  4  6          0.1562
           Det. refiner + TTA   ssim          0.0031   0.0020   0.0043
0.0312     1.8931                  6  6          0.1562
           Det. refiner + TTA   psnr          0.0914  -0.0017   0.2106
0.2188     0.6097                  4  6          0.2188
```

In [43…

```python
# %% ====== [P7] CAPACITY-MATCHED BASELINES: 4-model pix2pix ensemble, Re
def train_pix2pix_sub(name, offsets, epochs, exclude_fold=None):
    """pix2pix (ResUNet generator + PatchGAN, LSGAN + P2P_L1 x L1). exclu
    exclude fold k (same folds as Stage-1); None = all training patients.
    best_p, last_p = os.path.join(BL_DIR, f"{name}_best.pth"), os.path.jo
    G = Stage1ResUNet(len(offsets), 1, S1_BASE).to(device)
    if os.path.exists(_done(name)):
        G.load_state_dict(_ld(best_p)["G"]); return G.eval()
    seed_everything(SEED + (exclude_fold or 0))
```

```
    D = PatchD(len(offsets) + 1).to(device)
    d_tr, d_va = get_split("train", TRAIN_SNAMES, offsets), get_split("va
    idx = list(range(len(d_tr["pid"]))) if exclude_fold is None else (d_t
    loader = DataLoader(Subset(SplitDataset(d_tr, True), idx), batch_size
                        num_workers=NUM_WORKERS, drop_last=True)
    optG = torch.optim.Adam(G.parameters(), lr=P2P_LR, betas=(0.5, 0.999)
    optD = torch.optim.Adam(D.parameters(), lr=P2P_LR, betas=(0.5, 0.999)
    start, best = 0, float("inf")
    if os.path.exists(last_p):
        ck = _ld(last_p)
        G.load_state_dict(ck["G"]); D.load_state_dict(ck["D"]); optG.load
        start, best = ck["epoch"], ck["best"]; print(f"[resume] {name} fr
    half, t0 = epochs // 2, time.time()
    for ep in range(start + 1, epochs + 1):
        lr = P2P_LR if ep <= half else P2P_LR * (epochs - ep + 1) / (epoc
        for o in (optG, optD):
            for g in o.param_groups:
                g["lr"] = lr
        G.train(); D.train()
        for b in loader:
            mri, ct, mask = b["mri"].to(device), b["ct"].to(device), b["m
            ct_t = torch.where(mask > 0.5, ct, torch.full_like(ct, -1.0))
            fake = G(mri); fake_m = torch.where(mask > 0.5, fake, torch.f
            optD.zero_grad(set_to_none=True)
            d_real, d_fake = D(torch.cat([mri, ct_t], 1)), D(torch.cat([m
            (0.5 * (F.mse_loss(d_real, torch.ones_like(d_real)) + F.mse_l
            optG.zero_grad(set_to_none=True)
            d_g = D(torch.cat([mri, fake_m], 1))
            (F.mse_loss(d_g, torch.ones_like(d_g)) + P2P_L1 * _l1(fake, c
        if ep % 5 == 0 or ep == epochs:
            sc, m = direct_val_score(G, d_va)
            star = ""
            if sc < best:
                best, star = sc, "  <-- best"; torch.save({"G": G.state_d
            print(f"[{name}] ep {ep:3d} val Score {sc:.4f} PSNR {m['PSNR'
        torch.save({"G": G.state_dict(), "D": D.state_dict(), "optG": opt
                    "epoch": ep, "best": best}, last_p)
    open(_done(name), "w").write("ok")
    G.load_state_dict(_ld(best_p)["G"])
    return G.eval()


def run_matched_baselines():
    p2p = {k: train_pix2pix_sub(f"p2pfold{k}", BL_OFFSETS, ENS_P2P_EPOCHS
    res = train_resunet(f"resunet_25d_e{MATCHED_RESUNET_EPOCHS}", BL_OFFS
    pp = sum(nparams(m) for m in p2p.values())
    out = {}
    for split in ("val", "test"):
        names = VAL_SNAMES if split == "val" else TEST_SNAMES
        d = get_split(split, names, BL_OFFSETS)
        out[split] = {f"pix2pix 4-model ensemble + flip-TTA ({ENS_P2P_EPO
                      (np.clip(np.mean([predict_direct(m, d, flips=(False
                      f"ResUNet 2.5D ({MATCHED_RESUNET_EPOCHS} ep) + flip
    return out
```

```
def report_matched(out):
    for split, rows_x in out.items():
        gt, mask, pid, _ = _arrays(split)
        base = rows_t if split == "test" else rows_v
        keys = method_keys(base); lines = []
        for lab, (p, nfe, par) in list(rows_x.items()) + [(k, (base[v][0]
            m, _ = patient_level(p, gt, mask, pid)
            lines.append({"method": lab, "MAE": m["MAE"], "RMSE": m["RMSE
                          "NFE": nfe, "Params(M)": par})
        df = pd.DataFrame(lines); print(f"\nMATCHED BASELINES ({split})")
        df.to_csv(os.path.join(SUITE_DIR, f"matched_baselines_{split}.csv
        if split == "test":
            ours = keys["Ours (diffusion) + TTA"]
            for lab, (p, _, _) in rows_x.items():
                print(f"\n=== Ours minus [{lab}] (paired, n={len(set(pid)
                print(paired_full(p, base[ours][0], gt, mask, pid).round(


if RUN["matched_baselines"]:
    assert "rows_t" in globals() and "rows_v" in globals()
    MATCHED = run_matched_baselines(); report_matched(MATCHED)
```

In [44…

```
# %% ====== [P8] EXTRA SEEDS (final diffusion model + deterministic refin
def _seed_predictions(sd, split):
    names = VAL_SNAMES if split == "val" else TEST_SNAMES
    gt, mask, pid, _ = _arrays(split)
    b = get_bundle(BL_S1); d = get_split(split, names, b.offsets)
    cfg_f = make_cfg({**ABLATIONS[FINAL_NAME_], "seed": sd}, BL_S1)
    cfg_d = make_cfg({"mode": "det", "film": False, "seed": sd}, BL_S1)
    m_f = get_stage2(f"{FINAL_TAG_}_seed{sd}", cfg_f, FINAL_EPOCHS)
    m_d = get_stage2(f"det_noFiLM_s{sd}", cfg_d, FINAL_EPOCHS)
    c, r = aggregate(predict_stack(d, m_f, cfg_f, b, FINAL_PROTO["samples
    pf = finalize(c, r, mask)
    c, r = aggregate(predict_stack(d, m_d, cfg_d, b, 1, 1, folds=(1, 2, 3
    return pf, finalize(c, r, mask)


def run_extra_seeds(split="test"):
    gt, mask, pid, _ = _arrays(split)
    base = rows_t if split == "test" else rows_v
    keys = method_keys(base)
    per = {"Diffusion": {SEED: base[keys["Ours (diffusion) + TTA"]][0]},
    for sd in EXTRA_SEEDS:
        pf, pd_ = _seed_predictions(sd, split)
        per["Diffusion"][sd], per["Deterministic"][sd] = pf, pd_
    rows = []
    for fam, dd in per.items():
        for sd, p in dd.items():
            m, _ = patient_level(p, gt, mask, pid)
            rows.append({"family": fam, "seed": sd, **{k: m[k] for k in (
    df = pd.DataFrame(rows)
    summ = df.groupby("family")[["MAE", "RMSE", "SSIM", "PSNR", "Score"]]
```

```
    print(f"\nSEEDS ({split}): per-seed results"); print(df.round(4).to_s
    print(f"\nMean and SD over {len(per['Diffusion'])} seeds"); print(sum
    df.to_csv(os.path.join(SUITE_DIR, f"seeds_{split}_per_seed.csv"), ind
    summ.to_csv(os.path.join(SUITE_DIR, f"seeds_{split}_summary.csv"))
    print("\nDiffusion minus deterministic refiner, per seed (paired over
    for sd in per["Diffusion"]:
        s = paired_full(per["Deterministic"][sd], per["Diffusion"][sd], g
        print(f"  seed {sd}: " + "  ".join(f"{r.metric}: {r['mean_diff(b-
    return df, summ


if RUN["extra_seeds"]:
    assert "rows_t" in globals()
    SEEDS_TEST = run_extra_seeds("test")
    if SEED_EVAL_VAL:
        SEEDS_VAL = run_extra_seeds("val")
```

In [45…

```
# %% ====== [P9] EXTRA DISJOINT PATIENT-LEVEL SPLITS (generalisation; red
ALL_PIDS = sorted(PATIENT_SORTED)
ORIG = dict(TRAIN_PIDS=list(TRAIN_PIDS), VAL_PIDS=list(VAL_PIDS), TEST_PI
            DIR_S1=DIR_S1, DIR_S2=DIR_S2, DIR_METRICS=DIR_METRICS, DIR_FI


def set_split(train_pids, val_pids, test_pids, tag=None, fold_seed=None):
    """Switch the whole pipeline (data, Stage-1 folds, checkpoints, basel
    global TRAIN_PIDS, VAL_PIDS, TEST_PIDS, FOLDS, TRAIN_SNAMES, VAL_SNAM
    global DIR_S1, DIR_S2, DIR_METRICS, DIR_FIG, BL_DIR, BV_GT, BV_MASK,
    assert len(train_pids) == 12 * N_FOLDS, "this pipeline needs 48 train
    TRAIN_PIDS, VAL_PIDS, TEST_PIDS = sorted(train_pids), sorted(val_pids
    assert not (set(TRAIN_PIDS) & set(VAL_PIDS)) and not (set(TRAIN_PIDS)
    order = TRAIN_PIDS if fold_seed is None else [str(x) for x in np.rand
    FOLDS = {k + 1: sorted(order[k * 12:(k + 1) * 12]) for k in range(N_F
    PID_TO_FOLD.clear(); PID_TO_FOLD.update({pid: k for k, pids in FOLDS.
    TRAIN_SNAMES, VAL_SNAMES, TEST_SNAMES = snames_of(TRAIN_PIDS), snames
    SPLITS.clear(); BUNDLES.clear()
    if tag is None:
        DIR_S1, DIR_S2, DIR_METRICS, DIR_FIG, BL_DIR = (ORIG[k] for k in
    else:
        base = os.path.join(BASE_OUT_DIR, f"split_{tag}")
        DIR_S1, DIR_S2, DIR_METRICS, DIR_FIG, BL_DIR = [os.path.join(base
        for x in (DIR_S1, DIR_S2, DIR_METRICS, DIR_FIG, BL_DIR):
            os.makedirs(x, exist_ok=True)
    v0 = get_split("val", VAL_SNAMES, (0,))
    BV_GT, BV_MASK, BV_PID = v0["ct"][:, 0].numpy(), v0["mask"][:, 0].num


def restore_original():
    set_split(ORIG["TRAIN_PIDS"], ORIG["VAL_PIDS"], ORIG["TEST_PIDS"], ta


def make_extra_splits(n_splits, seed=123):
    """Disjoint test groups of 6 drawn from patients NOT in the original
    others = sorted(set(ALL_PIDS) - set(ORIG["TEST_PIDS"]))
    perm = [str(x) for x in np.random.RandomState(seed).permutation(other
```

```
    assert 6 * n_splits <= len(perm)
    out = []
    for g in range(n_splits):
        te = sorted(perm[6 * g:6 * g + 6])
        rest = [str(x) for x in np.random.RandomState(seed + g + 1).permu
        out.append((f"x{g + 1}", sorted(rest[6:]), sorted(rest[:6]), te))
    return out


def run_split(tag, tr, va, te):
    csv = os.path.join(SUITE_DIR, f"cv_{tag}_per_patient.csv")
    if os.path.exists(csv):
        print(f"[skip] split {tag}: {csv} exists"); return pd.read_csv(cs
    set_split(tr, va, te, tag, fold_seed=SEED)
    print(f"\n===== split {tag}: train {len(tr)} | val {len(va)} | test {
    cfg_f = make_cfg(ABLATIONS[FINAL_NAME_], BL_S1)
    cfg_d = make_cfg({"mode": "det", "film": False, "seed": SEED}, BL_S1)
    b = get_bundle(BL_S1)
    m_res = train_resunet("resunet_25d", BL_OFFSETS, epochs=CV_RESUNET_EP
    m_p2p = train_pix2pix_sub("pix2pix_25d", BL_OFFSETS, CV_P2P_EPOCHS) i
    m_det = get_stage2("det", cfg_d, CV_EPOCHS)
    m_dif = get_stage2("diff", cfg_f, CV_EPOCHS)
    d0 = get_split("test", TEST_SNAMES, (0,)); gt, mk, pid = d0["ct"][:,
    d = get_split("test", TEST_SNAMES, b.offsets)
    preds = {"Stage-1 (4-fold ens.)": finalize(stage1_ensemble(b, d), 0,
             "ResUNet 2.5D + TTA": predict_direct(m_res, d, flips=(False,
    if m_p2p is not None:
        preds["pix2pix + TTA"] = predict_direct(m_p2p, d, flips=(False, T
    c, r = aggregate(predict_stack(d, m_det, cfg_d, b, 1, 1, folds=(1, 2,
    preds["Det. refiner + TTA"] = finalize(c, r, mk)
    c, r = aggregate(predict_stack(d, m_dif, cfg_f, b, CV_SAMPLES, CV_STE
    preds["Ours (diffusion) + TTA"] = finalize(c, r, mk)
    frames = []
    for name, p in preds.items():
        _, df = patient_level(p, gt, mk, pid); df.insert(0, "method", nam
    out = pd.concat(frames, ignore_index=True); out.to_csv(csv, index=Fal
    return out


def cv_summary(per_patient):
    ours = "Ours (diffusion) + TTA"
    mean = per_patient.groupby("method")[["mae", "rmse", "ssim", "psnr"]]
    print(f"\nEXTRA SPLITS: mean and SD over {per_patient['patient_id'].n
    print(mean.round(4).to_string()); mean.to_csv(os.path.join(SUITE_DIR,
    tabs = []
    for base in [m for m in per_patient["method"].unique() if m != ours]:
        a = per_patient[per_patient["method"] == base].set_index("patient
        b_ = per_patient[per_patient["method"] == ours].set_index("patien
        rng = np.random.RandomState(0)
        for met, sign in (("mae", -1), ("ssim", 1), ("psnr", 1)):
            dd = (b_[met] - a[met]).values
            boots = rng.choice(dd, size=(10000, len(dd)), replace=True).m
            tabs.append({"baseline": base, "metric": met, "mean_diff(ours
                         "CI95_lo": np.percentile(boots, 2.5), "CI95_hi":
```

```
                            "wilcoxon_p": float(wilcoxon(dd).pvalue) if np.a
                            "cohens_dz": float(dd.mean() / dd.std(ddof=1)) i
                            "patients_improved": int((sign * dd > 0).sum()),
    df = pd.DataFrame(tabs); df["wilcoxon_p_holm"] = np.nan
    for met in df["metric"].unique():
        s = df["metric"] == met; df.loc[s, "wilcoxon_p_holm"] = holm(df.l
    print("\nPaired comparison pooled over all extra-split test patients"
    df.to_csv(os.path.join(SUITE_DIR, "cv_paired_stats.csv"), index=False
    return df


if RUN["extra_splits"]:
    try:
        frames = [run_split(*s) for s in make_extra_splits(CV_SPLITS)]
    finally:
        restore_original()
    CV_ALL = pd.concat(frames, ignore_index=True)
    CV_STATS = cv_summary(CV_ALL)
```

APPENDED ADD-ON 2: CCR v4 Improvements (C0–C3)

This add-on follows the paper suite. The original base notebook and the paper-suite cells above are retained.

In [45…

```
# ======================================================================
# CCR v4 -- IMPROVEMENT EXPERIMENTS (add-on; nothing here retrains your v
#
#   [C1] Uncertainty analysis (NO training): does the diffusion sample sp
#        than the disagreement of a deterministic ensemble?  -> gives dif
#   [C2] Deterministic refiner + adversarial loss ("Det+GAN"): the residu
#        L1 + SSIM + a small PatchGAN term (pix2pix did help the ResUNet)
#   [C3] Model-family blend (NO training): convex blend of diffusion / de
#        with weights chosen on VALIDATION only.
#
# RUN ORDER
#   v3 cells [0]-[5]  ->  baselines cells [B0]-[B7] (so rows_v / rows_t /
#
# RULES
#   * every selection (blend weights, GAN checkpoint) uses the 6 VALIDATI
#   * the test patients are scored once more at the end of [C3] (disclose
#   * all of this is exploratory: keep what helps on validation, report t
# ======================================================================
```

In [46…

```
# %% ====== [C0] CHECKS + CONFIG ====================================
import os, time, math, json, copy
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
```

```
import torch.nn.functional as F
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
from torch.utils.data import DataLoader
from scipy.stats import spearmanr

_need = ["get_split", "SplitDataset", "DiffUNet", "EMA", "DifferentiableS
         "patient_level", "paired_stats", "predict_stack", "aggregate", "
         "get_stage2", "TRAIN_SNAMES", "VAL_SNAMES", "TEST_SNAMES", "seed
         "T_DIM", "BATCH_SIZE", "NUM_WORKERS", "device", "use_amp", "SEED
         "EMA_DECAY", "DIR_METRICS", "DIR_FIG", "BL_S1", "ABL_EPOCHS", "A
_missing = [n for n in _need if n not in globals()]
assert not _missing, f"Run v3 cells [0]-[5] and baseline cells [B0]-[B7]

RUN_UNCERTAINTY = True
RUN_DETGAN = False                  # set True to train the Det+GAN refin
RUN_BLEND = True
INCLUDE_DETGAN_IN_BLEND = False     # only if the Det+GAN refiner was tra
GAN_EPOCHS, GAN_ADV_W, GAN_D_LR = FINAL_EPOCHS, 0.08, 1e-4   # 0.08 ~ pix
V4_DIR = os.path.join(BASE_OUT_DIR, "v4")
os.makedirs(V4_DIR, exist_ok=True)

FINAL_NAME_ = globals().get("FINAL_NAME_") or globals().get("FINAL_NAME")
FINAL_CFG_ = globals().get("FINAL_CFG_") or globals().get("FINAL_CFG") or
FINAL_TAG_ = globals().get("FINAL_TAG_") or globals().get("FINAL_TAG") or
DET_CFG = make_cfg({"mode": "det", "film": False, "seed": SEED}, BL_S1)


def _split_arrays(split):
    names = VAL_SNAMES if split == "val" else TEST_SNAMES
    d0 = get_split(split, names, (0,))
    return d0["ct"][:, 0].numpy(), d0["mask"][:, 0].numpy(), d0["pid"], n


def get_diff_stack(split):
    nm = "val_stack" if split == "val" else "test_stack"
    if nm in globals():
        return globals()[nm]
    m = globals().get("final_model") or get_stage2(FINAL_TAG_, FINAL_CFG_
    b = get_bundle(FINAL_CFG_["s1"]); _, _, _, names = _split_arrays(spli
    return predict_stack(get_split(split, names, b.offsets), m, FINAL_CFG


def get_det_stack(split):
    m = DET_MODELS[0][1] if "DET_MODELS" in globals() and DET_MODELS else
    b = get_bundle(DET_CFG["s1"]); _, _, _, names = _split_arrays(split)
    return predict_stack(get_split(split, names, b.offsets), m, DET_CFG,
```

In [47…

```
# %% ====== [C1] UNCERTAINTY ANALYSIS (numpy only; no training) =========
def members_from_stack(stack):
    """All ensemble members as final CTs: (F, L, S, N, H, W)."""
    return np.clip(stack["coarse"][:, :, None] + stack["res"], -1, 1)
```

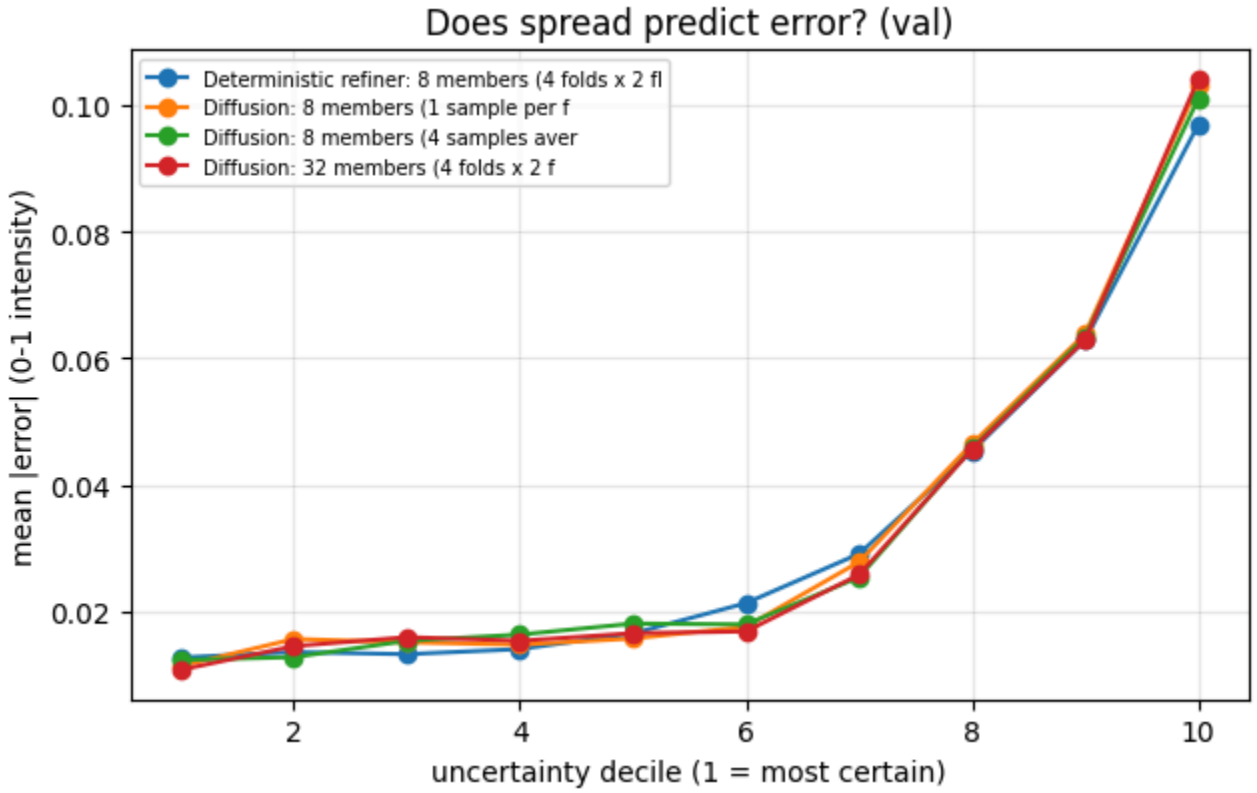
```python
def unc_vs_err(members, gt, mask, pid, n_sub=300000, seed=0):
    """Spread (std over members) vs absolute error of the member mean, ma
    mean, std = members.mean((0, 1, 2)), members.std((0, 1, 2))
    err = np.abs((mean + 1) / 2 - (np.clip(gt, -1, 1) + 1) / 2)
    m = mask > 0.5
    u, e = std[m], err[m]
    rng = np.random.RandomState(seed)
    ii = rng.choice(len(u), size=min(n_sub, len(u)), replace=False)
    rho = float(spearmanr(u[ii], e[ii])[0])
    per_patient = []
    for p in sorted(set(pid)):
        idx = [i for i, q in enumerate(pid) if q == p]
        mp = m[idx]
        up, ep = std[idx][mp], err[idx][mp]
        jj = rng.choice(len(up), size=min(60000, len(up)), replace=False)
        per_patient.append(float(spearmanr(up[jj], ep[jj])[0]))
    edges = np.quantile(u, np.linspace(0, 1, 11))
    b = np.clip(np.digitize(u, edges[1:-1]), 0, 9)
    dec = np.array([e[b == k].mean() for k in range(10)])
    return {"spearman_pooled": rho, "spearman_per_patient_mean": float(np
            "spearman_per_patient_min": float(np.min(per_patient)), "deci
            "top_over_bottom_decile": float(dec[-1] / max(dec[0], 1e-12))


def uncertainty_report(split):
    gt, mask, pid, _ = _split_arrays(split)
    sd, st = get_diff_stack(split), get_det_stack(split)
    src = {
        "Deterministic refiner: 8 members (4 folds x 2 flips)": members_f
        "Diffusion: 8 members (1 sample per fold x flip)": members_from_s
        "Diffusion: 8 members (4 samples averaged per fold x flip)": memb
        "Diffusion: 32 members (4 folds x 2 flips x 4 samples)": members_
    }
    rows, curves = [], {}
    for name, mem in src.items():
        r = unc_vs_err(mem, gt, mask, pid)
        curves[name] = r["decile_mae"]
        rows.append({"source": name, **{k: v for k, v in r.items() if k !
        del mem
    df = pd.DataFrame(rows)
    print(f"\nUNCERTAINTY vs ERROR ({split}); higher Spearman = spread tr
    print(df.round(4).to_string(index=False))
    df.to_csv(os.path.join(DIR_METRICS, f"uncertainty_{split}.csv"), inde
    fig, ax = plt.subplots(figsize=(6.5, 4.2))
    for name, dec in curves.items():
        ax.plot(range(1, 11), dec, marker="o", label=name.split(":")[0] +
    ax.set_xlabel("uncertainty decile (1 = most certain)"); ax.set_ylabel
    ax.set_title(f"Does spread predict error? ({split})"); ax.grid(alpha=
    plt.tight_layout(); plt.savefig(os.path.join(DIR_FIG, f"uncertainty_{
    return df

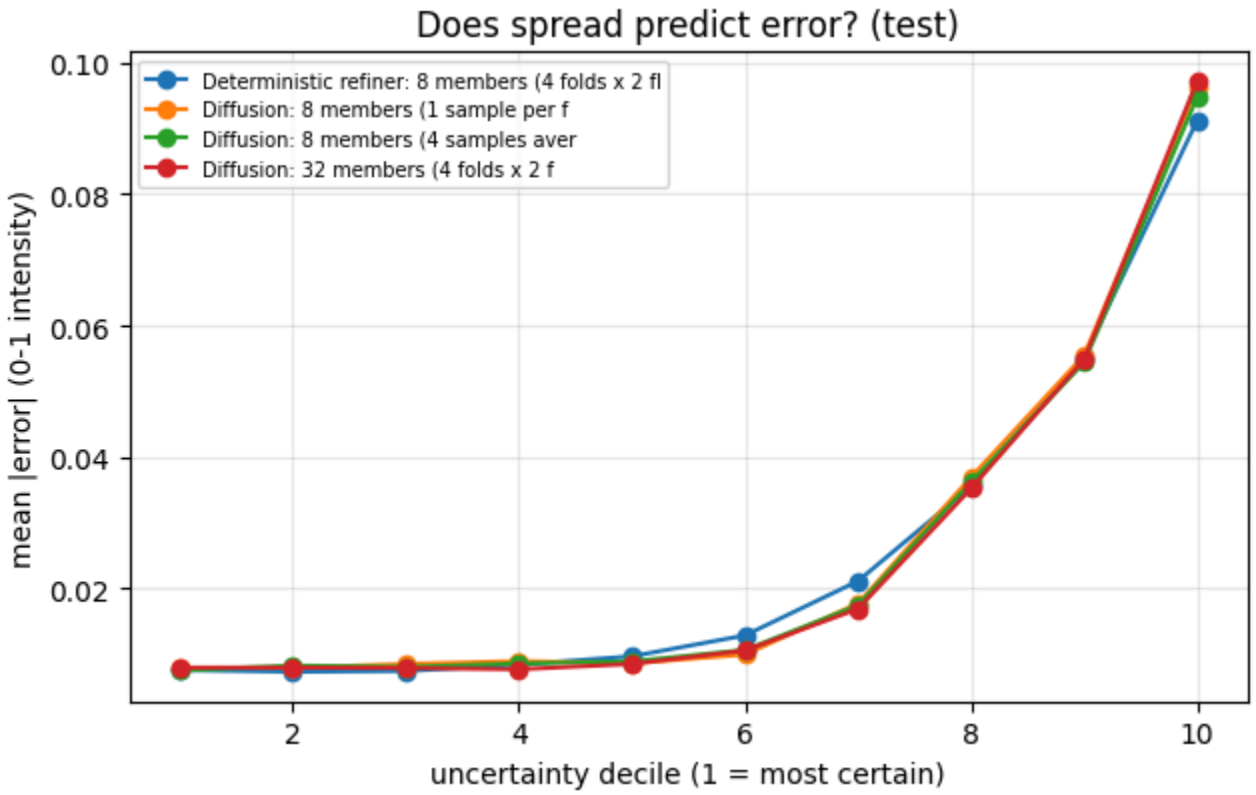

if RUN_UNCERTAINTY:
    unc_val = uncertainty_report("val")     # primary (validation)
```

```
    unc_test = uncertainty_report("test")   # descriptive only; nothing i
```

```
UNCERTAINTY vs ERROR (val); higher Spearman = spread tracks the real erro
r better
                                                  source  spearman_poole
d  spearman_per_patient_mean  spearman_per_patient_min  top_over_bottom_d
ecile  mean_std
     Deterministic refiner: 8 members (4 folds x 2 flips)          0.524
6                    0.5192                    0.3745
7.6024    0.0196
          Diffusion: 8 members (1 sample per fold x flip)          0.522
8                    0.5336                    0.3413
9.0380    0.0424
Diffusion: 8 members (4 samples averaged per fold x flip)          0.514
1                    0.5199                    0.2930
8.1682    0.0259
    Diffusion: 32 members (4 folds x 2 flips x 4 samples)          0.525
1                    0.5285                    0.2513
9.6314    0.0457
```



```
UNCERTAINTY vs ERROR (test); higher Spearman = spread tracks the real err
or better
                                                    source  spearman_poole
d  spearman_per_patient_mean  spearman_per_patient_min  top_over_bottom_d
ecile  mean_std
     Deterministic refiner: 8 members (4 folds x 2 flips)           0.584
7                     0.5907                    0.4531                1
2.1224    0.0176
          Diffusion: 8 members (1 sample per fold x flip)           0.572
6                     0.5714                    0.4797                1
2.2643    0.0381
Diffusion: 8 members (4 samples averaged per fold x flip)           0.581
1                     0.5835                    0.5234                1
2.7266    0.0229
    Diffusion: 32 members (4 folds x 2 flips x 4 samples)           0.587
7                     0.5915                    0.4879                1
2.3339    0.0409
```



In [48…

```python
# %% ====== [C2] DETERMINISTIC REFINER + ADVERSARIAL LOSS ("Det+GAN"), OP
def train_det_gan(name, cfg, epochs=GAN_EPOCHS, adv_w=GAN_ADV_W):
    """Same refiner as the deterministic baseline (UNet on [mask, coarse
    trained with masked L1 on the residual + 0.05 SSIM + adv_w * LSGAN fr
    assert cfg["mode"] == "det"
    assert "PatchD" in globals() and "_ld" in globals(), "Run baseline ce
    best_p, last_p, done_p = [os.path.join(V4_DIR, f"{name}_{s}") for s i
    bundle = get_bundle(cfg["s1"])
    in_ch = 2 + int(cfg["coarse_ch"]) + (len(bundle.offsets) if cfg["mri_
    G = DiffUNet(in_ch, cfg["base_ch"], T_DIM, bundle.cond_ch, film=cfg["
    if os.path.exists(done_p):
```

```
        G.load_state_dict(_ld(best_p)["ema"]); return G.eval()
    seed_everything(cfg["seed"])
    D = PatchD(len(bundle.offsets) + 1).to(device)
    d_tr, d_va = get_split("train", TRAIN_SNAMES, bundle.offsets), get_sp
    loader = DataLoader(SplitDataset(d_tr, True), batch_size=BATCH_SIZE,
                        drop_last=True, pin_memory=use_amp)
    ema = EMA(G, EMA_DECAY)
    optG = torch.optim.AdamW(G.parameters(), lr=S2_LR, weight_decay=S2_WD
    optD = torch.optim.Adam(D.parameters(), lr=GAN_D_LR, betas=(0.5, 0.99
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
    ssim_crit = DifferentiableSSIM().to(device)
    total, start, best, step = epochs * len(loader), 0, float("inf"), 0
    if os.path.exists(last_p):
        ck = _ld(last_p)
        G.load_state_dict(ck["G"]); D.load_state_dict(ck["D"]); ema.shado
        optG.load_state_dict(ck["optG"]); optD.load_state_dict(ck["optD"]
        start, best = ck["epoch"], ck["best"]; step = start * len(loader)

    def lr_at(s):
        return (1e-6 + (S2_LR - 1e-6) * 0.5 * (1 + math.cos(math.pi * min

    t0 = time.time()
    for ep in range(start + 1, epochs + 1):
        G.train(); D.train(); acc, n = np.zeros(4), 0
        for b in tqdm(loader, desc=f"{name} {ep}/{epochs}", leave=False):
            mri, ct, mask, folds = b["mri"].to(device), b["ct"].to(device
            coarse, feats = bundle.coarse_feats(mri, mask, folds)
            S = bundle.S
            z0 = ((ct - coarse) / S) * mask
            tz = torch.zeros(z0.size(0), dtype=torch.long, device=device)
            for g_ in optG.param_groups:
                g_["lr"] = lr_at(step)
            with torch.amp.autocast("cuda", enabled=use_amp):
                out = G(build_input(cfg, torch.zeros_like(z0), mask, coar
            out = out.float()
            sct = (coarse + out * mask * S).clamp(-1, 1)
            sct_m = torch.where(mask > 0.5, sct, torch.full_like(sct, -1.
            ct_t = torch.where(mask > 0.5, ct, torch.full_like(ct, -1.0))
            optD.zero_grad(set_to_none=True)
            d_real, d_fake = D(torch.cat([mri, ct_t], 1)), D(torch.cat([m
            lossD = 0.5 * (F.mse_loss(d_real, torch.ones_like(d_real)) +
            lossD.backward(); optD.step()
            optG.zero_grad(set_to_none=True)
            l_l1 = ((torch.abs(out - z0) * mask).sum((1, 2, 3)) / (mask.s
            l_ssim = ssim_crit(sct_m, ct_t, mask=mask)
            d_g = D(torch.cat([mri, sct_m], 1))
            l_adv = F.mse_loss(d_g, torch.ones_like(d_g))
            lossG = l_l1 + 0.05 * l_ssim + adv_w * l_adv
            scaler.scale(lossG).backward(); scaler.unscale_(optG)
            nn.utils.clip_grad_norm_(G.parameters(), 1.0)
            scaler.step(optG); scaler.update(); ema.update(G); step += 1
            acc += [lossG.item(), l_l1.item(), l_adv.item(), lossD.item()
        acc /= n
        msg = f"[{name}] ep {ep:3d} G {acc[0]:.4f} L1 {acc[1]:.4f} adv {a
```

```
        if ep % 10 == 0 or ep == epochs:
            m = quick_validate(ema.shadow, cfg, bundle, d_va)
            star = ""
            if m["Score"] < best:
                best, star = m["Score"], "  <-- best"
                torch.save({"ema": ema.shadow.state_dict(), "epoch": ep,
            msg += (f" | EMA val Score {m['Score']:.4f} MAE {m['MAE']:.4f
                    f"PSNR {m['PSNR']:.2f}{star} ({(time.time() - t0) / 6
        print(msg)
        torch.save({"G": G.state_dict(), "D": D.state_dict(), "ema": ema.
                    "optG": optG.state_dict(), "optD": optD.state_dict(),
                    "epoch": ep, "best": best}, last_p)
    open(done_p, "w").write("ok")
    G.load_state_dict(_ld(best_p)["ema"])
    return G.eval()


DETGAN_CFG = make_cfg({"mode": "det", "film": False, "seed": SEED}, BL_S1
DETGAN_LABEL = "Det+GAN refiner, 4 folds + flip-TTA"


def predict_detgan(model, split):
    gt, mask, pid, names = _split_arrays(split)
    b = get_bundle(DETGAN_CFG["s1"])
    st = predict_stack(get_split(split, names, b.offsets), model, DETGAN_
    c, r = aggregate(st)
    return finalize(c, r, mask), st


DETGAN_MODEL = None
if RUN_DETGAN:
    DETGAN_MODEL = train_det_gan("detgan_noFiLM", DETGAN_CFG)
    pred_v, _ = predict_detgan(DETGAN_MODEL, "val")
    gt_v, mk_v, pid_v, _ = _split_arrays("val")
    m_new, _ = patient_level(pred_v, gt_v, mk_v, pid_v)
    print("\nVALIDATION  Det+GAN (4 folds + TTA):", {k: round(v, 4) for k
    if "rows_v" in globals():
        for k, (p, _, _) in rows_v.items():
            if any(s in k for s in ("Deterministic refiner", "Ours: diffu
                m0, _ = patient_level(p, gt_v, mk_v, pid_v)
                print(f"VALIDATION  {k[:60]:<60}", {kk: round(v, 4) for k
        rows_v[DETGAN_LABEL] = (pred_v, 8, None)
    print("Keep Det+GAN only if it beats the plain deterministic refiner
```

In [49…

```
# %% ====== [C3] MODEL-FAMILY BLEND (no training; weights from VALIDATION
def _pick(rows, sub, must):
    c = [k for k in rows if sub in k and must in k]
    return c[-1] if c else None


def simplex(n, step=0.1):
    k = int(round(1 / step))
    if n == 1:
        return [(1.0,)]
```

```
    out = []

    def rec(prefix, left, slots):
        if slots == 1:
            out.append(tuple(prefix + [left / k])); return
        for i in range(left + 1):
            rec(prefix + [i / k], left - i, slots - 1)

    rec([], k, n)
    return out


def blend_families(preds, w):
    out = sum(wi * preds[name] for wi, name in zip(w, preds))
    return np.clip(out, -1, 1)


if RUN_BLEND:
    assert "rows_v" in globals() and "rows_t" in globals(), "Run baseline
    K = {"ours": _pick(rows_v, "Ours: diffusion, 4 folds", "TTA"),
         "det": _pick(rows_v, "Deterministic refiner", "flip-TTA"),
         "pix2pix": _pick(rows_v, "pix2pix", "flip-TTA")}
    K = {k: v for k, v in K.items() if v}
    gt_v, mk_v, pid_v, _ = _split_arrays("val")
    fam_v = {k: rows_v[v][0] for k, v in K.items()}
    use_detgan = INCLUDE_DETGAN_IN_BLEND and DETGAN_MODEL is not None
    if use_detgan:
        fam_v["detgan"] = predict_detgan(DETGAN_MODEL, "val")[0]
    names = list(fam_v)
    print("Families in the blend:", names)

    res = []
    for w in simplex(len(names), 0.1):
        m, _ = patient_level(blend_families(fam_v, w), gt_v, mk_v, pid_v)
        res.append({**{n: x for n, x in zip(names, w)}, "MAE": m["MAE"],
    grid = pd.DataFrame(res).sort_values("Score")
    grid.to_csv(os.path.join(DIR_METRICS, "family_blend_val_grid.csv"), i
    best_w = tuple(float(grid.iloc[0][n]) for n in names)
    print("\nTop 5 validation blends:"); print(grid.head(5).round(4).to_s
    print("Best validation weights:", dict(zip(names, best_w)))

    # ---- single, final test evaluation with the validation-selected wei
    gt_t, mk_t, pid_t, _ = _split_arrays("test")
    fam_t = {k: rows_t[v][0] for k, v in K.items()}
    if use_detgan:
        fam_t["detgan"] = predict_detgan(DETGAN_MODEL, "test")[0]
    eq = tuple(1.0 / len(names) for _ in names)
    cand = {f"{n} alone": tuple(1.0 if m_ == n else 0.0 for m_ in names)
    cand["equal-weight blend"] = eq
    cand["validation-selected blend"] = best_w
    out_rows, test_preds = [], {}
    for label, w in cand.items():
        p = blend_families(fam_t, w); test_preds[label] = p
        m, _ = patient_level(p, gt_t, mk_t, pid_t)
```

```
        out_rows.append({"model": label, "weights": str(tuple(round(x, 2)
                         "SSIM": m["SSIM"], "PSNR": m["PSNR"], "Score": m
    tab = pd.DataFrame(out_rows)
    print("\nTEST (6 held-out patients)"); print(tab.round(4).to_string(i
    tab.to_csv(os.path.join(DIR_METRICS, "family_blend_test.csv"), index=
    for base in ("ours alone", "det alone"):
        if base in test_preds:
            s = paired_stats(test_preds[base], test_preds["validation-sel
            print(f"\n=== validation-selected blend minus [{base}] (per-p
            print(s[s["metric"].isin(["mae", "ssim", "psnr"])].round(4).t
```

```
Families in the blend: ['ours', 'det', 'pix2pix']

Top 5 validation blends:
 ours  det  pix2pix    MAE   SSIM    PSNR  Score
  0.5  0.3      0.2 0.0320 0.8877 25.4740 0.0601
  0.4  0.4      0.2 0.0319 0.8875 25.4740 0.0601
  0.4  0.3      0.3 0.0319 0.8874 25.4647 0.0601
  0.5  0.2      0.3 0.0320 0.8875 25.4539 0.0601
  0.3  0.4      0.3 0.0319 0.8870 25.4558 0.0602
Best validation weights: {'ours': 0.5, 'det': 0.3, 'pix2pix': 0.2}

TEST (6 held-out patients)
                    model            weights    MAE   RMSE   SSIM    PSNR
Score
               ours alone    (1.0, 0.0, 0.0) 0.0259 0.0496 0.9058 26.3580
0.0495
                det alone    (0.0, 1.0, 0.0) 0.0261 0.0501 0.9026 26.2666
0.0504
            pix2pix alone    (0.0, 0.0, 1.0) 0.0260 0.0529 0.8988 25.8035
0.0513
       equal-weight blend (0.33, 0.33, 0.33) 0.0247 0.0481 0.9085 26.6170
0.0476
validation-selected blend    (0.5, 0.3, 0.2) 0.0249 0.0480 0.9087 26.6305
0.0477

=== validation-selected blend minus [ours alone] (per-patient paired,
n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0011  -0.0015  -0.0007      0.0312                  6
6
  ssim          0.0029   0.0019   0.0038      0.0312                  6
6
  psnr          0.2725   0.1819   0.3524      0.0312                  6
6

=== validation-selected blend minus [det alone] (per-patient paired, n=6)
===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  patients_improved
n_patients
   mae         -0.0012  -0.0018  -0.0006      0.0312                  6
6
  ssim          0.0061   0.0052   0.0073      0.0312                  6
6
  psnr          0.3639   0.2709   0.4448      0.0312                  6
6
```

In [51…

```python
from pathlib import Path
import pandas as pd

metrics_dir = Path("/kaggle/working/synthrad_ccr_v3/metrics")

for filename in [
    "family_blend_val_grid.csv",
```

```
    "family_blend_test.csv",
]:
    path = metrics_dir / filename
    print(f"{filename}: {'SAVED' if path.exists() else 'NOT FOUND'}")

test_path = metrics_dir / "family_blend_test.csv"
if test_path.exists():
    display(pd.read_csv(test_path).round(5))
```

```
family_blend_val_grid.csv: SAVED
family_blend_test.csv: SAVED
```

	model	weights	MAE	RMSE	SSIM	PSNR	Score
0	ours alone	(1.0, 0.0, 0.0)	0.02594	0.04960	0.90576	26.35798	0.04950
1	det alone	(0.0, 1.0, 0.0)	0.02607	0.05009	0.90262	26.26658	0.05042
2	pix2pix alone	(0.0, 0.0, 1.0)	0.02602	0.05294	0.89876	25.80353	0.05133
3	equal-weight blend	(0.33, 0.33, 0.33)	0.02471	0.04812	0.90851	26.61698	0.04758
4	validation-selected blend	(0.5, 0.3, 0.2)	0.02487	0.04804	0.90870	26.63049	0.04770

In [ ]:

APPENDED ADD-ON 3: CCR Run Priority Experiments

==========

CCR -- RUNNER FOR THE THREE PRIORITY EXPERIMENTS (staged for Kaggle sessions)

1. diffusion-free blend control (seconds; uses predictions already in memory)
2. matched baselines + extra seeds (hours; resumable)
3. extra disjoint patient-level splits (hours; resumable, one split per call)

RUN ORDER IN THE NOTEBOOK (same session) v3 cells [0]-[5] -> (restore-checkpoints cell + compatibility patch if the session was restarted) -> baseline cells [B0]-[B7] -> paper-suite cells [P0]-[P9] -> THIS FILE [P0]-[P9] only DEFINE the heavy functions; keep RUN["matched_baselines"/"extra_seeds"/"extra_splits"] = False there. Everything heavy is started from this file, one stage per session, with a backup zip after each stage.

MEASURED / ESTIMATED TIMES on the T4 (from your logs; "?" = not measured) final diffusion model, 120 ep ~ 100 min | deterministic refiner, 120 ep ~ 90 min full inference on the 363 test slices (1608 NFE/slice, 4.08 s/slice) ~ 25 min per extra seed ~ 3.5 h (diffusion + deterministic + inference) per extra split (CV_EPOCHS=60) ~ 1.7 h for the two refiners + Stage-1 / ResUNet / pix2pix training (?)

matched baselines (4 x pix2pix + ResUNet, 100 ep each) (?)

The original notebook cells above are preserved. The next compatibility cell handles the `_pick` helper name reused by the previously executed C3 cell; R0–R4 follow from the supplied priority-experiments script.

In [46…

```
# %% ====== [R-COMPAT] RUNNER COMPATIBILITY ============================
# The previous C3 cell defines _pick(rows, sub, must) with a required thi
# argument. The priority runner intentionally calls _pick with 2, 3 or 4
# arguments, as in the paper-suite helper. Restore that compatible signat
# This cell only adds a helper; it does not modify any pre-existing noteb
def _pick(rows, sub, must=None, mustnot=None):
    candidates = [
        key for key in rows
        if sub in key
        and (must is None or must in key)
        and (mustnot is None or mustnot not in key)
    ]
    return candidates[-1] if candidates else None

print("Priority runner compatibility helper installed.")
```

```
Priority runner compatibility helper installed.
```

In [47…

```
# %% ====== [R0] PREFLIGHT + BACKUP HELPER =============================
import os, time, shutil, zipfile
import numpy as np
import pandas as pd

STAGE_FLAGS = dict(control=True,        # 1. diffusion-free blend control
                   matched=False,       # 2a. 4-model pix2pix ensemble +
                   seeds=False,         # 2b. extra seeds of the final di
                   splits=False)        # 3. extra patient-level splits
SEEDS_TO_TRAIN = (43,)                  # seeds to TRAIN in this session
SEEDS_FOR_SUMMARY = (43, 44)            # seeds to include in the summary
SPLIT_INDEXES = (0,)                    # which extra splits to run in th
# CV_SPLITS, CV_EPOCHS, CV_SAMPLES, CV_STEPS, CV_RESUNET_EPOCHS, CV_P2P_E
# e.g. set CV_EPOCHS = 40 here for a faster first pass.

_need = ["run_matched_baselines", "report_matched", "run_extra_seeds", "_
```

```
         "run_split", "make_extra_splits", "restore_original", "cv_summar
         "_arrays", "rows_t", "rows_v", "SUITE_DIR", "FINAL_TAG_", "FINAL
         "CV_SPLITS", "ABLATIONS", "BL_S1", "SEED", "BASE_OUT_DIR", "_pat
_missing = [n for n in _need if n not in globals()]
assert not _missing, f"Run the v3, baseline and paper-suite cells first.

import torch
free_gb = shutil.disk_usage("/kaggle/working" if os.path.exists("/kaggle/
print(f"GPU: {torch.cuda.get_device_name(0) if torch.cuda.is_available()
print("Stages enabled:", [k for k, v in STAGE_FLAGS.items() if v])


def backup(tag, include_latest=False):
    """Zip result tables/figures (+ best checkpoints, optionally resume c
    out = f"/kaggle/working/backup_{tag}.zip" if os.path.exists("/kaggle/
    n = 0
    with zipfile.ZipFile(out, "w", zipfile.ZIP_DEFLATED) as z:
        for root, _, files in os.walk(BASE_OUT_DIR):
            for f in files:
                keep = f.endswith((".csv", ".json", ".png", ".npz")) or f
                keep = keep or (include_latest and f.endswith("_latest.pt
                if keep:
                    z.write(os.path.join(root, f), os.path.relpath(os.pat
    print(f"backup -> {out} ({n} files, {os.path.getsize(out) / 2 ** 20:.
    try:
        from IPython.display import FileLink, display
        display(FileLink(out))
    except Exception:
        pass
    return out
```

```
GPU: Tesla T4 | free disk: 8.3 GB
Stages enabled: ['control']
```

In [48…

```
# %% ====== [R1] DIFFUSION-FREE BLEND CONTROL (equal weights, nothing tun
def blend_control():
    """Does diffusion add anything to a model blend? Compare equal-weight
    results, test_preds = {}, {}
    for split, rows in (("val", rows_v), ("test", rows_t)):
        gt, mk, pid, _ = _arrays(split)
        keys = {"ours": _pick(rows, "Ours: diffusion, 4 folds"), "det": _
                "p2p": _pick(rows, "pix2pix", "flip-TTA", "ensemble"), "r
        P = {n: rows[l][0] for n, l in keys.items() if l}
        combos = {"ours alone": ["ours"], "det. refiner alone": ["det"],
                  "det + pix2pix  [no diffusion]": ["det", "p2p"], "ours
                  "ours + det": ["ours", "det"], "ours + det + pix2pix":
                  "det + pix2pix + ResUNet  [no diffusion]": ["det", "p2p
                  "ours + det + pix2pix + ResUNet": ["ours", "det", "p2p"
        preds = {n: np.clip(np.mean([P[m] for m in mem], 0), -1, 1) for n
        lines = []
        for n, p in preds.items():
            m, _ = patient_level(p, gt, mk, pid)
            lines.append({"blend (equal weights)": n, "MAE": m["MAE"], "R
                          "Score": m["Score"], "HU_MAE": m["MAE"] * HU_SP
        df = pd.DataFrame(lines); results[split] = df; test_preds[split]
```

```
        print(f"\nBLEND CONTROL ({split}); HU_MAE uses the {'verified' if
        print(df.round(4).to_string(index=False))
        df.to_csv(os.path.join(SUITE_DIR, f"blend_control_{split}.csv"),
    preds, gt, mk, pid = test_preds["test"]
    pairs = [("ours + det + pix2pix", "det + pix2pix  [no diffusion]"),
             ("ours + det + pix2pix + ResUNet", "det + pix2pix + ResUNet
    for with_d, without_d in pairs:
        if with_d in preds and without_d in preds:
            s = paired_full(preds[without_d], preds[with_d], gt, mk, pid,
            print(f"\n=== TEST: [{with_d}] minus [{without_d}]  (what dif
            print(s[s["metric"].isin(["mae", "ssim", "psnr"])].round(4).t
    return results


if STAGE_FLAGS["control"]:
    BLEND_CONTROL = blend_control()
```

```
BLEND CONTROL (val); HU_MAE uses the verified window
                  blend (equal weights)    MAE   RMSE   SSIM    PSNR  Sco
re  HU_MAE
                              ours alone 0.0332 0.0568 0.8850 25.2262 0.06
20 83.5734
                     det. refiner alone 0.0329 0.0574 0.8811 25.1173 0.06
26 82.7991
                           pix2pix alone 0.0339 0.0607 0.8743 24.6417 0.06
53 85.1689
          det + pix2pix  [no diffusion] 0.0323 0.0564 0.8837 25.2767 0.06
14 81.2597
                         ours + pix2pix 0.0323 0.0563 0.8855 25.3094 0.06
09 81.2922
                              ours + det 0.0322 0.0556 0.8866 25.4074 0.06
06 81.0890
                   ours + det + pix2pix 0.0319 0.0554 0.8871 25.4523 0.06
01 80.2943
det + pix2pix + ResUNet  [no diffusion] 0.0328 0.0569 0.8819 25.1947 0.06
23 82.4223
         ours + det + pix2pix + ResUNet 0.0323 0.0559 0.8850 25.3540 0.06
11 81.3029

BLEND CONTROL (test); HU_MAE uses the verified window
                  blend (equal weights)    MAE   RMSE   SSIM    PSNR  Sco
re  HU_MAE
                              ours alone 0.0259 0.0496 0.9058 26.3580 0.04
95 65.2568
                     det. refiner alone 0.0261 0.0501 0.9026 26.2666 0.05
04 65.5936
                           pix2pix alone 0.0260 0.0529 0.8988 25.8035 0.05
13 65.4539
          det + pix2pix  [no diffusion] 0.0249 0.0490 0.9061 26.4594 0.04
84 62.6144
                         ours + pix2pix 0.0250 0.0491 0.9074 26.4525 0.04
81 62.7892
                              ours + det 0.0253 0.0484 0.9073 26.5638 0.04
85 63.6048
                   ours + det + pix2pix 0.0247 0.0481 0.9085 26.6170 0.04
76 62.1649
det + pix2pix + ResUNet  [no diffusion] 0.0256 0.0499 0.9038 26.2904 0.04
97 64.4644
         ours + det + pix2pix + ResUNet 0.0253 0.0490 0.9063 26.4560 0.04
87 63.6152

=== TEST: [ours + det + pix2pix] minus [det + pix2pix  [no diffusion]]
(what diffusion adds to the blend; n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  cohens_dz  patient
s_improved  n
   mae         -0.0002  -0.0005   0.0001      0.2188    -0.4521
5  6
  ssim          0.0024   0.0014   0.0035      0.0312     1.6840
6  6
  psnr          0.1576   0.0861   0.2462      0.0312     1.4311
6  6
```

```
=== TEST: [ours + det + pix2pix + ResUNet] minus [det + pix2pix + ResUNet
[no diffusion]]  (what diffusion adds to the blend; n=6) ===
metric  mean_diff(b-a)  CI95_lo  CI95_hi  wilcoxon_p  cohens_dz  patient
s_improved  n
   mae         -0.0003  -0.0005  -0.0001      0.0625    -1.2716
5  6
  ssim          0.0025   0.0019   0.0031      0.0312     2.8340
6  6
  psnr          0.1656   0.1244   0.2238      0.0312     2.3613
6  6
```

In [49…

```python
# %% ====== [R2] MATCHED BASELINES (4-model pix2pix ensemble + ResUNet tr
# Stage-wise: run alone in one session. Resumable: re-running continues f
if STAGE_FLAGS["matched"]:
    t0 = time.time()
    MATCHED = run_matched_baselines()
    report_matched(MATCHED)
    print(f"matched baselines done in {(time.time() - t0) / 60:.0f} min")
    backup("matched")
```

In [50…

```python
# %% ====== [R3] EXTRA SEEDS (final diffusion model + deterministic refin
# Predictions are cached per seed so that a restart never repeats the 25-
if "_seed_predictions_orig" not in globals():                    # guard: n
    _seed_predictions_orig = _seed_predictions


def _seed_predictions(sd, split):
    f = os.path.join(SUITE_DIR, f"seedpred_{sd}_{split}.npz")
    if os.path.exists(f):
        z = np.load(f); return z["pf"], z["pd"]
    pf, pd_ = _seed_predictions_orig(sd, split)
    np.savez_compressed(f, pf=pf, pd=pd_)
    return pf, pd_


def seed_names(sd):
    return f"{FINAL_TAG_}_seed{sd}", f"det_noFiLM_s{sd}"


def seed_ready(sd):
    return all(os.path.exists(_paths(n, FINAL_EPOCHS)[1]) for n in seed_n


def train_seed_models(sd):
    cfg_f = make_cfg({**ABLATIONS[FINAL_NAME_], "seed": sd}, BL_S1)
    cfg_d = make_cfg({"mode": "det", "film": False, "seed": sd}, BL_S1)
    n_f, n_d = seed_names(sd)
    get_stage2(n_f, cfg_f, FINAL_EPOCHS)                          # ~100
    get_stage2(n_d, cfg_d, FINAL_EPOCHS)                          # ~90 m


if STAGE_FLAGS["seeds"]:
    for sd in SEEDS_TO_TRAIN:
        t0 = time.time(); train_seed_models(sd)
```

```
        print(f"seed {sd}: models ready in {(time.time() - t0) / 60:.0f}
    EXTRA_SEEDS = tuple(sd for sd in SEEDS_FOR_SUMMARY if seed_ready(sd))
    print("Seeds finished so far:", EXTRA_SEEDS)
    if EXTRA_SEEDS:
        SEEDS_TEST = run_extra_seeds("test")                    # mean
    backup("seeds")
```

In [51…

```
# %% ====== [R4] EXTRA DISJOINT PATIENT-LEVEL SPLITS (one or more per ses
def collect_split_results():
    tags = [t for t, _, _, _ in make_extra_splits(CV_SPLITS)]
    frames = [pd.read_csv(os.path.join(SUITE_DIR, f"cv_{t}_per_patient.cs
              if os.path.exists(os.path.join(SUITE_DIR, f"cv_{t}_per_pati
    return pd.concat(frames, ignore_index=True) if frames else None


if STAGE_FLAGS["splits"]:
    all_splits = make_extra_splits(CV_SPLITS)
    for i in SPLIT_INDEXES:
        t0 = time.time()
        try:
            run_split(*all_splits[i])                           # trai
        finally:
            restore_original()                                  # alwa
        print(f"split {all_splits[i][0]} done in {(time.time() - t0) / 60
        backup(f"split_{all_splits[i][0]}")
    CV_ALL = collect_split_results()
    if CV_ALL is not None:
        print(f"Finished splits so far: {sorted(CV_ALL['split'].unique())
        CV_STATS = cv_summary(CV_ALL)
```

In [ ]: